

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**

o0o



**ĐỒ ÁN TỐT NGHIỆP KHOA
CÔNG NGHỆ THÔNG TIN**

**Tên đề tài: Xây dựng hệ thống quản lý bán hàng và
sửa chữa điện thoại trên nền tảng Web và Mobile**

Đặng Tiến Dũng

MSSV: B21DCCN264

Nguyễn Trọng Đức

MSSV: B21DCCN252

Tổng Quang Nam

MSSV: B21DCCN556

Nguyễn Ngọc Quang Huy

MSSV: B21DCCN434

Giảng viên hướng dẫn: Ths. Nguyễn Hoàng Anh

HÀ NỘI, 12/2025

[illegible]

Hà Nội, ngày tháng năm 20...

Giảng viên hướng dẫn

[illegible]

Hà Nội, ngày tháng năm 20...

Giảng viên phản biện

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin phép gửi lời cảm ơn chân thành nhất tới các thầy cô khoa Công nghệ thông tin 1, cùng toàn thể các thầy cô tại Học viện Công nghệ Bưu chính Viễn thông, đã dành sự quan tâm, chỉ dạy, truyền đạt kiến thức cho chúng em trong suốt hơn 5 năm học đại học vừa qua.

Đặc biệt, chúng em xin gửi lời cảm ơn sâu sắc nhất tới thầy Nguyễn Hoàng Anh, giảng viên hướng dẫn nhóm để thực hiện đồ án này. Với sự giúp đỡ tận tình của thầy, nhóm em đã hoàn thành được đồ án tốt nghiệp. Xin chân thành cảm ơn thầy!

Cuối cùng, nhóm xin chân thành cảm ơn gia đình và bạn bè, đã luôn tạo điều kiện, quan tâm, giúp đỡ, động viên chúng em trong suốt quá trình học tập và hoàn thành đồ án tốt nghiệp.

Với điều kiện thời gian cũng như kinh nghiệm còn hạn chế của một nhóm các sinh viên, đồ án này không thể tránh được những thiếu sót. Nhóm em rất mong nhận được sự chỉ bảo, đóng góp ý kiến của các thầy cô để chúng em có điều kiện bổ sung, nâng cao kiến thức của mình, phục vụ tốt hơn công tác thực tế sau này.

Nhóm em xin chân thành cảm ơn!

BẢNG PHÂN CÔNG CÔNG VIỆC THÀNH VIÊN

STT	Họ và tên	MSV	Nội dung thực hiện
1	Đặng Tiến Dũng	B21DCCN264	- Thiết kế, xây dựng nghiệp vụ của hệ thống. - Xây dựng cơ sở dữ liệu. - Xây dựng Backend (Spring Boot). - Thiết lập Docker, Server, Deploy. - Tổng hợp, chỉnh sửa báo cáo.
2	Nguyễn Trọng Đức	B21DCCN252	- Phân tích nghiệp vụ dự án. - Thiết kế giao diện Website (UI/UX). - Xây dựng Frontend Web (VueJS). - Thiết kế sơ đồ tổng quan usecase. - Thiết kế các Use case của nhân viên và luồng "Thông báo".
3	Tổng Quang Nam	B21DCCN556	- Xây dựng core Mobile App. - Xây dựng chức năng Notification (FCM). - Xây dựng Backend của dự án. - Tích hợp nền tảng lưu trữ đám mây (Cloudinary). - Thiết lập Docker, Server, Deploy.
4	Nguyễn Ngọc Quang Huy	B21DCCN434	- Xây dựng Frontend Mobile App (Kotlin + Jetpack Compose). - Hỗ trợ chỉnh sửa Backend. - Soạn thảo Slide thuyết trình. - Viết báo cáo chính của đề án. - Tích hợp thanh toán ZaloPay.

Nhận xét của Giảng viên hướng dẫn về mức độ hoàn thành:

.....
.....

Hà Nội, ngày tháng năm 20...

Giảng viên hướng dẫn

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Giới thiệu chương	1
1.2 Lý do chọn đề tài	1
1.3 Mục tiêu của đề tài	2
1.3.1 Mục tiêu tổng quát	2
1.3.2 Mục tiêu cụ thể	3
a Mục tiêu về quản lý sản phẩm	3
b Mục tiêu về quản lý đơn hàng và giỏ hàng	3
c Mục tiêu về thanh toán	3
d Mục tiêu về xác thực và phân quyền	4
e Mục tiêu về thông báo	4
f Mục tiêu về quản lý nhân viên và lịch làm việc	4
g Mục tiêu về công nghệ và kiến trúc	4
1.4 Đối tượng và phạm vi nghiên cứu.	5
1.4.1 Đối tượng nghiên cứu	5
a Đối tượng nghiệp vụ	5
b Đối tượng người dùng	5
1.4.2 Phạm vi nghiên cứu	6
a Phạm vi chức năng	6
b Phạm vi công nghệ	7
c Phạm vi giới hạn	7
1.5 Cấu trúc đồ án	7
1.6 Kết luận chương	10
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ	11
2.1 Giới thiệu chương	11
2.2 Tổng quan về kiến trúc hệ thống	11
2.2.1 Mô hình Client – Server	11
2.2.2 Kiến trúc RESTful API	12
2.2.3 Khả năng đáp ứng đồng thời MVVM (Mobile) và MVC (Web).	13
a Luồng tích hợp với Mobile (MVVM)	14
b Luồng tích hợp với Web (MVC)	14

2.3	Các công nghệ sử dụng	15
2.3.1	Backend: Java Spring Boot	15
a	Giới thiệu về Java và Spring Boot	15
b	Lý do lựa chọn Java Spring Boot	15
c	Công nghệ ORM: Hibernate/JPA	16
d	Các module Spring Boot được sử dụng	17
2.3.2	Frontend Web: Vue.js	17
a	Giới thiệu về Vue.js	17
b	Ứng dụng trong Admin Dashboard	18
c	Tích hợp với Backend API	18
2.3.3	Mobile App: Android với Kotlin và Jetpack Components	19
a	Giới thiệu về Kotlin	19
b	Android Jetpack Components	19
c	Ứng dụng trong Mobile App	20
2.3.4	Hệ quản trị cơ sở dữ liệu (DBMS): MySQL.	21
a	Giới thiệu về MySQL	21
b	Lý do chọn MySQL	21
c	Cấu trúc Database trong dự án	21
2.3.5	Các công nghệ hỗ trợ khác	22
a	Firebase Cloud Messaging (FCM)	22
b	ZaloPay - Cổng thanh toán điện tử	22
c	JWT (JSON Web Token) - Cơ chế xác thực	23
d	Cloudinary - Nền tảng lưu trữ đám mây	24
e	RabbitMQ - Message Broker	24
f	MapStruct - Object Mapping	25
g	Lombok - Code Generation	26
h	Docker và Docker Compose - Containerization	26
i	Maven - Build Tool và Dependency Management	27
2.4	Kết luận chương	28
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG		29
3.1	Giới thiệu chương	29
3.2	Phân tích yêu cầu hệ thống	29
3.2.1	Xác định các tác nhân (Actors)	29
3.2.2	Yêu cầu phi chức năng.	30

3.3 Phân tích và thiết kế Use Case	31
3.3.1 Sơ đồ Use Case tổng quát	31
3.3.2 Chi tiết các nhóm chức năng	32
a Nhóm chức năng dành cho Khách hàng	32
b Nhóm chức năng Quản lý (Admin/Staff)	32
3.3.3 Đặc tả các Use Case chính	33
a Tác nhân: Khách hàng (Customer/User)	33
b Tác nhân: Admin (Quản trị viên)	53
c Tác nhân: Nhân viên bán hàng (Sales Staff)	55
d Tác nhân: Kỹ thuật viên (Technician)	59
3.4 Thiết kế Cơ sở dữ liệu	60
3.4.1 Mô hình thực thể liên kết (ERD)	60
a Identity Module (Quản lý danh tính)	60
b Product Module (Quản lý sản phẩm)	60
c Cart & Order Module (Giỏ hàng và Đơn hàng)	61
d Work Schedule Module (Lịch làm việc)	61
3.4.2 Thiết kế danh sách các bảng	62
a Khái quát các nhóm bảng chính	62
b Khái quát quan hệ giữa các bảng	63
3.5 Thiết kế hệ thống chi tiết	64
3.5.1 Biểu đồ trình tự (Sequence Diagram)	64
a Tác nhân A: Khách hàng (Customer)	64
b Tác nhân B: Quản trị viên (Admin)	80
c Tác nhân C: Nhân viên bán hàng (Sales Staff)	82
d Tác nhân D: Nhân viên sửa chữa (Technician)	85
3.5.2 Biểu đồ lớp (Class Diagram)	87
a Mục tiêu và phạm vi	87
b Tổng quan kiến trúc domain theo package	87
c Phân tích quan hệ UML chính trong sơ đồ	92
d Luồng nghiệp vụ được thể hiện qua Class Diagram	92
e Kết luận	92
3.6 Kết luận chương	93
CHƯƠNG 4. TRIỂN KHAI VÀ ĐÁNH GIÁ	94
4.1 Giới thiệu chương	94
4.2 IDE và Công cụ phát triển	94
4.2.1 Visual Studio Code	94

4.2.2	Android Studio	97
4.2.3	Docker	98
4.3	Cài đặt hệ thống chi tiết	98
4.3.1	Backend (Java Spring Boot)	98
4.3.2	Web UI (Vue.js)	98
4.3.3	Android App (Native)	98
4.4	Triển khai hệ thống và Deploy	98
4.5	Giao diện chương trình (Demo).	99
4.5.1	Giao diện Web (Khách hàng)	99
a	Trang chủ và Sản phẩm	99
b	Chức năng Giỏ hàng và Đặt hàng	100
c	Chức năng Đặt lịch sửa chữa	100
d	Quản lý tài khoản và Đơn hàng	101
4.5.2	Giao diện Web (Quản trị viên)	102
a	Thống kê (Dashboard)	103
b	Quản lý Hàng hóa	104
c	Quản lý Đơn hàng	105
d	Quản lý Người dùng	106
4.5.3	Giao diện Mobile App (Khách hàng)	107
4.5.4	Đánh giá kết quả	110
4.6	Kết luận chương	110
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		112
TÀI LIỆU THAM KHẢO		115

DANH MỤC HÌNH VẼ

Hình 3.1	Sơ đồ kiến trúc Use Case tổng quát	31
Hình 3.2	Sơ đồ Use Case phân hệ Khách hàng	32
Hình 3.3	Sơ đồ Use Case phân hệ Quản trị viên	33
Hình 3.4	Sơ đồ Use Case - Đăng ký tài khoản	34
Hình 3.5	Sơ đồ Use Case - Đăng nhập	36
Hình 3.6	Sơ đồ Use Case - Đăng xuất	37
Hình 3.7	Sơ đồ Use Case - Xem thông tin cá nhân	38
Hình 3.8	Sơ đồ Use Case - Xem danh sách sản phẩm	39
Hình 3.9	Sơ đồ Use Case - Cập nhật thông tin cá nhân	40
Hình 3.10	Sơ đồ Use Case - Xem chi tiết sản phẩm	41
Hình 3.11	Sơ đồ Use Case - Xem danh mục sản phẩm	42
Hình 3.12	Sơ đồ Use Case - Xem giỏ hàng	43
Hình 3.13	Sơ đồ Use Case - Thêm sản phẩm vào giỏ hàng	44
Hình 3.14	Sơ đồ Use Case - Cập nhật số lượng sản phẩm	45
Hình 3.15	Sơ đồ Use Case - Xóa sản phẩm khỏi giỏ hàng	46
Hình 3.16	Sơ đồ Use Case - Đặt hàng	47
Hình 3.17	Sơ đồ Use Case - Thanh toán online	48
Hình 3.18	Sơ đồ Use Case - Xem lịch sử đơn hàng	49
Hình 3.19	Sơ đồ Use Case - Xem chi tiết đơn hàng	50
Hình 3.20	Sơ đồ Use Case - Xem tình trạng đơn hàng	51
Hình 3.21	Sơ đồ Use Case - Quản lý thông báo cá nhân	52
Hình 3.22	Sơ đồ Use Case - Xem thống kê	53
Hình 3.23	Sơ đồ Use Case - Quản lý người dùng và phân quyền	54
Hình 3.24	Sơ đồ Use Case - Quản lý đơn hàng	55
Hình 3.25	Sơ đồ Use Case - Quản lý sản phẩm	56
Hình 3.26	Sơ đồ Use Case - Thanh toán tại quầy	57
Hình 3.27	Sơ đồ Use Case - Xuất hóa đơn	58
Hình 3.28	Sơ đồ Use Case - Xem đơn sửa chữa phụ trách	59
Hình 3.29	Sơ đồ cơ sở dữ liệu	62
Hình 3.30	Sequence Diagram - Đăng ký tài khoản	64
Hình 3.31	Sequence Diagram - Đăng nhập	65
Hình 3.32	Sequence Diagram - Đăng xuất	66
Hình 3.33	Sequence Diagram - Xem thông tin cá nhân	67
Hình 3.34	Sequence Diagram - Xem danh sách sản phẩm	68
Hình 3.35	Sequence Diagram - Cập nhật thông tin cá nhân	69

Hình 3.36	Sequence Diagram - Xem chi tiết sản phẩm	70
Hình 3.37	Sequence Diagram - Xem danh mục sản phẩm	71
Hình 3.38	Sequence Diagram - Xem giỏ hàng	72
Hình 3.39	Sequence Diagram - Thêm sản phẩm vào giỏ	73
Hình 3.40	Sequence Diagram - Cập nhật số lượng giỏ hàng	74
Hình 3.41	Sequence Diagram - Xóa sản phẩm khỏi giỏ hàng	75
Hình 3.42	Sequence Diagram - Đặt hàng	76
Hình 3.43	Sequence Diagram - Thanh toán Online (ZaloPay)	77
Hình 3.44	Sequence Diagram - Xem lịch sử đơn hàng	78
Hình 3.45	Sequence Diagram - Xem chi tiết đơn hàng	79
Hình 3.46	Sequence Diagram - Xem trạng thái đơn hàng	79
Hình 3.47	Sequence Diagram - Xem thống kê	80
Hình 3.48	Sequence Diagram - Quản lý người dùng và phân quyền . . .	81
Hình 3.49	Sequence Diagram - Cập nhật trạng thái đơn hàng	82
Hình 3.50	Sequence Diagram - Cập nhật sản phẩm trong đơn sửa chữa .	84
Hình 3.51	Sequence Diagram - Xem chi tiết đơn sửa chữa phụ trách . .	85
Hình 3.52	Class Diagram - Biểu đồ lớp (Domain Model)	87
Hình 3.53	Class Diagram - Biểu đồ lớp về sản phẩm (Domain Model) .	88
Hình 3.54	Class Diagram - Biểu đồ lớp về giỏ và đơn hàng (Domain Model)	89
Hình 3.55	Class Diagram - Biểu đồ lớp về xác thực và phân quyền (Domain Model)	90
Hình 3.56	Class Diagram - Biểu đồ lớp còn lại (Domain Model)	91
Hình 4.1	Trang chủ Visual Studio Code	95
Hình 4.2	Giao diện cài đặt Extensions trên Visual Studio Code	96
Hình 4.3	Giao diện Android Studio	97
Hình 4.4	Trang chủ Android Studio	97
Hình 4.5	Giao diện Trang chủ với danh sách sản phẩm và banner khuyến mãi	99
Hình 4.6	Giao diện Chi tiết sản phẩm: Xem thông tin, chọn phiên bản/màu sắc	99
Hình 4.7	Giao diện Giỏ hàng: Xem danh sách sản phẩm và nhập thông tin đặt hàng	100
Hình 4.8	Form Đặt lịch sửa chữa: Nhập thông tin thiết bị và mô tả lỗi .	100
Hình 4.9	Giao diện Cập nhật thông tin cá nhân	101
Hình 4.10	Danh sách đơn hàng cá nhân	101
Hình 4.11	Chi tiết đơn hàng (Giao diện người dùng)	102

Hình 4.12	Hệ thống thông báo trạng thái đơn hàng	102
Hình 4.13	Thống kê tổng quan: Doanh thu, đơn hàng, tỷ lệ dịch vụ . . .	103
Hình 4.14	Thống kê sản phẩm bán chạy và dịch vụ sửa chữa phổ biến .	103
Hình 4.15	Danh sách hàng hóa trong kho	104
Hình 4.16	Xem chi tiết thông tin hàng hóa	104
Hình 4.17	Giao diện Thêm mới / Cập nhật hàng hóa	105
Hình 4.18	Danh sách tất cả đơn hàng (Mua bán và Sửa chữa)	105
Hình 4.19	Chi tiết đơn hàng và cập nhật trạng thái đơn hàng	106
Hình 4.20	Danh sách Khách hàng và Nhân viên	106
Hình 4.21	Cập nhật thông tin và phân quyền người dùng	107
Hình 4.22	Giao diện điều hướng và tìm kiếm sản phẩm	107
Hình 4.23	Chức năng xem chi tiết và quản lý giỏ hàng	108
Hình 4.24	Luồng thanh toán trực tuyến qua ví điện tử	108
Hình 4.25	Các chức năng quản lý tài khoản người dùng	109
Hình 4.26	Quản lý đơn hàng và Dịch vụ sửa chữa	109

DANH MỤC BẢNG BIỂU

Bảng 3.1	Kịch bản Use Case Đăng ký tài khoản	35
Bảng 3.2	Kịch bản Use Case Đăng nhập	36
Bảng 3.3	Kịch bản Use Case Đăng xuất	37
Bảng 3.4	Kịch bản Use Case Xem thông tin cá nhân	38
Bảng 3.5	Kịch bản Use Case Xem danh sách sản phẩm	39
Bảng 3.6	Kịch bản Use Case Cập nhật thông tin cá nhân	40
Bảng 3.7	Kịch bản Use Case Xem chi tiết sản phẩm	42
Bảng 3.8	Kịch bản Use Case Xem danh mục sản phẩm	43
Bảng 3.9	Kịch bản Use Case Xem giỏ hàng	44
Bảng 3.10	Kịch bản Use Case Thêm sản phẩm vào giỏ hàng	45
Bảng 3.11	Kịch bản Use Case Cập nhật số lượng sản phẩm	46
Bảng 3.12	Kịch bản Use Case Xóa sản phẩm khỏi giỏ hàng	47
Bảng 3.13	Kịch bản Use Case Đặt hàng	48
Bảng 3.14	Kịch bản Use Case Thanh toán online	49
Bảng 3.15	Kịch bản Use Case Xem lịch sử đơn hàng	50
Bảng 3.16	Kịch bản Use Case Xem chi tiết đơn hàng	51
Bảng 3.17	Kịch bản Use Case Xem tình trạng đơn hàng	52
Bảng 3.18	Kịch bản Use Case Quản lý thông báo cá nhân	53
Bảng 3.19	Kịch bản Use Case Xem thống kê	54
Bảng 3.20	Kịch bản Use Case Quản lý người dùng	55
Bảng 3.21	Kịch bản Use Case Quản lý đơn hàng	56
Bảng 3.22	Kịch bản Use Case Quản lý sản phẩm	57
Bảng 3.23	Kịch bản Use Case Thanh toán tại quầy	58
Bảng 3.24	Kịch bản Use Case Xuất hóa đơn	59
Bảng 3.25	Kịch bản Use Case Xem đơn sửa chữa phụ trách	60

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
CSS	Ngôn ngữ định dạng phần tử (Cascading Style Sheets)
DOM	Mô hình đối tượng tài liệu (Document Object Model)
ERD	Sơ đồ thực thể liên kết (Entity Relationship Diagram)
EUD	Phát triển ứng dụng người dùng cuối (End-User Development)
GWT	Bộ công cụ Web của Google (Google Web Toolkit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
HTTP	Giao thức truyền tải siêu văn bản (HyperText Transfer Protocol)
IaaS	Dịch vụ cơ sở hạ tầng (Infrastructure as a Service)
IDE	Môi trường phát triển tích hợp (Integrated Development Environment)
JSON	Ký hiệu đối tượng JavaScript (JavaScript Object Notation)
JWT	Mã thông báo web JSON (JSON Web Token)
MVC	Mô hình Model-View-Controller
REST	Kiến trúc chuyển giao trạng thái đại diện (Representational State Transfer)
SQL	Ngôn ngữ truy vấn có cấu trúc (Structured Query Language)
UI	Giao diện người dùng (User Interface)
UX	Trải nghiệm người dùng (User Experience)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Giới thiệu chương

Chương 1 đóng vai trò là nền tảng định hướng cho toàn bộ đề án, tập trung giải quyết vấn đề xác định "bài toán" và "phạm vi" của hệ thống quản lý bán hàng và sửa chữa điện thoại. Trước khi đi sâu vào các giải pháp kỹ thuật chi tiết, chương này sẽ phân tích bối cảnh thực tế, chỉ ra những bất cập trong phương pháp quản lý thủ công hiện tại.

Nội dung chương sẽ đi từ việc làm rõ lý do chọn đề tài, xác định các mục tiêu cụ thể cần đạt được, đến việc khoanh vùng đối tượng và phạm vi nghiên cứu. Đây là cơ sở quan trọng để xây dựng các chức năng nghiệp vụ ở các chương sau.

1.2 Lý do chọn đề tài

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của công nghệ thông tin và thương mại điện tử, nhu cầu ứng dụng các hệ thống phần mềm vào hoạt động quản lý kinh doanh ngày càng trở nên phổ biến. Đối với các cửa hàng bán và sửa chữa điện thoại – một lĩnh vực có tính cạnh tranh cao và khối lượng nghiệp vụ lớn – việc quản lý hiệu quả các hoạt động như bán hàng, sửa chữa, kho hàng và chăm sóc khách hàng đóng vai trò vô cùng quan trọng.

Tuy nhiên, trên thực tế, nhiều cửa hàng hiện nay vẫn đang sử dụng các phương pháp quản lý thủ công hoặc các phần mềm đơn lẻ, thiếu sự liên kết và đồng bộ giữa các nghiệp vụ. Việc quản lý đơn hàng, thông tin khách hàng, tồn kho sản phẩm và các đơn sửa chữa thường được thực hiện rời rạc, dẫn đến nhiều hạn chế như:

- **Khó theo dõi trạng thái và tiến độ xử lý đơn hàng:** Khách hàng không thể tự theo dõi trạng thái đơn hàng của mình một cách real-time, phải liên hệ trực tiếp với cửa hàng để cập nhật thông tin, gây mất thời gian và không thuận tiện.
- **Sai sót trong quản lý số lượng sản phẩm tồn kho:** Việc quản lý tồn kho thủ công dễ dẫn đến sai sót trong việc cập nhật số lượng sản phẩm, đặc biệt là với các sản phẩm có nhiều biến thể (variant) như màu sắc, dung lượng, kích thước. Điều này có thể dẫn đến tình trạng hết hàng không được phát hiện kịp thời hoặc nhầm lẫn trong việc quản lý các thuộc tính sản phẩm.
- **Chậm trễ trong việc xử lý sửa chữa và thanh toán:** Quy trình từ tiếp nhận thiết bị sửa chữa, chẩn đoán, sửa chữa, đến thông báo cho khách hàng và thanh toán thường mất nhiều thời gian do phải thực hiện thủ công. Khách hàng không được thông báo kịp thời về tiến độ sửa chữa, dẫn đến trải nghiệm không tốt.

- **Gặp nhiều khó khăn trong việc tổng hợp số liệu, thống kê và lập báo cáo kinh doanh:** Việc tổng hợp dữ liệu từ nhiều nguồn khác nhau (sổ sách, Excel, các phần mềm riêng lẻ) mất nhiều thời gian và dễ sai sót. Cửa hàng khó có được cái nhìn tổng quan về tình hình kinh doanh, sản phẩm bán chạy, hoặc hiệu quả của các chiến dịch marketing.
- **Thiếu tính minh bạch trong quy trình thanh toán:** Khách hàng không thể theo dõi được quá trình thanh toán, không có lịch sử giao dịch rõ ràng, và phải đến cửa hàng để thanh toán, không có nhiều lựa chọn thanh toán trực tuyến.
- **Khó khăn trong việc quản lý nhân viên và phân công công việc:** Việc phân công ca làm việc, quản lý lịch làm việc của nhân viên, và theo dõi tiến độ công việc thường được thực hiện thủ công, dễ dẫn đến xung đột lịch hoặc phân công không hợp lý.
- **Thiếu kênh thông báo hiệu quả:** Cửa hàng khó có thể thông báo cho khách hàng về các sự kiện, khuyến mãi, hoặc trạng thái đơn hàng một cách nhanh chóng và đồng loạt. Việc gọi điện hoặc nhắn tin thủ công tốn nhiều thời gian và chi phí.

Xuất phát từ những thách thức thực tế trên, đề tài được lựa chọn nhằm xây dựng một hệ thống quản lý bán hàng và sửa chữa điện thoại tích hợp, hiện đại, giải quyết các vấn đề nêu trên thông qua việc ứng dụng các công nghệ web và mobile hiện đại, đảm bảo tính tự động hóa, minh bạch và hiệu quả trong quản lý.

1.3 Mục tiêu của đề tài

Từ những vấn đề thực tiễn nêu trên, đề tài đặt ra mục tiêu xây dựng một hệ thống quản lý bán hàng và sửa chữa điện thoại toàn diện, đáp ứng các yêu cầu sau:

1.3.1 Mục tiêu tổng quát

- **Quản lý thống nhất và tập trung:** Đồng bộ hóa tất cả các nghiệp vụ bán hàng và sửa chữa trên một hệ thống duy nhất, giúp dữ liệu được cập nhật tức thời và chính xác. Hệ thống quản lý tập trung các module như quản lý sản phẩm, đơn hàng, khách hàng, thanh toán, thông báo, và lịch làm việc trên cùng một nền tảng backend, đảm bảo tính nhất quán và giảm thiểu rủi ro về đồng bộ hóa dữ liệu.
- **Đa nền tảng và đa kênh truy cập:** Hỗ trợ sử dụng trên cả nền tảng Web (cho quản lý, nhân viên) và Mobile (cho khách hàng), đảm bảo sự tiện lợi và linh hoạt. Hệ thống được xây dựng với kiến trúc RESTful API, cho phép nhiều client khác nhau (web browser, mobile app) cùng truy cập vào một backend duy nhất, đảm bảo dữ liệu luôn đồng bộ giữa các nền tảng.

- **Tối ưu quy trình và tự động hóa:** Đảm bảo tính chính xác, nhanh chóng trong các khâu tiếp nhận, xử lý và thanh toán, mang lại sự thuận tiện cho nhân viên cũng như nâng cao trải nghiệm khách hàng. Hệ thống tự động hóa các quy trình như cập nhật trạng thái đơn hàng sau thanh toán, gửi thông báo đến khách hàng, và quản lý giỏ hàng.
- **Khả năng mở rộng và bảo trì:** Hệ thống được thiết kế theo kiến trúc hiện đại với Spring Boot, **áp dụng các design patterns và best practices**, đáp ứng yêu cầu về khả năng mở rộng, bảo trì và phát triển thêm các tính năng mới trong tương lai mà không ảnh hưởng đến các module hiện có.

1.3.2 Mục tiêu cụ thể

Dựa trên các tính năng đã được triển khai trong hệ thống, các mục tiêu cụ thể bao gồm:

a, Mục tiêu về quản lý sản phẩm

- Xây dựng hệ thống quản lý sản phẩm với cấu trúc phân cấp linh hoạt (Product → ProductVariant → ProductAttribute), cho phép quản lý các biến thể sản phẩm phức tạp với nhiều thuộc tính khác nhau như màu sắc, dung lượng, kích thước, giá cả, và số lượng tồn kho.
- Hỗ trợ quản lý danh mục sản phẩm (Category) với khả năng phân cấp và quản lý hình ảnh sản phẩm thông qua Cloudinary, đảm bảo hiệu suất tốt với CDN.
- Cho phép admin quản lý toàn bộ vòng đời sản phẩm từ tạo mới, cập nhật, đến ngừng kinh doanh, với khả năng theo dõi số lượng đã bán và tồn kho tự động.

b, Mục tiêu về quản lý đơn hàng và giỏ hàng

- Xây dựng hệ thống giỏ hàng (Shopping Cart) cho phép khách hàng thêm, cập nhật, xóa sản phẩm, và đặt hàng trực tuyến với đầy đủ thông tin giao hàng.
- **Quản lý đơn hàng với các trạng thái rõ ràng (pending, processing, paid, shipped, delivered, cancelled), cho phép cả khách hàng và admin theo dõi tiến độ đơn hàng.**
- Hỗ trợ nhiều loại đơn hàng (online, offline, subscription) và quản lý chi tiết từng sản phẩm trong đơn hàng với giá cả và số lượng cụ thể.

c, Mục tiêu về thanh toán

- Tích hợp đa phương thức thanh toán trực tuyến với ZaloPay, với cơ chế xử lý callback tự động để cập nhật trạng thái đơn hàng sau khi thanh toán thành công.
- Đảm bảo tính bảo mật trong giao dịch thanh toán thông qua MAC (ZaloPay),

đảm bảo tính toàn vẹn của các giao dịch.

- Hỗ trợ thanh toán qua QR code và payment URL, mang lại sự tiện lợi cho khách hàng.

d, Mục tiêu về xác thực và phân quyền

- Xây dựng hệ thống xác thực an toàn với JWT (JSON Web Token) và OAuth2 Resource Server, đảm bảo tính bảo mật cao với khả năng kiểm tra signature, expiration time, và token blacklist.
- Triển khai hệ thống phân quyền đa cấp với Role-Based Access Control (RBAC) và Permission-Based Access Control (PBAC), cho phép kiểm soát truy cập chi tiết đến từng chức năng của hệ thống.
- Hỗ trợ nhiều vai trò người dùng như ADMIN, USER, SHOP_MANAGER, EMPLOYEE_MANAGER, mỗi vai trò có các quyền hạn cụ thể phù hợp với nhiệm vụ.

e, Mục tiêu về thông báo

- Xây dựng hệ thống thông báo đa kênh, bao gồm Firebase Cloud Messaging (FCM) cho push notification trên mobile, WebSocket với RabbitMQ STOMP cho real-time notification trên web, và lưu trữ thông báo trong database để người dùng có thể xem lại lịch sử.
- Hỗ trợ cả thông báo cá nhân và broadcast notification, cho phép cửa hàng thông báo đến từng khách hàng cụ thể hoặc tất cả khách hàng một cách nhanh chóng và hiệu quả.
- Tự động gửi thông báo khi có các sự kiện quan trọng như cập nhật trạng thái đơn hàng, thanh toán thành công, hoặc các sự kiện hệ thống khác thông qua Spring Events.

f, Mục tiêu về quản lý nhân viên và lịch làm việc

- Xây dựng module quản lý lịch làm việc với khả năng tạo ca làm việc (Shift), phân công nhiệm vụ (Task), và lên lịch làm việc cho nhân viên theo ngày/tháng.
- Hỗ trợ gán nhiều nhân viên vào một ca làm việc và quản lý lịch làm việc linh hoạt, giúp cửa hàng tối ưu hóa việc phân công công việc.

g, Mục tiêu về công nghệ và kiến trúc

- Xây dựng hệ thống với Spring Boot 3.4.3 và Java 21, áp dụng các công nghệ hiện đại như JPA/Hibernate cho ORM, MapStruct cho object mapping, và Docker cho containerization.

- **Thiết kế kiến trúc RESTful API** với response format thống nhất, error handling tập trung, và CORS configuration để hỗ trợ đa nền tảng.
- Đảm bảo tính mở rộng và bảo trì dễ dàng thông qua việc tách biệt các layer (Controller, Service, Repository), sử dụng dependency injection, và áp dụng các design patterns phù hợp.

1.4 Đối tượng và phạm vi nghiên cứu

1.4.1 Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu các đối tượng sau:

a, Đối tượng nghiệp vụ

- **Quy trình quản lý kho hàng:** Nghiên cứu các quy trình nhập hàng, xuất hàng, kiểm kê tồn kho, và quản lý các biến thể sản phẩm (variant) với nhiều thuộc tính khác nhau. Hệ thống hỗ trợ quản lý tồn kho theo từng ProductAttribute với các thông tin về số lượng tồn kho (stockQuantity) và số lượng đã bán (soldQuantity), giúp theo dõi chính xác tình trạng hàng hóa.
- **Quy trình bán lẻ:** Nghiên cứu quy trình từ việc khách hàng xem sản phẩm, thêm vào giỏ hàng, đặt hàng, thanh toán, đến việc cửa hàng xử lý và giao hàng. Hệ thống hỗ trợ đầy đủ các bước trong quy trình mua hàng với các trạng thái đơn hàng rõ ràng (status: 0=pending, 1=processing, 2=paid, 3=failed, 4=cancelled, 5=delivered).
- **Dịch vụ kỹ thuật và sửa chữa điện thoại:** Nghiên cứu quy trình tiếp nhận thiết bị cần sửa chữa, chẩn đoán, sửa chữa, và giao lại cho khách hàng. Hệ thống hỗ trợ quản lý đơn sửa chữa với các trạng thái và ghi chú (OrderNote) để theo dõi tiến độ sửa chữa.
- **Quy trình thanh toán:** Nghiên cứu các phương thức thanh toán trực tuyến phổ biến tại Việt Nam (ZaloPay) và quy trình xử lý callback để cập nhật trạng thái đơn hàng tự động sau khi thanh toán thành công.
- **Quy trình quản lý nhân viên và phân công công việc:** Nghiên cứu cách thức quản lý lịch làm việc, phân công ca làm việc, và theo dõi tiến độ công việc của nhân viên thông qua hệ thống WorkSchedule, Shift, và Task.

b, Đối tượng người dùng

- **Khách hàng:** Nghiên cứu hành vi mua sắm và nhu cầu tra cứu thông tin sửa chữa của khách hàng trong kỷ nguyên số. Khách hàng có nhu cầu xem sản phẩm, so sánh giá cả, đặt hàng trực tuyến, theo dõi đơn hàng, và nhận thông báo về trạng thái đơn hàng một cách real-time.

- **Nhân viên cửa hàng:** Nghiên cứu nhu cầu của nhân viên trong việc quản lý đơn hàng, cập nhật trạng thái đơn hàng, quản lý sản phẩm, và xem lịch làm việc của mình. Nhân viên cần các công cụ để thực hiện công việc một cách hiệu quả và giảm thiểu sai sót.
- **Quản lý cửa hàng (Admin):** Nghiên cứu nhu cầu của quản lý trong việc quản lý toàn bộ hệ thống, bao gồm quản lý sản phẩm, đơn hàng, người dùng, phân quyền, và gửi thông báo. Quản lý cần có cái nhìn tổng quan về hoạt động kinh doanh và khả năng kiểm soát toàn bộ hệ thống.

1.4.2 Phạm vi nghiên cứu

Đề tài được giới hạn trong các phạm vi sau:

a, Phạm vi chức năng

Hệ thống tập trung vào hai mảng chính là **Bán hàng (Sales)** và **Sửa chữa (Repair Service)**, với các module cụ thể như sau:

- **Module Quản lý Sản phẩm:** Bao gồm quản lý danh mục sản phẩm (Category), sản phẩm chính (Product), các biến thể sản phẩm (ProductVariant), thuộc tính sản phẩm (ProductAttribute), hình ảnh sản phẩm (ProductImage), thương hiệu (Brand), và nguyên liệu cung cấp (Supply). Module này hỗ trợ đầy đủ các thao tác CRUD và quản lý trạng thái sản phẩm (đang bán/ngừng kinh doanh).
- **Module Quản lý Đơn hàng:** Bao gồm quản lý giỏ hàng (CartItem), đơn hàng (Orders), chi tiết đơn hàng (OrderItem), và ghi chú đơn hàng (OrderNote). Module này hỗ trợ đặt hàng trực tuyến, theo dõi trạng thái đơn hàng, và quản lý các loại đơn hàng khác nhau (online, offline, subscription).
- **Module Thanh toán:** Tích hợp với ZaloPay để xử lý thanh toán trực tuyến, bao gồm tạo payment URL, xử lý callback, và cập nhật trạng thái đơn hàng tự động. Module này đảm bảo tính bảo mật và toàn vẹn của các giao dịch thanh toán.
- **Module Xác thực và Phân quyền:** Quản lý người dùng (User), vai trò (Role), quyền hạn (Permission), và token đã vô hiệu hóa (InvalidatedToken). Module này đảm bảo tính bảo mật của hệ thống thông qua JWT authentication và phân quyền chi tiết.
- **Module Thông báo:** Quản lý thông báo hệ thống (Notification) và FCM device tokens (FcmDeviceToken), hỗ trợ gửi thông báo qua Firebase Cloud Messaging và WebSocket. Module này cho phép gửi thông báo cá nhân và broadcast notification đến người dùng.

- **Module Quản lý Lịch làm việc:** Quản lý lịch công việc (WorkSchedule), ca làm việc (Shift), và nhiệm vụ (Task). Module này giúp cửa hàng quản lý và phân công công việc cho nhân viên một cách hiệu quả.
- **Module Quản lý Banner:** Quản lý banner quảng cáo với khả năng upload hình ảnh, thiết lập thứ tự hiển thị, và quản lý trạng thái banner.

b, Phạm vi công nghệ

- **Backend:** Xây dựng ứng dụng RESTful API với Spring Boot 3.4.3 và Java 21, sử dụng MySQL làm cơ sở dữ liệu, JPA/Hibernate cho ORM, Spring Security với OAuth2 Resource Server cho authentication và authorization, RabbitMQ cho message queue, Firebase Admin SDK cho push notification, và Cloudinary cho quản lý hình ảnh.
- **Frontend Web:** Ứng dụng Web App được xây dựng với Vue.js, giao tiếp với backend thông qua RESTful API và WebSocket cho real-time notification.
- **Frontend Mobile:** Ứng dụng Mobile App được xây dựng với Android/Kotlin, tích hợp Firebase SDK, và giao tiếp với backend thông qua RESTful API.
- **Infrastructure:** Hệ thống được container hóa với Docker và Docker Compose, bao gồm MySQL database, RabbitMQ message broker, và Spring Boot application, đảm bảo dễ dàng triển khai và mở rộng.

c, Phạm vi giới hạn

- **Chưa tích hợp AI/ML:** Hệ thống hiện tại chưa tích hợp các tính năng sử dụng trí tuệ nhân tạo như gợi ý sản phẩm thông minh, dự báo nhu cầu, hoặc chatbot tự động.
- **Chưa có module vận chuyển tự động:** Hệ thống chưa tích hợp với các đơn vị vận chuyển như Giao Hàng Nhanh, Viettel Post để tự động tạo đơn vận chuyển và theo dõi vận đơn.
- **Chưa có hệ thống đánh giá và review:** Người dùng chưa thể đánh giá sản phẩm hoặc dịch vụ sửa chữa và để lại review.
- **Chưa có module marketing và khuyến mãi:** Hệ thống chưa hỗ trợ voucher, coupon code, hoặc các chương trình khuyến mãi.

1.5 Cấu trúc đồ án

Đồ án được chia thành 5 chương với nội dung chi tiết như sau:

- **Chương 1: Giới thiệu tổng quan về đề tài**
 - Trình bày lý do chọn đề tài dựa trên các vấn đề thực tế của cửa hàng bán

và sửa chữa điện thoại

- Nêu rõ mục tiêu tổng quát và mục tiêu cụ thể của đề tài, bao gồm các mục tiêu về quản lý sản phẩm, đơn hàng, thanh toán, xác thực, thông báo, và quản lý nhân viên
- Xác định đối tượng nghiên cứu (ng nghiệp vụ và người dùng) và phạm vi nghiên cứu (chức năng, công nghệ, và giới hạn)
- Giới thiệu cấu trúc đề án và nội dung từng chương

• **Chương 2: Cơ sở lý thuyết và công nghệ**

- **Phần Backend:** Giới thiệu Spring Boot framework, Spring Security với OAuth2 Resource Server, JWT authentication, JPA/Hibernate ORM, MySQL database, RabbitMQ message broker, Firebase Cloud Messaging, và Cloudinary
- **Phần Frontend Web:** Giới thiệu Vue.js framework, RESTful API integration, WebSocket cho real-time communication
- **Phần Frontend Mobile:** Giới thiệu Android development với Kotlin, Firebase SDK integration, và RESTful API consumption
- **Phần Infrastructure:** Giới thiệu Docker và Docker Compose cho containerization
- Trình bày các design patterns và best practices được áp dụng trong dự án như MVC, MVVM pattern, Repository pattern, DTO pattern, và Dependency Injection

• **Chương 3: Phân tích và thiết kế hệ thống**

- **Phân tích yêu cầu:** Phân tích các use case chính của hệ thống như đăng nhập/dăng xuất, quản lý sản phẩm, đặt hàng, thanh toán, quản lý đơn hàng, gửi thông báo, và quản lý lịch làm việc. Vẽ biểu đồ Use Case Diagram để mô tả các actor và use case
- **Thiết kế cơ sở dữ liệu:** Thiết kế ERD (Entity Relationship Diagram) mô tả các entity và mối quan hệ giữa chúng, bao gồm các bảng User, Role, Permission, Product, ProductVariant, ProductAttribute, Category, Orders, OrderItem, CartItem, Notification, WorkSchedule, Shift, Task, Banner, và các bảng liên quan khác. Mô tả chi tiết các trường dữ liệu và ràng buộc
- **Thiết kế kiến trúc hệ thống:** Mô tả kiến trúc tổng thể của hệ thống với các layer (Presentation Layer, Business Logic Layer, Data Access Layer),

kiến trúc RESTful API, và cách các component tương tác với nhau. Vẽ sơ đồ kiến trúc hệ thống

- **Thiết kế API:** Thiết kế chi tiết các REST API endpoints với request/response format, authentication requirements, và error handling. Bao gồm các API cho authentication, quản lý sản phẩm, giỏ hàng, đơn hàng, thanh toán, thông báo, và quản lý lịch làm việc
- **Thiết kế luồng nghiệp vụ:** Mô tả chi tiết các luồng nghiệp vụ chính như luồng đặt hàng và thanh toán, luồng quản lý sản phẩm, luồng gửi thông báo, và luồng quản lý lịch làm việc. Vẽ các biểu đồ Sequence Diagram hoặc Activity Diagram để minh họa

• Chương 4: Kết quả triển khai và giao diện

- **Môi trường phát triển và triển khai:** Mô tả môi trường phát triển (IDE, JDK, Node.js, Android Studio), cấu hình Docker Compose, và quy trình build và deploy ứng dụng
- **Triển khai Backend:** Trình bày các tính năng đã triển khai trên backend bao gồm:
 - * Hệ thống xác thực và phân quyền với JWT và OAuth2
 - * Module quản lý sản phẩm với cấu trúc phân cấp Product-Variant-Attribute
 - * Module giỏ hàng và đơn hàng với đầy đủ các thao tác CRUD
 - * Tích hợp thanh toán ZaloPay với xử lý callback tự động
 - * Hệ thống thông báo đa kênh với FCM và WebSocket
 - * Module quản lý lịch làm việc và nhân viên
 - * Quản lý banner và upload hình ảnh với Cloudinary
- **Giao diện Web App:** Chụp màn hình và mô tả các giao diện chính của ứng dụng web bao gồm trang đăng nhập, trang quản lý sản phẩm, trang quản lý đơn hàng, trang thanh toán, và các trang quản lý khác
- **Giao diện Mobile App:** Chụp màn hình và mô tả các màn hình chính của ứng dụng mobile bao gồm màn hình đăng nhập, màn hình danh sách sản phẩm, màn hình giỏ hàng, màn hình đặt hàng, màn hình theo dõi đơn hàng, và màn hình thông báo
- **Kiểm thử hệ thống:** Trình bày các test case đã thực hiện, kết quả kiểm thử các chức năng chính, và đánh giá hiệu suất hệ thống

• Chương 5: Kết luận và hướng phát triển

- **Tổng kết kết quả đạt được:** Đánh giá tổng quan về các thành tựu đã đạt được, bao gồm việc giải quyết các vấn đề ban đầu, các tính năng đã triển khai thành công, và những đóng góp của đề tài
- **Những hạn chế còn tồn tại:** Phân tích các hạn chế của hệ thống hiện tại như thiếu tính năng AI/ML, chưa tích hợp vận chuyển tự động, thiếu hệ thống đánh giá và review, chưa có module marketing và khuyến mãi, và thiếu dashboard phân tích
- **Hướng phát triển trong tương lai:** Đề xuất các hướng phát triển như tích hợp AI/ML cho gợi ý sản phẩm và chatbot, mở rộng phương thức thanh toán quốc tế, phát triển hệ thống đánh giá và review, xây dựng dashboard phân tích và BI, tích hợp API vận chuyển tự động, và tối ưu hóa hiệu suất với caching và microservices
- **Kết luận:** Tóm tắt lại giá trị và ý nghĩa của đề tài, đánh giá mức độ đạt được các mục tiêu ban đầu, và đề xuất các nghiên cứu tiếp theo

1.6 Kết luận chương

Như vậy, chương 1 đã xác định rõ ràng bài toán cần giải quyết là khắc phục sự rời rạc trong quản lý bán hàng và sửa chữa truyền thống thông qua việc xây dựng một hệ thống tích hợp, tự động hóa trên nền tảng Web và Mobile. Các mục tiêu, đối tượng và phạm vi chức năng cũng đã được hoạch định cụ thể, làm kim chỉ nam cho quá trình phát triển hệ thống.

Để hiện thực hóa các mục tiêu đã đề ra, việc lựa chọn nền tảng công nghệ và cơ sở lý thuyết phù hợp là yếu tố tiên quyết. Chương tiếp theo sẽ đi sâu vào tìm hiểu cơ sở lý thuyết và các công nghệ cốt lõi sẽ được sử dụng trong đồ án như Spring Boot, Vue.js, Android Kotlin và kiến trúc RESTful API.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Giới thiệu chương

Để hiện thực hóa các mục tiêu và giải quyết các bài toán nghiệp vụ đã đặt ra ở Chương 1, việc lựa chọn kiến trúc hệ thống và nền tảng công nghệ phù hợp là yếu tố tiên quyết. Chương 2 sẽ trình bày chi tiết về cơ sở lý thuyết và các công nghệ cốt lõi được sử dụng để xây dựng hệ thống quản lý bán hàng và sửa chữa điện thoại.

Nội dung chương đi từ cái nhìn tổng quan về kiến trúc Client-Server và chuẩn RESTful API, đến việc phân tích sâu các công nghệ đặc thù cho từng thành phần: Java Spring Boot cho Backend, Vue.js cho Web Admin, Android Kotlin cho Mobile App và MySQL cho cơ sở dữ liệu. Bên cạnh đó, các công nghệ hỗ trợ quan trọng như RabbitMQ, Firebase, JWT hay các cổng thanh toán điện tử cũng được giới thiệu nhằm làm rõ bức tranh kỹ thuật toàn diện của đồ án.

2.2 Tổng quan về kiến trúc hệ thống

2.2.1 Mô hình Client – Server

Hệ thống được xây dựng theo mô hình Client-Server, một kiến trúc phổ biến và hiệu quả cho các ứng dụng web và mobile hiện đại. Trong mô hình này, client (ứng dụng Web và Mobile) và server (Backend API) được tách biệt hoàn toàn, giao tiếp với nhau thông qua các giao thức chuẩn như HTTP/HTTPS.

Lý do chọn mô hình này:

- **Tách biệt trách nhiệm:** Giao diện người dùng (Frontend/Mobile) và logic nghiệp vụ (Backend) được tách biệt rõ ràng, giúp mỗi phần tập trung vào nhiệm vụ của mình. Frontend chỉ lo việc hiển thị và tương tác với người dùng, trong khi Backend xử lý logic nghiệp vụ và quản lý dữ liệu.
- **Dễ bảo trì và mở rộng:** Khi cần thay đổi giao diện hoặc thêm tính năng mới, có thể chỉnh sửa từng phần độc lập mà không ảnh hưởng đến phần còn lại. Điều này đặc biệt quan trọng khi hệ thống cần hỗ trợ nhiều nền tảng khác nhau (Web, Android, iOS).
- **Đa nền tảng:** Một Backend API duy nhất có thể phục vụ nhiều client khác nhau (Web app, Mobile app, thậm chí là các ứng dụng khác trong tương lai), đảm bảo dữ liệu luôn đồng bộ và giảm thiểu công sức phát triển.
- **Bảo mật tập trung:** Tất cả logic bảo mật, xác thực và phân quyền được tập trung ở Backend, đảm bảo tính nhất quán và an toàn cho toàn bộ hệ thống.
- **Khả năng mở rộng:** Backend có thể được triển khai trên nhiều server và sử

dùng load balancing để xử lý lượng truy cập lớn, trong khi các client có thể được phân phối qua CDN hoặc app stores.

Trong hệ thống này, Backend được xây dựng với Spring Boot chạy trên port 8080 với context path '/bej3', cung cấp RESTful API cho các client. Các client gửi request đến Backend thông qua HTTP/HTTPS và nhận response dưới dạng JSON.

2.2.2 Kiến trúc RESTful API

REST (Representational State Transfer) là một kiến trúc phần mềm được sử dụng rộng rãi cho việc thiết kế các web service. RESTful API sử dụng các phương thức HTTP chuẩn để thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên các tài nguyên.

Các nguyên tắc của RESTful API trong hệ thống:

- **Sử dụng các phương thức HTTP chuẩn:**

- GET: Lấy thông tin tài nguyên (ví dụ: GET /home để lấy danh sách sản phẩm)
- POST: Tạo mới tài nguyên (ví dụ: POST /cart/add/{attId} để thêm sản phẩm vào giỏ hàng)
- PUT: Cập nhật tài nguyên (ví dụ: PUT /manage/product/update/{id} để cập nhật sản phẩm)
- DELETE: Xóa tài nguyên (ví dụ: DELETE /cart/remove/{cartItemId} để xóa sản phẩm khỏi giỏ hàng)

- **Định dạng dữ liệu JSON:** Tất cả request và response đều sử dụng định dạng JSON (JavaScript Object Notation), một định dạng nhẹ và dễ đọc, được hỗ trợ rộng rãi bởi các ngôn ngữ lập trình và framework hiện đại.

- **URL có cấu trúc rõ ràng:** Các endpoint được thiết kế với cấu trúc URL có ý nghĩa, ví dụ:

- /home - Trang chủ, danh sách sản phẩm công khai
- /cart/* - Các thao tác liên quan đến giỏ hàng
- /manage/* - Các thao tác quản lý (yêu cầu quyền admin)
- /auth/* - Các thao tác xác thực
- /api/notifications/* - Quản lý thông báo

- **Response format thống nhất:** Tất cả API response đều tuân theo format chuẩn:

```
{  
    "result": { /* dữ liệu thực tế */ },  
    "code": 1000, // 1000 = Success, 1001 = Error, etc.  
    "message": "Success"  
}
```

Điều này giúp client dễ dàng xử lý response và hiển thị thông báo lỗi phù hợp.

- **HTTP Status Codes:** Sử dụng các mã trạng thái HTTP chuẩn để biểu thị kết quả của request:
 - 200 OK: Request thành công
 - 201 Created: Tài nguyên được tạo thành công
 - 400 Bad Request: Request không hợp lệ
 - 401 Unauthorized: Chưa xác thực hoặc token không hợp lệ
 - 403 Forbidden: Không có quyền truy cập
 - 404 Not Found: Tài nguyên không tồn tại
 - 500 Internal Server Error: Lỗi server
- **Stateless:** Mỗi request đều độc lập và chứa đầy đủ thông tin cần thiết để xử lý, không phụ thuộc vào các request trước đó. Thông tin xác thực được gửi kèm trong header của mỗi request dưới dạng JWT token.

Lợi ích của RESTful API:

- **Dễ hiểu và sử dụng:** RESTful API sử dụng các phương thức HTTP chuẩn và URL có cấu trúc rõ ràng, giúp developers dễ dàng hiểu và sử dụng API.
- **Độc lập với platform:** API có thể được sử dụng bởi bất kỳ client nào hỗ trợ HTTP, không phụ thuộc vào ngôn ngữ lập trình hay platform cụ thể.
- **Có thể cache:** Các response có thể được cache ở client hoặc proxy server để cải thiện hiệu suất.
- **Khả năng mở rộng:** RESTful API dễ dàng mở rộng bằng cách thêm các endpoint mới mà không ảnh hưởng đến các endpoint hiện có.

2.2.3 Khả năng đáp ứng đồng thời MVVM (Mobile) và MVC (Web)

Backend trong hệ thống được thiết kế theo hướng **API-first** và **client-agnostic** (không phụ thuộc mô hình trình bày của client). Vì vậy, cùng một tập RESTful API có thể phục vụ đồng thời:

- **Ứng dụng Mobile theo MVVM:** View chỉ hiển thị, ViewModel xử lý trạng thái/logic UI và gọi dữ liệu qua tầng Repository/UseCase; Backend đóng vai trò **nguồn dữ liệu** và **thực thi nghiệp vụ** thông qua API.
- **Ứng dụng Web theo MVC:** Controller (web) nhận thao tác người dùng, gọi API Backend để lấy/cập nhật dữ liệu (Model), sau đó render ra View; Backend đóng vai trò **dịch vụ nghiệp vụ** và **cổng truy cập dữ liệu**.

Điểm chung giúp Backend dùng được cho cả hai mô hình:

- **Hợp đồng API thống nhất:** Endpoint, request/response JSON và quy ước mã lỗi được chuẩn hoá, giúp cả Mobile và Web xử lý giống nhau.
- **Bảo mật tập trung:** JWT Authentication và phân quyền (roles/permissions) được kiểm soát ở Backend, đảm bảo tính nhất quán dù client là MVVM hay MVC.
- **Stateless và tách biệt UI:** Backend không lưu trạng thái giao diện; mọi trạng thái hiển thị (loading, paging, filter, view-state) được quản lý ở phía client theo MVVM/MVC tương ứng.
- **Tái sử dụng nghiệp vụ:** Mọi logic nghiệp vụ (giỏ hàng, đơn hàng, quản trị, thông báo, ...) được triển khai một lần ở Backend và tái sử dụng cho cả hai nền tảng.

a, Luồng tích hợp với Mobile (MVVM)

- **View** kích hoạt hành động (click, nhập liệu) → **ViewModel** gọi **Repository** → gửi request đến Backend (HTTP/HTTPS, JSON, JWT).
- Backend trả response theo format chuẩn (result/code/message) → Repository/UseCase ánh xạ dữ liệu → ViewModel cập nhật state để View render.

b, Luồng tích hợp với Web (MVC)

- **Controller (web)** nhận request từ người dùng → gọi REST API của Backend để lấy/cập nhật dữ liệu (Model).
- Controller đưa dữ liệu vào **View** để render (server-side) hoặc trả JSON cho tầng View (client-side), tùy cách triển khai web.

Nhờ thiết kế RESTful API chuẩn hoá, stateless và bảo mật tập trung, Backend có thể phục vụ đồng thời nhiều client khác nhau, trong đó có Web theo MVC và Mobile theo MVVM, mà không cần thay đổi logic nghiệp vụ ở phía server.

2.3 Các công nghệ sử dụng

2.3.1 Backend: Java Spring Boot

a, Giới thiệu về Java và Spring Boot

Java là một ngôn ngữ lập trình hướng đối tượng, đa nền tảng, được phát triển bởi Sun Microsystems (nay thuộc Oracle). Java được biết đến với khẩu hiệu "Write Once, Run Anywhere" (WORA), cho phép code Java chạy trên bất kỳ platform nào có cài đặt Java Virtual Machine (JVM).

Trong dự án này, hệ thống sử dụng **Java 21** - phiên bản mới nhất của Java với nhiều tính năng hiện đại như pattern matching, records, sealed classes, và virtual threads giúp cải thiện hiệu suất và năng suất phát triển.

Spring Boot là một framework Java được xây dựng trên nền tảng Spring Framework, được thiết kế để đơn giản hóa việc phát triển các ứng dụng Java enterprise. Spring Boot cung cấp:

- **Auto-configuration:** Tự động cấu hình các component dựa trên các dependencies có trong classpath, giảm thiểu công sức cấu hình thủ công.
- **Starter dependencies:** Các dependency được đóng gói sẵn với các thư viện phù hợp, giúp nhanh chóng bắt đầu dự án mà không cần tìm kiếm và cấu hình từng thư viện riêng lẻ.
- **Embedded server:** Tích hợp sẵn Tomcat, Jetty, hoặc Undertow, cho phép chạy ứng dụng như một standalone JAR file mà không cần deploy lên application server riêng.
- **Production-ready features:** Cung cấp các tính năng sẵn có như health checks, metrics, externalized configuration, giúp dễ dàng monitor và quản lý ứng dụng trong môi trường production.

Hệ thống sử dụng **Spring Boot 3.4.3**, phiên bản mới nhất với nhiều cải tiến về hiệu suất, bảo mật, và hỗ trợ tốt hơn cho Java 21.

b, Lý do lựa chọn Java Spring Boot

- **Tính ổn định và trưởng thành:** Java và Spring Framework đã được sử dụng rộng rãi trong nhiều năm, có cộng đồng lớn và tài liệu phong phú. Điều này đảm bảo tính ổn định và khả năng tìm kiếm giải pháp cho các vấn đề gặp phải.
- **Bảo mật mạnh mẽ:** Spring Security cung cấp một framework bảo mật toàn diện với các tính năng như authentication, authorization, CSRF protection, session management. Trong dự án, Spring Security được tích hợp với OAuth2 Resource Server để xử lý JWT authentication một cách an toàn.

- **Ecosystem phong phú:** Spring ecosystem bao gồm nhiều module như Spring Data JPA (cho database access), Spring Web (cho REST API), Spring Web-Socket (cho real-time communication), Spring AMQP (cho message queue), giúp giải quyết hầu hết các nhu cầu của một ứng dụng enterprise.
- **Dependency Injection:** Spring Framework sử dụng Dependency Injection (DI) và Inversion of Control (IoC), giúp code dễ test, dễ bảo trì, và giảm coupling giữa các component.
- **Hiệu suất cao:** JVM được tối ưu hóa tốt và có khả năng xử lý nhiều request đồng thời. Spring Boot cũng hỗ trợ reactive programming với Spring WebFlux cho các ứng dụng cần xử lý nhiều request đồng thời.
- **Dễ dàng tích hợp:** Spring Boot dễ dàng tích hợp với các công nghệ khác như MySQL, RabbitMQ, Firebase, Cloudinary thông qua các starter dependencies và auto-configuration.

c, Công nghệ ORM: Hibernate/JPA

JPA (Java Persistence API) là một specification của Java cho việc quản lý dữ liệu quan hệ trong các ứng dụng Java. JPA cung cấp một cách tiếp cận object-oriented để làm việc với database, cho phép developers làm việc với các Java objects thay vì SQL queries trực tiếp.

Hibernate là một implementation phổ biến của JPA specification, cung cấp một framework ORM (Object-Relational Mapping) mạnh mẽ để ánh xạ các Java objects vào database tables.

Lợi ích của JPA/Hibernate trong dự án:

- **Giảm thiểu boilerplate code:** JPA tự động generate các SQL queries dựa trên các annotations và method names, giảm thiểu việc viết SQL thủ công. Ví dụ, method `findByStatusOrderByCreateDateDesc(int status)` sẽ tự động được translate thành SQL query phù hợp.
- **Type safety:** Làm việc với Java objects thay vì raw SQL giúp phát hiện lỗi tại compile time thay vì runtime.
- **Database independence:** Code JPA có thể hoạt động với nhiều loại database khác nhau (MySQL, PostgreSQL, H2, etc.) mà không cần thay đổi code, chỉ cần thay đổi driver và connection string.
- **Cascade operations:** Hibernate hỗ trợ cascade operations, cho phép tự động thực hiện các thao tác liên quan khi một entity được save, update, hoặc delete. Ví dụ, khi xóa một Product, tất cả các ProductVariant và ProductAttribute liên

quan cũng được xóa tự động nếu được cấu hình với `CascadeType.ALL`.

- **Lazy loading và Eager loading:** Hibernate hỗ trợ lazy loading để tối ưu hiệu suất bằng cách chỉ load dữ liệu khi cần thiết, và eager loading để load dữ liệu liên quan ngay lập tức.
- **Transaction management:** Hibernate tích hợp tốt với Spring's transaction management, đảm bảo tính toàn vẹn dữ liệu thông qua ACID properties.

Trong dự án, các entity được định nghĩa với các annotations như `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@ManyToOne`, `@OneToMany`, `@ManyToMany` để mô tả cấu trúc database và các quan hệ giữa các bảng. Spring Data JPA được sử dụng để tạo các repository interfaces tự động implement các method CRUD và custom queries.

d, Các module Spring Boot được sử dụng

- **spring-boot-starter-web:** Cung cấp các tính năng để xây dựng RESTful API, bao gồm Spring MVC, embedded Tomcat server, và JSON support.
- **spring-boot-starter-data-jpa:** Cung cấp Spring Data JPA để làm việc với database, bao gồm Hibernate và các database drivers.
- **spring-boot-starter-oauth2-resource-server:** Cung cấp OAuth2 Resource Server để xử lý JWT authentication và authorization.
- **spring-boot-starter-websocket:** Cung cấp WebSocket support cho real-time communication.
- **spring-boot-starter-amqp:** Cung cấp support cho RabbitMQ message broker.
- **spring-boot-starter-validation:** Cung cấp Bean Validation API để validate dữ liệu đầu vào.
- **spring-boot-starter-thymeleaf:** Cung cấp Thymeleaf template engine cho việc render HTML (sử dụng cho các trang payment callback).

2.3.2 Frontend Web: Vue.js

a, Giới thiệu về Vue.js

Vue.js là một progressive JavaScript framework được tạo ra bởi Evan You vào năm 2014. Vue.js được thiết kế để xây dựng các Single Page Applications (SPA) và user interfaces một cách dễ dàng và hiệu quả.

Đặc điểm nổi bật của Vue.js:

- **Progressive Framework:** Vue.js có thể được tích hợp từng phần vào các dự

án hiện có, không cần phải viết lại toàn bộ ứng dụng.

- **Component-based Architecture:** Ứng dụng được xây dựng từ các component có thể tái sử dụng, mỗi component có logic và template riêng, giúp code dễ tổ chức và bảo trì.
- **Reactive Data Binding:** Vue.js sử dụng reactive data binding, tự động cập nhật DOM khi dữ liệu thay đổi, giảm thiểu công sức thao tác DOM thủ công.
- **Virtual DOM:** Vue.js sử dụng Virtual DOM để tối ưu hóa việc render, chỉ cập nhật những phần cần thiết của DOM thay vì render lại toàn bộ trang.
- **Easy to Learn:** Vue.js có syntax đơn giản và dễ hiểu, phù hợp cho cả developers mới bắt đầu và developers có kinh nghiệm.

b, Ứng dụng trong Admin Dashboard

Trong hệ thống, Vue.js được sử dụng để xây dựng Admin Dashboard - giao diện quản trị cho phép admin và nhân viên quản lý các chức năng của hệ thống:

- **Quản lý sản phẩm:** Giao diện để thêm, sửa, xóa, và quản lý sản phẩm với các biến thể và thuộc tính phức tạp. Vue.js components giúp tổ chức code một cách rõ ràng và dễ bảo trì.
- **Quản lý đơn hàng:** Hiển thị danh sách đơn hàng, chi tiết đơn hàng, và cập nhật trạng thái đơn hàng với real-time updates thông qua WebSocket.
- **Quản lý người dùng và phân quyền:** Giao diện để quản lý users, roles, và permissions với các form validation và error handling.
- **Thống kê và báo cáo:** Hiển thị các biểu đồ và thống kê về doanh thu, sản phẩm bán chạy, v.v. với các thư viện visualization như Chart.js hoặc ECharts.
- **Real-time notifications:** Sử dụng WebSocket để nhận thông báo real-time từ server, hiển thị notification badge và popup notifications.

c, Tích hợp với Backend API

Vue.js frontend giao tiếp với Spring Boot backend thông qua:

- **Axios/Fetch API:** Sử dụng Axios hoặc Fetch API để gửi HTTP requests đến RESTful API endpoints.
- **JWT Token Management:** Lưu trữ JWT token trong localStorage hoặc sessionStorage, và tự động thêm token vào Authorization header của mỗi request.
- **Error Handling:** Xử lý các lỗi từ API và hiển thị thông báo lỗi phù hợp cho người dùng.
- **Request/Response Interceptors:** Sử dụng interceptors để tự động thêm token,

xử lý lỗi 401 (unauthorized) để redirect đến trang login, và format response data.

2.3.3 Mobile App: Android với Kotlin và Jetpack Components

a, Giới thiệu về Kotlin

Kotlin là một ngôn ngữ lập trình statically typed được phát triển bởi JetBrains, được Google công nhận là ngôn ngữ chính thức cho Android development vào năm 2017. Kotlin được thiết kế để tương thích hoàn toàn với Java và có thể chạy trên JVM.

Ưu điểm của Kotlin:

- **Concise Syntax:** Kotlin có syntax ngắn gọn hơn Java, giảm thiểu boilerplate code và tăng năng suất phát triển.
- **Null Safety:** Kotlin có hệ thống type system giúp tránh NullPointerException tại compile time, một trong những lỗi phổ biến nhất trong Java.
- **Coroutines:** Kotlin Coroutines cung cấp một cách đơn giản để xử lý asynchronous programming và concurrency, thay thế cho callbacks và threads truyền thống.
- **Extension Functions:** Cho phép thêm functions vào các classes hiện có mà không cần kế thừa hoặc sử dụng design patterns như Decorator.
- **Data Classes:** Cung cấp cách đơn giản để tạo các classes chỉ chứa data với các methods như equals(), hashCode(), toString() được tự động generate.
- **Interoperability with Java:** Kotlin có thể sử dụng các thư viện Java hiện có và có thể được gọi từ Java code, giúp migration từ Java sang Kotlin dễ dàng.

b, Android Jetpack Components

Android Jetpack là một bộ thư viện, tools, và guidelines được Google phát triển để giúp developers xây dựng các ứng dụng Android chất lượng cao một cách nhanh chóng và dễ dàng.

Các Jetpack Components được sử dụng trong dự án:

- **ViewModel:** Quản lý UI-related data trong lifecycle-aware manner, giữ data khi configuration changes (như screen rotation). ViewModel giúp tách biệt business logic khỏi UI code.
- **LiveData:** Một observable data holder class cho phép các components (Activity, Fragment) observe changes trong data và tự động update UI khi data thay đổi. LiveData là lifecycle-aware, chỉ update UI khi component đang ở trạng

thái active.

- **Room Database:** Một abstraction layer trên SQLite, cung cấp compile-time verification của SQL queries và hỗ trợ RxJava và LiveData. Room được sử dụng để cache dữ liệu từ API và hỗ trợ offline mode.
- **Retrofit:** Một type-safe HTTP client cho Android và Java, được sử dụng để gọi RESTful API. Retrofit tự động serialize/deserialize JSON responses và hỗ trợ coroutines.
- **Navigation Component:** Quản lý navigation giữa các screens trong ứng dụng, hỗ trợ deep linking, và tích hợp với back stack.
- **WorkManager:** Quản lý background tasks một cách đáng tin cậy, đảm bảo tasks được thực hiện ngay cả khi app bị đóng hoặc device restart.
- **Material Design Components:** Cung cấp các UI components theo Material Design guidelines của Google, giúp tạo giao diện đẹp và nhất quán.

c, Ứng dụng trong Mobile App

Mobile app được xây dựng với Kotlin và Jetpack Components để cung cấp trải nghiệm tốt cho khách hàng:

- **Đăng nhập và xác thực:** Sử dụng ViewModel và LiveData để quản lý authentication state, lưu trữ JWT token một cách an toàn với EncryptedSharedPreferences.
- **Danh sách sản phẩm:** Hiển thị danh sách sản phẩm với RecyclerView, sử dụng Paging Library để load dữ liệu theo từng trang và cache với Room Database.
- **Giỏ hàng và đặt hàng:** Quản lý giỏ hàng với Room Database để hỗ trợ offline, và đồng bộ với server khi có kết nối internet.
- **Theo dõi đơn hàng:** Hiển thị trạng thái đơn hàng real-time với WebSocket hoặc polling, sử dụng WorkManager để đảm bảo notifications được nhận ngay cả khi app ở background.
- **Push Notifications:** Tích hợp Firebase Cloud Messaging để nhận push notifications về trạng thái đơn hàng, khuyến mãi, và các thông báo quan trọng khác.

2.3.4 Hệ quản trị cơ sở dữ liệu (DBMS): MySQL

a, Giới thiệu về MySQL

MySQL là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở, được phát triển bởi Oracle Corporation. MySQL là một trong những database phổ biến nhất trên thế giới, được sử dụng bởi nhiều ứng dụng web lớn như Facebook, Twitter, YouTube.

Đặc điểm của MySQL:

- **Relational Database:** MySQL là một relational database, lưu trữ dữ liệu trong các bảng với các quan hệ được định nghĩa rõ ràng thông qua foreign keys.
- **ACID Compliance:** MySQL hỗ trợ ACID (Atomicity, Consistency, Isolation, Durability) properties để đảm bảo tính toàn vẹn dữ liệu trong các transactions.
- **High Performance:** MySQL được tối ưu hóa cho performance với các tính năng như indexing, query optimization, và connection pooling.
- **Scalability:** MySQL hỗ trợ replication và sharding để scale horizontally khi cần xử lý lượng dữ liệu lớn.
- **Security:** MySQL cung cấp các tính năng bảo mật như user authentication, role-based access control, và data encryption.

b, Lý do chọn MySQL

- **Phù hợp với dữ liệu có cấu trúc chặt chẽ:** Hệ thống quản lý các entity có quan hệ phức tạp như Product-Variant-Attribute, User-Order-OrderItem, WorkSchedule-Shift-Task. MySQL với relational model phù hợp để quản lý các quan hệ này một cách hiệu quả.
- **Tích hợp tốt với Spring Boot:** Spring Boot có hỗ trợ tốt cho MySQL thông qua Spring Data JPA, với auto-configuration và connection pooling tự động.
- **Phổ biến và ổn định:** MySQL được sử dụng rộng rãi và có cộng đồng lớn, dễ dàng tìm kiếm tài liệu và giải pháp cho các vấn đề gặp phải.
- **Cost-effective:** MySQL là mã nguồn mở và miễn phí, phù hợp cho các dự án startup và vừa.
- **Docker support:** MySQL có official Docker image, dễ dàng triển khai và quản lý với Docker Compose như trong dự án này.

c, Cấu trúc Database trong dự án

Database được thiết kế với các bảng chính:

- **Identity Module:** user, role, permission, user_roles, role_permissions,

invalidated_token, fcm_device_token

- **Product Module:** category, product, product_variant, product_attribute, product_image, brand, supply
- **Cart & Order Module:** cart_item, orders, order_item, order_note
- **Work Schedule Module:** work_schedule, shift, task, work_schedule_users
- **Notification Module:** notification
- **Banner Module:** banner

Hệ thống sử dụng MySQL 8 với các tính năng như JSON data type, window functions, và improved performance so với các phiên bản trước.

2.3.5 Các công nghệ hỗ trợ khác

a, Firebase Cloud Messaging (FCM)

Firebase Cloud Messaging là một dịch vụ cross-platform của Google để gửi messages và notifications đến các thiết bị iOS, Android, và Web.

Ứng dụng trong dự án:

- **Push Notifications:** Gửi thông báo đến mobile app về trạng thái đơn hàng, khuyến mãi, và các sự kiện quan trọng khác.
- **Firebase Admin SDK:** Backend sử dụng Firebase Admin SDK (Java) để gửi notifications đến các devices thông qua FCM tokens được lưu trữ trong database.
- **Multicast Messaging:** Hỗ trợ gửi thông báo đến nhiều devices cùng lúc (broadcast notification) và gửi thông báo đến một user cụ thể (personal notification).
- **Device Token Management:** Hệ thống quản lý FCM device tokens trong database, tự động xóa các token không hợp lệ khi gửi thất bại.

Lợi ích:

- **Reliability:** FCM đảm bảo messages được deliver một cách đáng tin cậy, ngay cả khi app không chạy.
- **Free:** FCM là miễn phí với giới hạn rất cao, phù hợp cho hầu hết các ứng dụng.
- **Cross-platform:** Hỗ trợ cả Android, iOS, và Web với cùng một API.

b, ZaloPay - Cổng thanh toán điện tử

ZaloPay là hai cổng thanh toán điện tử phổ biến tại Việt Nam, cho phép khách hàng thanh toán trực tuyến qua nhiều phương thức như thẻ ngân hàng, ví điện tử,

và QR code.

Tích hợp trong dự án:

- **ZaloPay Integration:**

- Tạo payment request với app_trans_id unique (format: YYMMDD_timestamp_random)
- Sử dụng HMAC SHA256 để tạo MAC (Message Authentication Code) cho request
- Xử lý cả server-to-server callback và user return callback
- Verify MAC với key2 để đảm bảo tính toàn vẹn của callback

- **Bảo mật:** Cổng thanh toán sử dụng cryptographic hashing để đảm bảo tính toàn vẹn và xác thực của các request/response.

- **Automatic Order Status Update:** Sau khi thanh toán thành công, hệ thống tự động cập nhật trạng thái đơn hàng (status = 2) thông qua callback handlers, không cần can thiệp thủ công.

c, JWT (JSON Web Token) - Cơ chế xác thực

JWT là một chuẩn mở (RFC 7519) định nghĩa cách truyền thông tin một cách compact và self-contained giữa các parties dưới dạng JSON object.

Cấu trúc của JWT:

- **Header:** Chứa thông tin về loại token (JWT) và thuật toán mã hóa (HS512 trong dự án)
- **Payload:** Chứa các claims (thông tin về user, roles, expiration time, etc.)
- **Signature:** Được tạo bằng cách mã hóa header và payload với secret key để đảm bảo tính toàn vẹn

Ứng dụng trong dự án:

- **Authentication:** Sau khi đăng nhập thành công, server tạo JWT token chứa thông tin user và roles, client lưu token và gửi kèm trong Authorization header của mỗi request.
- **Custom JWT Decoder:** Hệ thống sử dụng CustomJwtDecoder để validate token, kiểm tra signature với HS512 algorithm và secret key, và verify token không nằm trong blacklist (InvalidatedToken table).
- **Token Introspection:** Hệ thống cung cấp endpoint /auth/introspect để kiểm tra tính hợp lệ của token mà không cần decode toàn bộ token.
- **Token Blacklist:** Khi user logout, token được thêm vào InvalidatedToken table

để ngăn chặn việc sử dụng lại token đã hết hạn.

- **Stateless Authentication:** JWT cho phép stateless authentication, server không cần lưu trữ session, giúp hệ thống dễ scale và giảm memory usage.

Lợi ích:

- **Security:** Token được ký bằng secret key, đảm bảo không thể bị giả mạo
- **Self-contained:** Token chứa đầy đủ thông tin cần thiết, không cần query database mỗi lần request
- **Cross-domain:** Có thể sử dụng token giữa các domain khác nhau
- **Mobile-friendly:** Phù hợp cho mobile apps vì không cần cookies

d, Cloudinary - Nền tảng lưu trữ đám mây

Cloudinary là một cloud-based image and video management platform cung cấp các dịch vụ upload, storage, manipulation, và delivery cho media files.

Ứng dụng trong dự án:

- **Image Upload:** Upload hình ảnh sản phẩm, banner, và các hình ảnh khác lên Cloudinary thông qua API.
- **Automatic Optimization:** Cloudinary tự động optimize hình ảnh (resize, compress, format conversion) để cải thiện hiệu suất load trang.
- **CDN Delivery:** Cloudinary sử dụng CDN để phân phối hình ảnh nhanh chóng trên toàn cầu, giảm thời gian load.
- **Transformations:** Hỗ trợ các transformations như resize, crop, watermark, filters thông qua URL parameters mà không cần xử lý trên server.
- **Secure URLs:** Hỗ trợ signed URLs để bảo vệ hình ảnh khỏi truy cập trái phép.

Lợi ích:

- **Giảm tải cho server:** Hình ảnh được lưu trữ và phân phối bởi Cloudinary, không chiếm dung lượng storage của server.
- **Hiệu suất cao:** CDN đảm bảo hình ảnh được load nhanh từ server gần nhất.
- **Dễ sử dụng:** API đơn giản và có SDK cho nhiều ngôn ngữ, tích hợp dễ dàng vào ứng dụng.

e, RabbitMQ - Message Broker

RabbitMQ là một message broker mã nguồn mở, implement Advanced Message Queuing Protocol (AMQP), được sử dụng để xử lý message queuing và real-time communication.

Ứng dụng trong dự án:

- **WebSocket Message Broker:** RabbitMQ được sử dụng làm STOMP message broker cho WebSocket connections, cho phép gửi real-time notifications từ server đến web clients.
- **STOMP Protocol:** Hệ thống sử dụng STOMP (Simple Text Oriented Messaging Protocol) over WebSocket để gửi messages, với các destinations như /topic/notifications (broadcast) và /user/{userId}/queue/notifications (personal).
- **Reliability:** RabbitMQ đảm bảo messages được deliver một cách đáng tin cậy, ngay cả khi client tạm thời disconnect.
- **Scalability:** RabbitMQ hỗ trợ clustering và high availability, có thể scale để xử lý nhiều connections đồng thời.

Cấu hình trong dự án:

- RabbitMQ được chạy trong Docker container với các ports:
 - 5672: AMQP port cho Spring AMQP
 - 61613: STOMP port cho WebSocket
 - 15672: Management UI port
 - 15674: WebSocket port cho STOMP
- WebSocketConfig được cấu hình để sử dụng RabbitMQ STOMP broker relay thay vì in-memory broker, cho phép scale và persistence.

f, MapStruct - Object Mapping

MapStruct là một code generator để tạo các mapper implementations giữa Java beans, giúp giảm boilerplate code khi convert giữa Entity và DTO.

Ứng dụng trong dự án:

- **Entity to DTO Mapping:** Tự động generate code để convert giữa Entity (như Product, User, Orders) và DTO (như ProductResponse, UserResponse, OrderDetailsResponse).
- **Custom Mappings:** Hỗ trợ custom mappings với annotations như @Mapping để map các fields có tên khác nhau hoặc cần xử lý đặc biệt.
- **Null Handling:** Hỗ trợ các strategies để xử lý null values như NullValuePropertyMappingStrategy.
- **Collection Mapping:** Tự động map collections và nested objects.
- **Compile-time Safety:** MapStruct generate code tại compile time, đảm bảo

type safety và phát hiện lỗi sớm.

Ví dụ trong dự án:

```
@Mapper(componentModel = "spring")
public interface ProductMapper {
    ProductResponse toProductResponse(Product product);
    List<ProductResponse> toResponseList(List<Product> products);
}
```

MapStruct sẽ tự động generate implementation class ProductMapperImpl với các methods này.

g, Lombok - Code Generation

Project Lombok là một Java library tự động generate boilerplate code như getters, setters, constructors, equals/hashCode, toString thông qua annotations.

Ứng dụng trong dự án:

- **@Getter/@Setter**: Tự động generate getters và setters cho các fields.
- **@NoArgsConstructor/@AllArgsConstructor**: Tự động generate constructors.
- **@Builder**: Tạo builder pattern để khởi tạo objects một cách linh hoạt.
- **@RequiredArgsConstructor**: Generate constructor cho các final fields, thường dùng với dependency injection.
- **@Slf4j**: Tự động tạo logger instance với tên log.
- **@Data**: Kết hợp @Getter, @Setter, @ToString, @EqualsAndHashCode, và @RequiredArgsConstructor.

Lợi ích:

- Giảm thiểu boilerplate code, code ngắn gọn và dễ đọc hơn
- Giảm khả năng lỗi khi viết getters/setters thủ công
- Tự động cập nhật equals/hashCode khi thêm/sửa fields

h, Docker và Docker Compose - Containerization

Docker là một platform để containerize applications, đóng gói ứng dụng cùng với dependencies vào một container có thể chạy trên bất kỳ môi trường nào có Docker.

Docker Compose là một tool để định nghĩa và chạy multi-container Docker applications với một file cấu hình duy nhất.

Ứng dụng trong dự án:

- **MySQL Container:** Chạy MySQL 8 trong Docker container với persistent volume để lưu trữ dữ liệu.
- **RabbitMQ Container:** Chạy RabbitMQ với STOMP plugin trong Docker container với persistent volume.
- **Spring Boot App Container:** Chạy Spring Boot application trong Maven container với volume mapping để hot reload code.
- **Service Dependencies:** Docker Compose đảm bảo các services được start theo đúng thứ tự (app phụ thuộc vào mysql và rabbitmq).
- **Health Checks:** Các containers có health checks để đảm bảo services sẵn sàng trước khi các services khác kết nối.
- **Environment Variables:** Sử dụng .env file để quản lý cấu hình một cách tập trung và bảo mật.

Lợi ích:

- **Consistency:** Đảm bảo môi trường phát triển, test, và production giống nhau
- **Isolation:** Mỗi service chạy trong container riêng, không ảnh hưởng lẫn nhau
- **Easy Deployment:** Dễ dàng deploy lên bất kỳ server nào có Docker
- **Resource Efficiency:** Containers chia sẻ OS kernel, sử dụng ít tài nguyên hơn VMs
- **Version Control:** Có thể version control cấu hình Docker và dễ dàng rollback

i, Maven - Build Tool và Dependency Management

Apache Maven là một build automation tool và dependency management tool cho Java projects.

Ứng dụng trong dự án:

- **Dependency Management:** Quản lý tất cả các dependencies trong file pom.xml, Maven tự động download và resolve conflicts.
- **Build Lifecycle:** Định nghĩa các phases như compile, test, package, install, deploy.
- **Plugin System:** Sử dụng các plugins như:
 - spring-boot-maven-plugin: Để build và run Spring Boot application
 - maven-compiler-plugin: Để compile code với Java 21 và annotation

processors (Lombok, MapStruct)

- **Multi-module Support:** Có thể tổ chức project thành nhiều modules nếu cần.

Lợi ích:

- Tự động quản lý dependencies và versions
- Standardized project structure
- Tích hợp tốt với IDEs và CI/CD tools
- Hỗ trợ nhiều build profiles cho các môi trường khác nhau

2.4 Kết luận chương

Như vậy, chương 2 đã cung cấp một cái nhìn tổng quan và chi tiết về bức tranh công nghệ được sử dụng trong đề án. Sự kết hợp giữa kiến trúc RESTful API linh hoạt, sức mạnh của Spring Boot ở Backend, tính tương tác cao của Vue.js và Android Kotlin ở Frontend, cùng sự ổn định của MySQL và các công nghệ hỗ trợ hiện đại, tất cả tạo nên một nền tảng vững chắc cho hệ thống.

Việc hiểu rõ các công nghệ này không chỉ giúp đảm bảo quá trình phát triển diễn ra thuận lợi mà còn là cơ sở để tối ưu hóa hiệu năng, bảo mật và khả năng mở rộng của sản phẩm. Với nền tảng công nghệ đã được xác lập, chương tiếp theo sẽ đi sâu vào việc phân tích yêu cầu nghiệp vụ chi tiết và thiết kế hệ thống cụ thể thông qua các biểu đồ UML và sơ đồ cơ sở dữ liệu.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Giới thiệu chương

Trên cơ sở nền tảng lý thuyết và công nghệ đã được trình bày ở Chương 2, Chương 3 sẽ đi sâu vào việc phân tích và thiết kế chi tiết cho hệ thống "Quản lý bán hàng và sửa chữa điện thoại". Mục tiêu của chương này là chuyển đổi các yêu cầu nghiệp vụ thực tế thành các mô hình kỹ thuật cụ thể, làm cơ sở cho việc lập trình và triển khai.

Nội dung chương bao gồm việc xác định các tác nhân và yêu cầu phi chức năng, mô hình hóa các chức năng hệ thống thông qua biểu đồ Use Case, thiết kế cấu trúc dữ liệu (ERD, Database Schema) và chi tiết hóa các luồng xử lý thông qua biểu đồ trình tự (Sequence Diagram) và biểu đồ lớp (Class Diagram).

3.2 Phân tích yêu cầu hệ thống

3.2.1 Xác định các tác nhân (Actors)

Dựa trên hệ thống phân quyền được triển khai, các tác nhân trong hệ thống bao gồm:

- **Khách hàng (Customer/User):** Người mua hàng hoặc người có nhu cầu sửa chữa máy. Đây là đối tượng chính của hệ thống, có thể đăng ký tài khoản, mua sắm sản phẩm, đặt lịch sửa chữa, và theo dõi đơn hàng. Khách hàng có vai trò USER trong hệ thống và có thể truy cập các chức năng công khai và các chức năng dành cho người dùng đã đăng nhập.
- **Nhân viên bán hàng (Sales Staff):** Xử lý đơn hàng online/offline, quản lý sản phẩm, và hỗ trợ khách hàng. Nhân viên bán hàng có thể có vai trò SHOP_MANAGER với quyền MANAGE_PRODUCT để quản lý sản phẩm và đơn hàng.
- **Kỹ thuật viên (Technician):** Người trực tiếp kiểm tra và sửa chữa thiết bị. Kỹ thuật viên có thể xem danh sách đơn sửa chữa được phân công và cập nhật trạng thái sửa chữa. Trong hệ thống hiện tại, kỹ thuật viên có thể được quản lý thông qua module WorkSchedule để phân công công việc.
- **Quản trị viên (Admin):** Quản lý toàn bộ hệ thống, bao gồm quản lý tài khoản người dùng, phân quyền, quản lý sản phẩm, đơn hàng, và các cấu hình hệ thống. Admin có vai trò ADMIN với đầy đủ các quyền như MANAGE_PRODUCT, MANAGE_ROLE, MANAGE_STAFF.
- **Quản lý nhân viên (Employee Manager):** Quản lý nhân viên và lịch làm việc. Có vai trò EMPLOYEE_MANAGER với quyền MANAGE_STAFF để quản lý

thông tin nhân viên và phân công lịch làm việc.

3.2.2 Yêu cầu phi chức năng

- **Bảo mật thông tin khách hàng:**

- Hệ thống sử dụng JWT (JSON Web Token) với thuật toán HS512 để xác thực người dùng, đảm bảo tính toàn vẹn và bảo mật của token.
- Mật khẩu được mã hóa bằng BCrypt với strength 10 trước khi lưu vào database.
- Token blacklist mechanism được triển khai để ngăn chặn việc sử dụng lại token sau khi logout.
- Phân quyền chi tiết với Role-Based Access Control (RBAC) và Permission-Based Access Control (PBAC), đảm bảo người dùng chỉ có thể truy cập các tài nguyên được phép.
- CORS được cấu hình để chỉ cho phép các origin được phép truy cập API.

- **Hệ thống phản hồi nhanh (độ trễ thấp):**

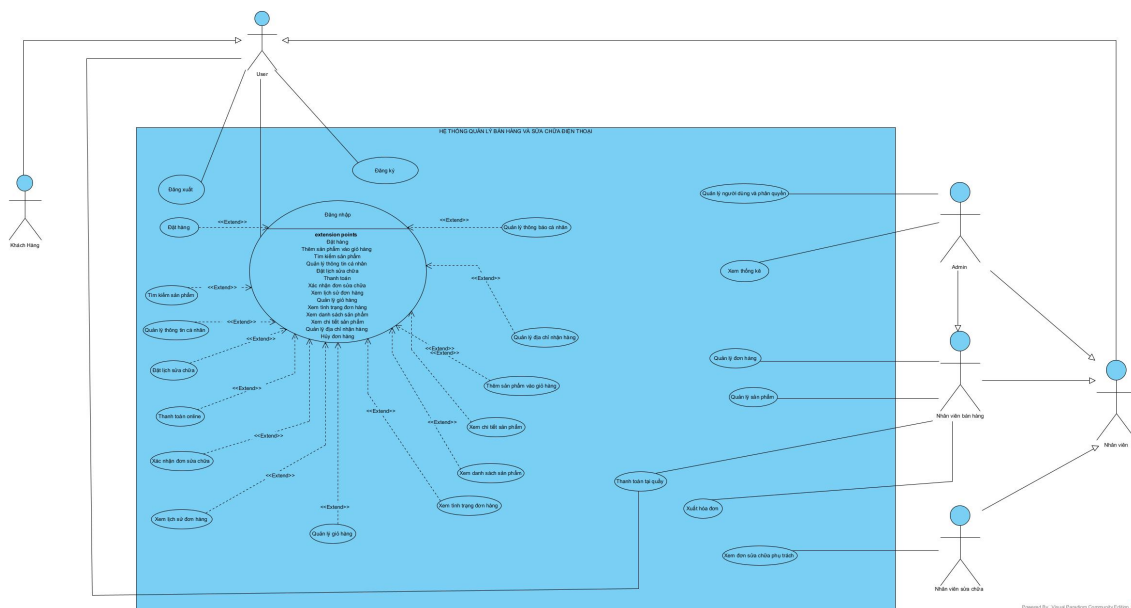
- Sử dụng JPA/Hibernate với lazy loading để tối ưu hóa các truy vấn database, chỉ load dữ liệu khi cần thiết.
- Response format thống nhất với ApiResponse wrapper giúp client xử lý nhanh chóng.
- WebSocket với RabbitMQ STOMP broker cho real-time notifications, đảm bảo thông báo được gửi ngay lập tức mà không cần polling.
- Cloudinary CDN cho việc phân phối hình ảnh, giảm thời gian load trang.
- Database indexing được áp dụng trên các trường thường xuyên được query như `user.email`, `product.status`, `orders.user_id`.

- **Giao diện thân thiện trên mobile:**

- RESTful API được thiết kế để dễ dàng tích hợp với mobile app, với response format JSON chuẩn và error handling rõ ràng.
- Firebase Cloud Messaging (FCM) được tích hợp để gửi push notifications đến mobile devices, đảm bảo người dùng luôn được thông báo về trạng thái đơn hàng và các sự kiện quan trọng.
- API hỗ trợ pagination và filtering để tối ưu hóa việc load dữ liệu trên mobile với bandwidth hạn chế.
- Hỗ trợ offline mode thông qua việc lưu trữ dữ liệu local trên mobile app

- Kiến trúc phân lớp rõ ràng (Controller → Service → Repository) giúp dễ dàng thêm tính năng mới và bảo trì code.
- Sử dụng MapStruct để tự động generate mapper code, giảm boilerplate và đảm bảo type safety.
- Docker containerization cho phép dễ dàng deploy và scale hệ thống.
- Environment variables được sử dụng để quản lý cấu hình, dễ dàng thay đổi giữa các môi trường khác nhau.

Mô hình Use Case tổng quát cung cấp cái nhìn toàn cảnh về các tác nhân (Actors) và các nhóm chức năng chính của hệ thống.

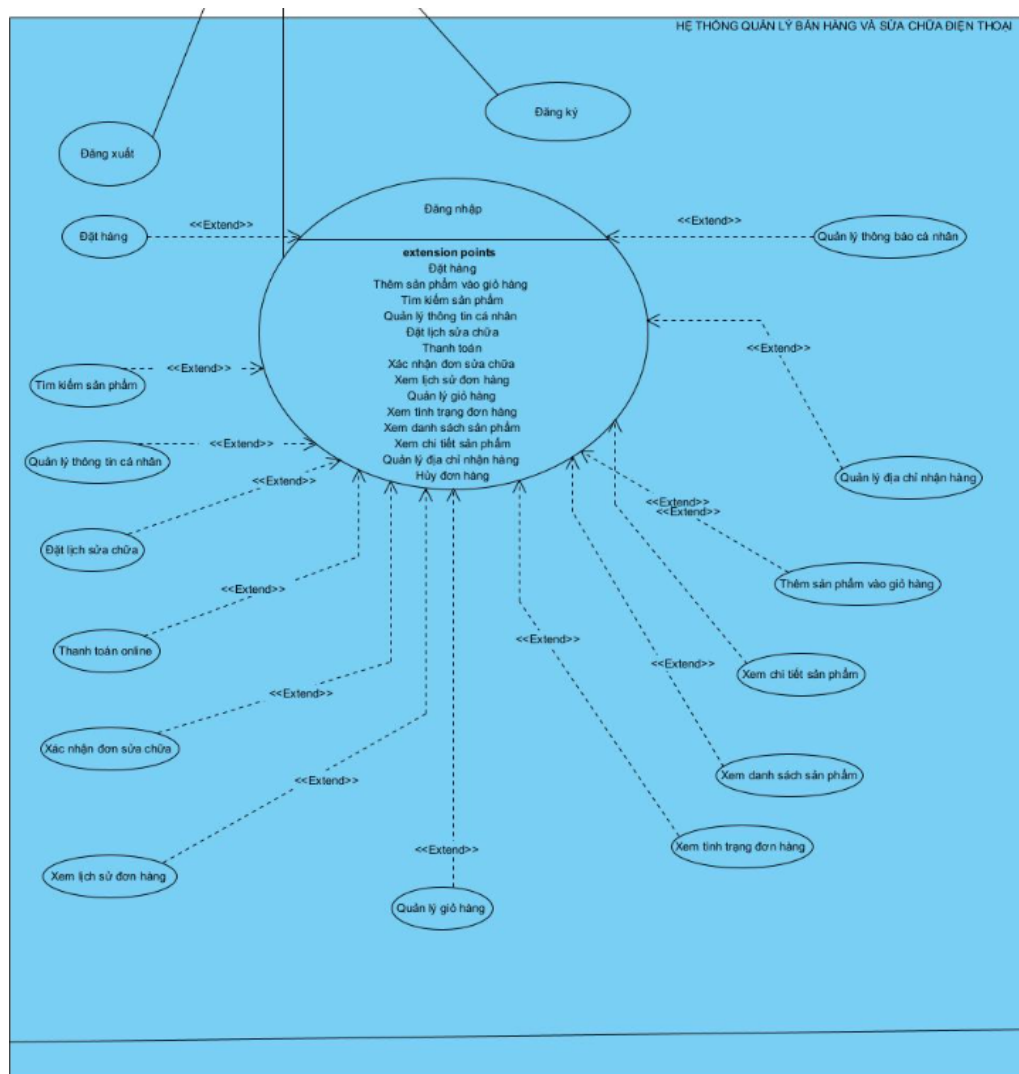


- **Phân hệ Khách hàng (Client App):** Bao gồm các chức năng xem, tìm kiếm, đặt hàng và theo dõi sửa chữa.
- **Phân hệ Quản trị (Web Admin):** Dành cho nhân viên và quản lý để vận hành hệ thống, quản lý kho và xử lý đơn hàng.

3.3.2 Chi tiết các nhóm chức năng

a, Nhóm chức năng dành cho Khách hàng

Người dùng cuối sử dụng ứng dụng di động để tương tác với hệ thống. Các chức năng trọng tâm bao gồm:

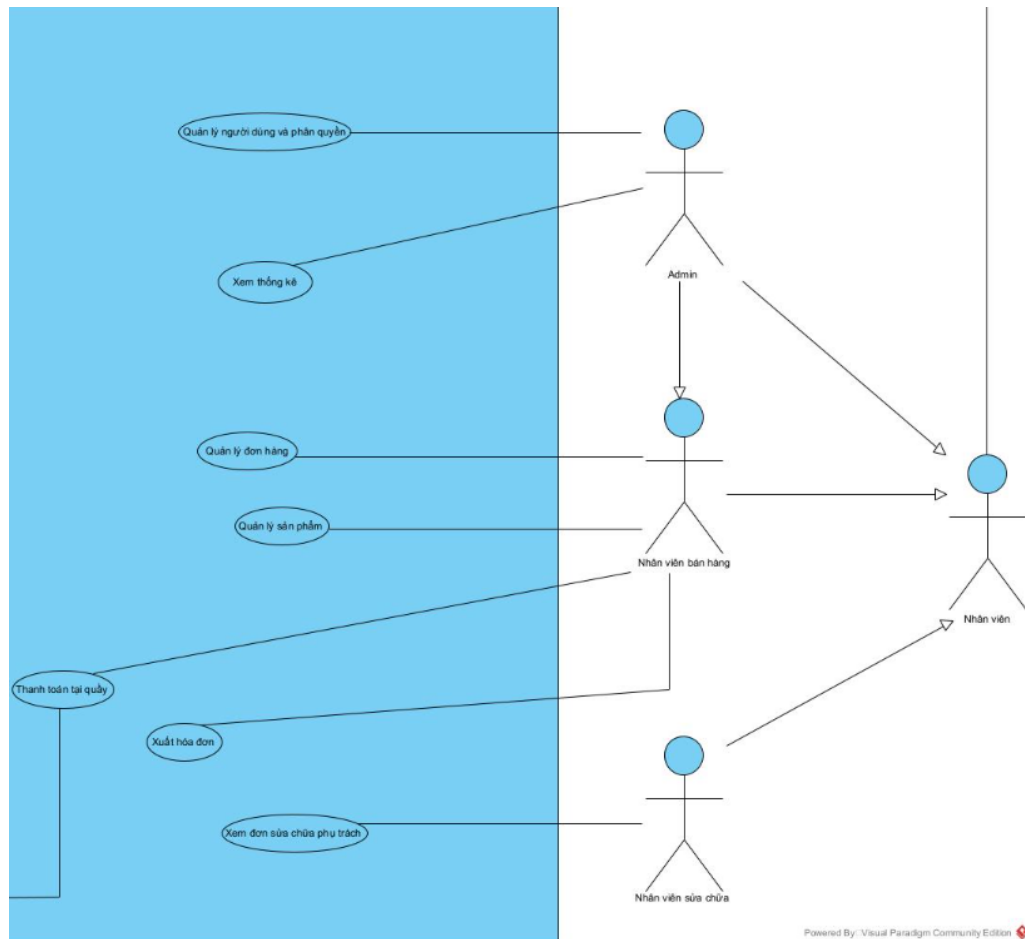


Hình 3.2: Sơ đồ Use Case phân hệ Khách hàng

- **Xác thực:** Đăng nhập, Đăng ký, Quản lý thông tin cá nhân.
- **Mua hàng:** Xem danh sách sản phẩm, Tìm kiếm, Thêm vào giỏ, Thanh toán Online (ZaloPay).
- **Dịch vụ:** Đặt lịch sửa chữa, Theo dõi trạng thái thiết bị theo thời gian thực.

b, Nhóm chức năng Quản lý (Admin/Staff)

Phân hệ quản trị trên nền tảng Web cung cấp công cụ cho đội ngũ vận hành:



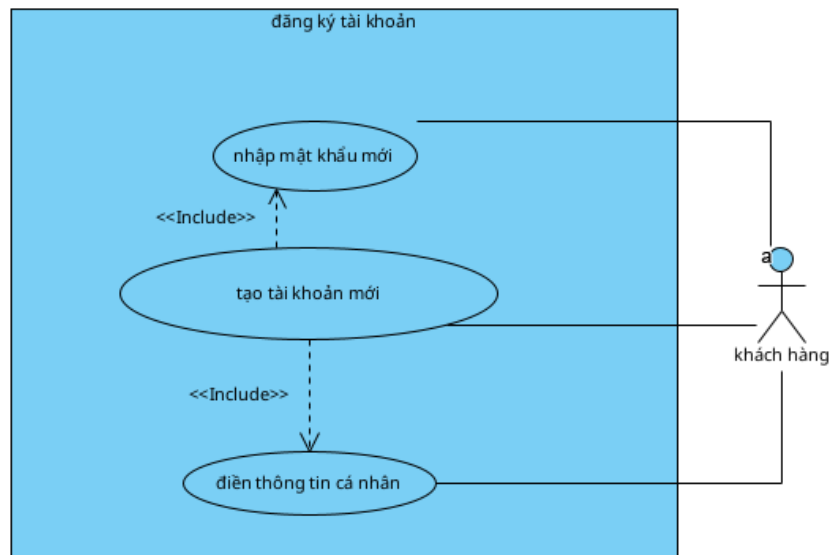
Hình 3.3: Sơ đồ Use Case phân hệ Quản trị viên

- **Quản lý Sản phẩm:** Thêm mới, cập nhật giá, quản lý tồn kho (Import/Export).
- **Xử lý Đơn hàng:** Duyệt đơn mua bán, cập nhật tiến độ sửa chữa, gán nhân viên kỹ thuật.
- **Thông kê & Báo cáo:** Xem biểu đồ doanh thu, thống kê sản phẩm bán chạy và hiệu suất làm việc.

3.3.3 Đặc tả các Use Case chính

a, Tác nhân: Khách hàng (Customer/User)

1. Use Case: Đăng ký tài khoản



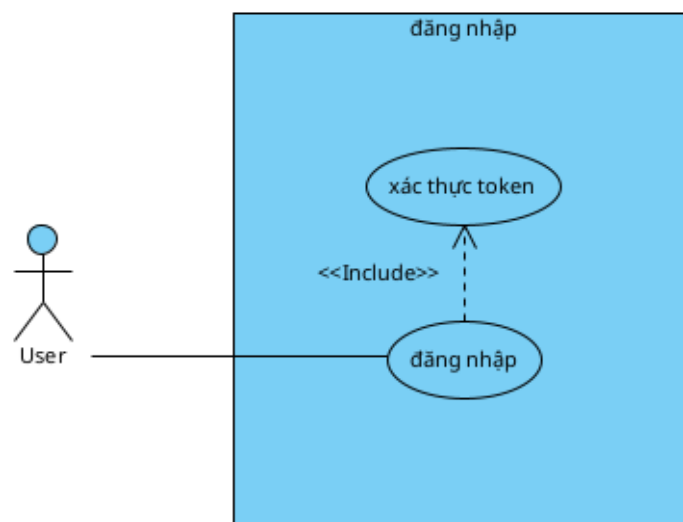
Hình 3.4: Sơ đồ Use Case - Đăng ký tài khoản

- **Mô tả:** Use case này cho phép khách hàng tạo tài khoản mới. Khách hàng cung cấp số điện thoại, mật khẩu, họ tên và email. Hệ thống kiểm tra tính duy nhất, mã hóa mật khẩu, gán quyền USER và lưu vào cơ sở dữ liệu.

Tên Use Case	Đăng ký tài khoản
Tác nhân	Khách hàng
Tiền điều kiện	• Khách hàng chưa có tài khoản trong hệ thống. • Hệ thống hoạt động bình thường, database kết nối thành công. • Role "USER" đã được khởi tạo trong hệ thống.
Hậu điều kiện	• Tài khoản mới được tạo thành công và lưu vào database. • Password được mã hóa bằng BCrypt. • Role "USER" được gán tự động. • Khách hàng nhận được thông tin tài khoản đã tạo.
Kịch bản chính	1. Khách hàng truy cập trang đăng ký, điền SĐT, mật khẩu, họ tên, email. 2. Khách hàng nhấn nút "Đăng ký". 3. Hệ thống kiểm tra SĐT và Email đã tồn tại hay chưa. 4. Nếu chưa tồn tại, hệ thống tạo tài khoản mới. 5. Hệ thống mã hóa mật khẩu (BCrypt) và gán quyền "USER". 6. Hệ thống lưu thông tin tài khoản vào cơ sở dữ liệu. 7. Hệ thống trả về thông tin tài khoản và thông báo thành công. 8. Khách hàng có thể dùng tài khoản vừa tạo để đăng nhập.
Ngoại lệ	• Số điện thoại hoặc Email đã tồn tại. • Dữ liệu nhập vào sai định dạng hoặc thiếu dữ liệu.

Bảng 3.1: Kịch bản Use Case Đăng ký tài khoản

2. Use Case: Đăng nhập



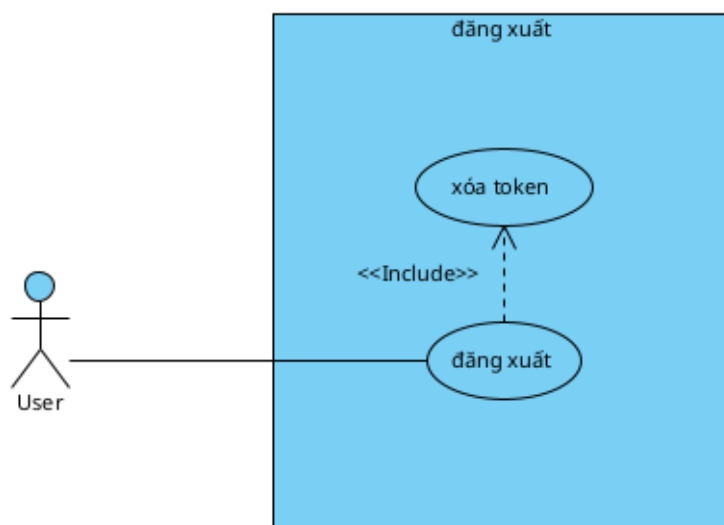
Hình 3.5: Sơ đồ Use Case - Đăng nhập

- **Mô tả:** Use case này cho phép User truy cập vào hệ thống bằng số điện thoại và mật khẩu. Nếu hợp lệ, hệ thống trả về JWT token để xác thực và dọn dẹp các device token cũ (FCM).

Tên Use Case	Đăng nhập
Tác nhân	Khách hàng, Admin, Nhân viên
Tiền điều kiện	• User đã có tài khoản. • Hệ thống hoạt động bình thường.
Hậu điều kiện	• User nhận được JWT token có thời hạn. • Các thiết bị cũ không còn nhận được thông báo (device token cũ bị xóa).
Kịch bản chính	1. User nhập số điện thoại và mật khẩu, nhấn "Đăng nhập". 2. Hệ thống tìm tài khoản và kiểm tra mật khẩu. 3. Nếu mật khẩu đúng, hệ thống xóa các device token cũ. 4. Hệ thống tạo JWT token và trả về kết quả thành công. 5. Giao diện lưu token và chuyển User vào màn hình chính.
Ngoại lệ	• Tài khoản không tồn tại hoặc mật khẩu sai. • Lỗi khi xóa device token cũ.

Bảng 3.2: Kịch bản Use Case Đăng nhập

3. Use Case: Đăng xuất



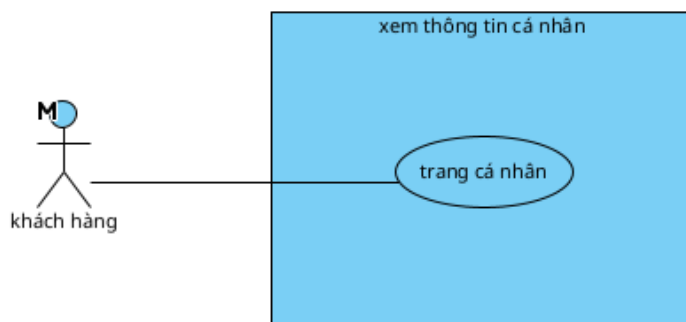
Hình 3.6: Sơ đồ Use Case - Đăng xuất

- **Mô tả:** Cho phép User thoát khỏi hệ thống an toàn. Hệ thống vô hiệu hóa JWT token hiện tại (blacklist) và xóa FCM device token để ngừng nhận thông báo đẩy.

Tên Use Case	Đăng xuất
Tác nhân	Khách hàng, Admin, Nhân viên
Tiền điều kiện	• User đang đăng nhập và có token hợp lệ.
Hậu điều kiện	• Token hiện tại bị vô hiệu hóa (Blacklist). • FCM device token của user bị xóa khỏi hệ thống.
Kịch bản chính	1. User chọn chức năng "Đăng xuất". 2. Hệ thống kiểm tra tính hợp lệ của JWT token. 3. Hệ thống đưa token vào danh sách blacklist (InvalidatedToken). 4. Hệ thống xóa toàn bộ FCM device token của user. 5. Hệ thống trả về kết quả thành công. 6. Giao diện xóa token ở client và đưa về màn hình public.
Ngoại lệ	• Token không hợp lệ, hết hạn hoặc đã bị blacklist. • Lỗi không xóa được FCM token.

Bảng 3.3: Kịch bản Use Case Đăng xuất

4. Use Case: Xem thông tin cá nhân



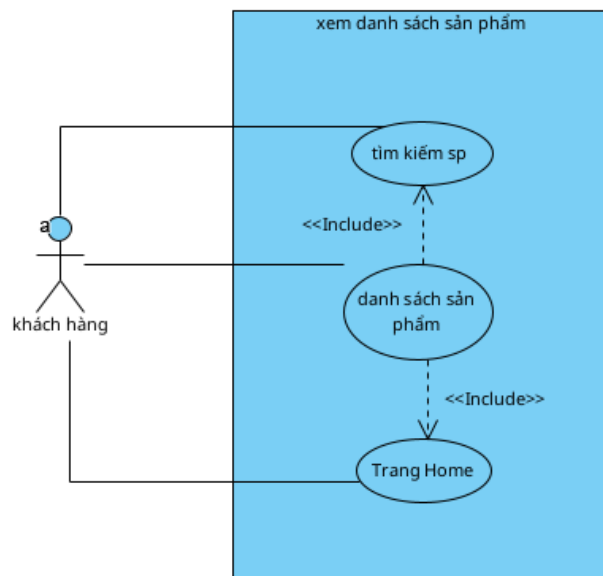
Hình 3.7: Sơ đồ Use Case - Xem thông tin cá nhân

- **Mô tả:** Khách hàng xem lại thông tin tài khoản của chính mình (họ tên, SĐT, email, địa chỉ, vai trò) đang lưu trong hệ thống.

Tên Use Case	Xem thông tin cá nhân
Tác nhân	Khách hàng
Tiền điều kiện	• Khách hàng đã đăng nhập thành công.
Hậu điều kiện	• Thông tin cá nhân được hiển thị. • Không làm thay đổi dữ liệu.
Kịch bản chính	1. Khách hàng truy cập khu vực "Tài khoản". 2. Hệ thống xác thực token và lấy định danh người dùng. 3. Hệ thống truy vấn cơ sở dữ liệu lấy thông tin user. 4. Hệ thống trả dữ liệu về giao diện để hiển thị.
Ngoại lệ	• Token không hợp lệ. • Không tìm thấy thông tin người dùng trong DB.

Bảng 3.4: Kịch bản Use Case Xem thông tin cá nhân

5. Use Case: Xem danh sách sản phẩm



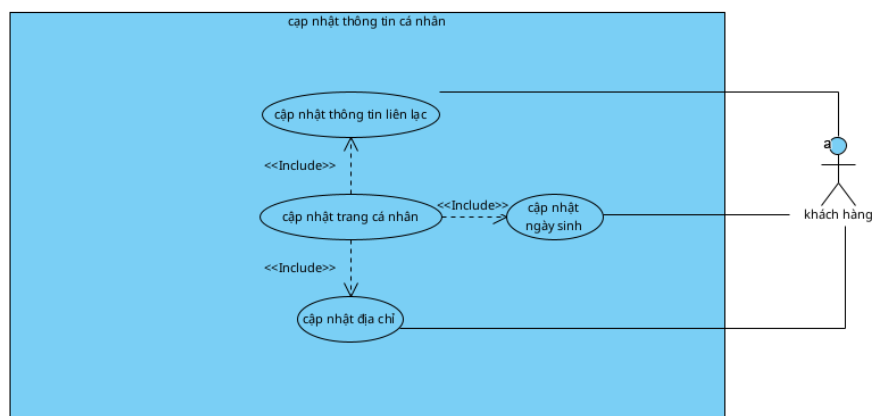
Hình 3.8: Sơ đồ Use Case - Xem danh sách sản phẩm

- **Mô tả:** Cho phép khách hàng xem danh sách các sản phẩm đang được bày bán. Danh sách hiển thị thông tin cơ bản như tên, ảnh đại diện và giá tóm tắt.

Tên Use Case	Xem danh sách sản phẩm
Tác nhân	Khách hàng
Tiền điều kiện	• Hệ thống và database hoạt động bình thường.
Hậu điều kiện	Danh sách sản phẩm được hiển thị trên giao diện.
Kịch bản chính	1. Khách hàng truy cập trang Home. 2. Hệ thống truy vấn các sản phẩm đang bán, sắp xếp theo thời gian. 3. Hệ thống chuẩn bị thông tin hiển thị (tên, ảnh, giá tóm tắt). 4. Hệ thống trả dữ liệu về giao diện. 5. Giao diện hiển thị danh sách sản phẩm.
Ngoại lệ	• Không có sản phẩm nào đang bán. • Lỗi kết nối cơ sở dữ liệu.

Bảng 3.5: Kịch bản Use Case Xem danh sách sản phẩm

6. Use Case: Cập nhật thông tin cá nhân



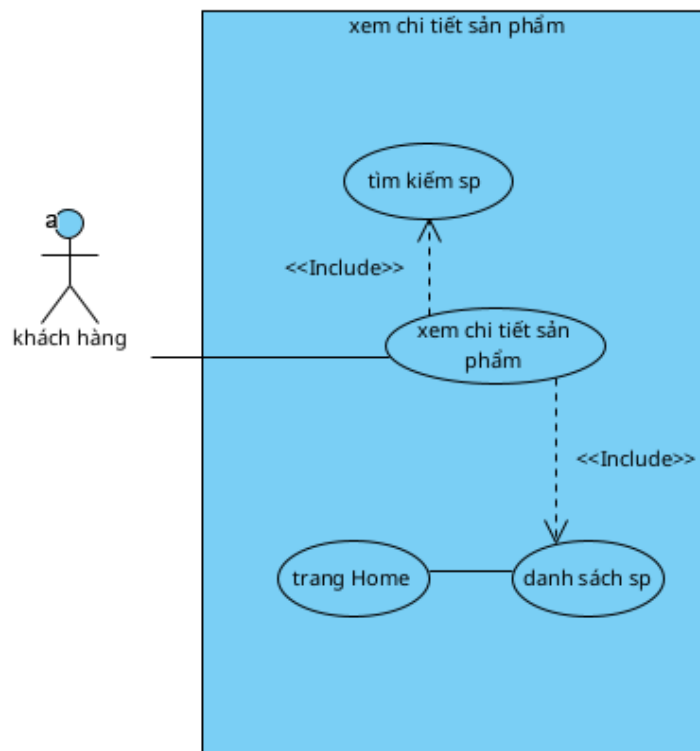
Hình 3.9: Sơ đồ Use Case - Cập nhật thông tin cá nhân

- **Mô tả:** Cho phép khách hàng chỉnh sửa thông tin cá nhân của chính mình. Hệ thống xác định người dùng qua token để đảm bảo tính bảo mật và quyền sở hữu.

Tên Use Case	Cập nhật thông tin cá nhân
Tác nhân	Khách hàng
Tiền điều kiện	• Khách hàng đã đăng nhập thành công.
Hậu điều kiện	• Thông tin mới được cập nhật trong database. • Giao diện hiển thị thông tin sau cập nhật.
Kịch bản chính	1. Khách hàng chọn "Chỉnh sửa thông tin", nhập thay đổi và nhấn "Lưu". 2. Hệ thống xác thực token và lấy định danh người dùng. 3. Hệ thống cập nhật các trường thông tin thay đổi vào database. 4. Hệ thống trả về thông tin người dùng sau cập nhật. 5. Giao diện hiển thị thông báo thành công.
Ngoại lệ	• Token không hợp lệ hoặc không tìm thấy người dùng. • Dữ liệu cập nhật không hợp lệ.

Bảng 3.6: Kịch bản Use Case Cập nhật thông tin cá nhân

7. Use Case: Xem chi tiết sản phẩm



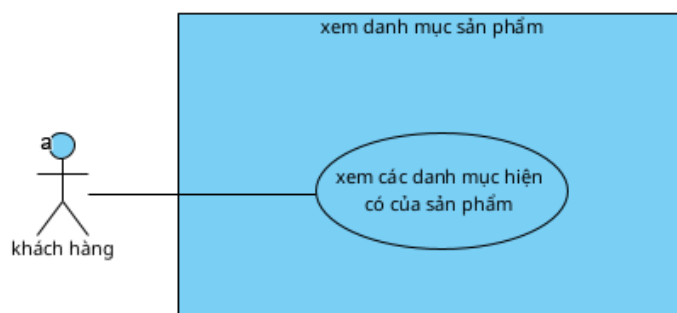
Hình 3.10: Sơ đồ Use Case - Xem chi tiết sản phẩm

- **Mô tả:** Use case này cho phép khách hàng xem đầy đủ thông tin của một sản phẩm cụ thể (tên, ảnh, mô tả, giá) và danh sách các biến thể (màu sắc, cấu hình) trước khi quyết định mua.

Tên Use Case	Xem chi tiết sản phẩm
Tác nhân	Khách hàng
Tiền điều kiện	- Sản phẩm cần xem tồn tại trong hệ thống. - Hệ thống hoạt động bình thường.
Hậu điều kiện	Thông tin chi tiết và các biến thể sản phẩm được hiển thị.
Kịch bản chính	- Khách hàng chọn một sản phẩm từ danh sách. - Giao diện gửi yêu cầu lấy thông tin chi tiết sản phẩm. - Hệ thống kiểm tra sự tồn tại của sản phẩm. - Hệ thống lấy thông tin cơ bản, danh sách ảnh và các biến thể. - Hệ thống trả dữ liệu chi tiết về cho giao diện. - Giao diện hiển thị trang chi tiết sản phẩm.
Ngoại lệ	- Không tìm thấy sản phẩm. - Lỗi kết nối cơ sở dữ liệu.

Bảng 3.7: Kịch bản Use Case Xem chi tiết sản phẩm

8. Use Case: Xem danh mục sản phẩm



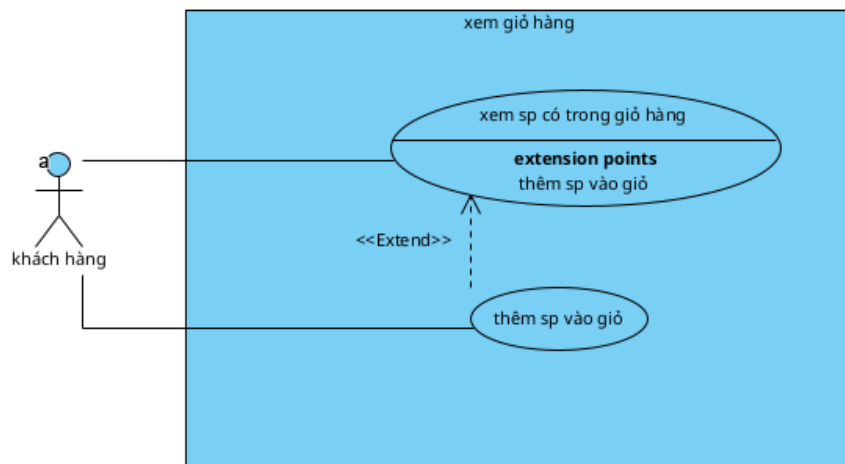
Hình 3.11: Sơ đồ Use Case - Xem danh mục sản phẩm

- **Mô tả:** Cho phép khách hàng xem danh sách các danh mục sản phẩm (Ví dụ: Điện thoại, Phụ kiện) để thuận tiện cho việc lọc và tìm kiếm.

Tên Use Case	Xem danh mục sản phẩm
Tác nhân	Khách hàng
Tiền điều kiện	Danh mục đã được tạo sẵn trong hệ thống.
Hậu điều kiện	Danh sách danh mục được hiển thị cho khách hàng.
Kịch bản chính	- Khách hàng truy cập phần "Danh mục" trên giao diện. - Hệ thống truy vấn toàn bộ danh mục hiện có. - Hệ thống trả về danh sách (mã, tên danh mục). - Giao diện hiển thị danh sách danh mục. - Khách hàng chọn một danh mục để xem sản phẩm tương ứng.
Ngoại lệ	Không có danh mục nào trong hệ thống.

Bảng 3.8: Kịch bản Use Case Xem danh mục sản phẩm

9. Use Case: Xem giỏ hàng



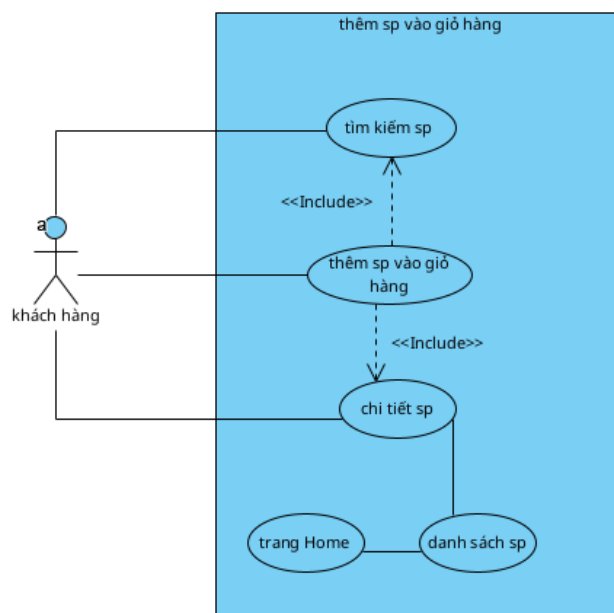
Hình 3.12: Sơ đồ Use Case - Xem giỏ hàng

- **Mô tả:** Cho phép khách hàng xem lại các sản phẩm đã thêm vào giỏ hàng, bao gồm tên sản phẩm, thuộc tính, số lượng, đơn giá và tổng tiền tạm tính.

Tên Use Case	Xem giỏ hàng
Tác nhân	Khách hàng
Tiền điều kiện	Khách hàng đã đăng nhập và có token hợp lệ.
Hậu điều kiện	Danh sách sản phẩm trong giỏ hàng được hiển thị.
Kịch bản chính	- Khách hàng truy cập màn hình "Giỏ hàng". - Hệ thống xác định khách hàng qua token. - Hệ thống lấy danh sách các mục trong giỏ hàng từ DB. - Hệ thống tính toán và trả về dữ liệu hiển thị. - Giao diện hiển thị chi tiết giỏ hàng hoặc thông báo nếu rỗng.
Ngoại lệ	- Token không hợp lệ. - Không tìm thấy người dùng.

Bảng 3.9: Kịch bản Use Case Xem giỏ hàng

10. Use Case: Thêm sản phẩm vào giỏ hàng



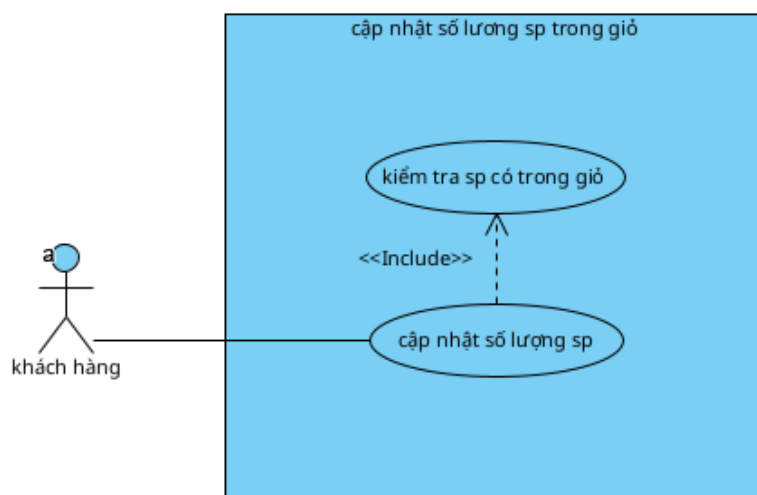
Hình 3.13: Sơ đồ Use Case - Thêm sản phẩm vào giỏ hàng

- Mô tả:** Cho phép khách hàng thêm một lựa chọn sản phẩm vào giỏ. Nếu sản phẩm đã có trong giỏ, hệ thống sẽ tự động tăng số lượng lên 1.

Tên Use Case	Thêm sản phẩm vào giỏ hàng
Tác nhân	Khách hàng
Tiền điều kiện	- Khách hàng đã đăng nhập. - Sản phẩm/Biến thể được chọn phải tồn tại.
Hậu điều kiện	- Mục giỏ hàng được tạo mới hoặc cập nhật số lượng. - Giỏ hàng được lưu vào cơ sở dữ liệu.
Kịch bản chính	- Khách hàng chọn thuộc tính sản phẩm và nhấn "Thêm vào giỏ". - Hệ thống xác thực token và kiểm tra sản phẩm. - Hệ thống kiểm tra sản phẩm đã có trong giỏ hàng chưa. - Nếu chưa có: Tạo mục mới với số lượng = 1. - Nếu đã có: Tăng số lượng hiện tại lên 1. - Hệ thống lưu thay đổi và trả về thông tin giỏ hàng mới.
Ngoại lệ	- Sản phẩm không tồn tại. - Token lỗi hoặc hết hạn.

Bảng 3.10: Kịch bản Use Case Thêm sản phẩm vào giỏ hàng

11. Use Case: Cập nhật số lượng sản phẩm trong giỏ



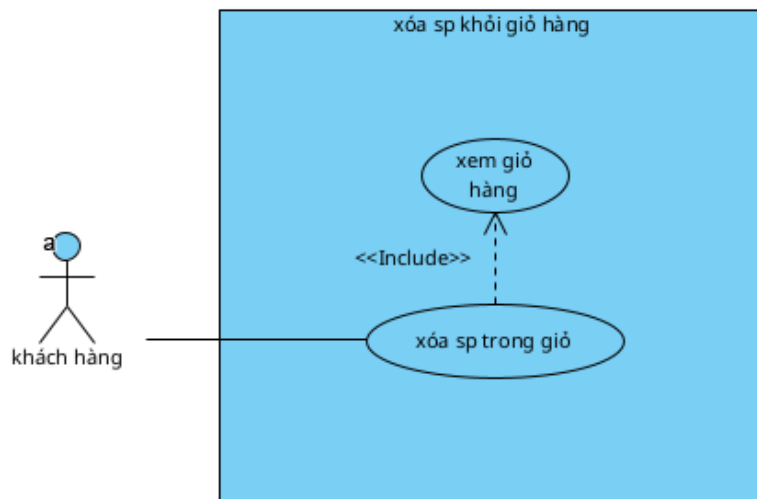
Hình 3.14: Sơ đồ Use Case - Cập nhật số lượng sản phẩm

- **Mô tả:** Cho phép khách hàng tăng hoặc giảm số lượng của một sản phẩm trong giỏ hàng. Hệ thống sẽ cập nhật lại tổng giá tiền tương ứng.

Tên Use Case	Cập nhật số lượng sản phẩm
Tác nhân	Khách hàng
Tiền điều kiện	- Mục giỏ hàng tồn tại và thuộc về khách hàng. - Số lượng mới hợp lệ (nguyên dương).
Hậu điều kiện	- Số lượng và giá của mục giỏ hàng được cập nhật. - Tổng tiền giỏ hàng thay đổi theo.
Kịch bản chính	- Khách hàng nhấn nút tăng/giảm số lượng trong giỏ hàng. - Hệ thống kiểm tra quyền sở hữu mục giỏ hàng. - Hệ thống cập nhật số lượng mới. - Hệ thống tính lại giá dựa trên giá hiện tại của sản phẩm. - Giao diện hiển thị số lượng và tổng tiền mới.
Ngoại lệ	- Không tìm thấy mục giỏ hàng. - Khách hàng không phải chủ sở hữu.

Bảng 3.11: Kịch bản Use Case Cập nhật số lượng sản phẩm

12. Use Case: Xóa sản phẩm khỏi giỏ hàng



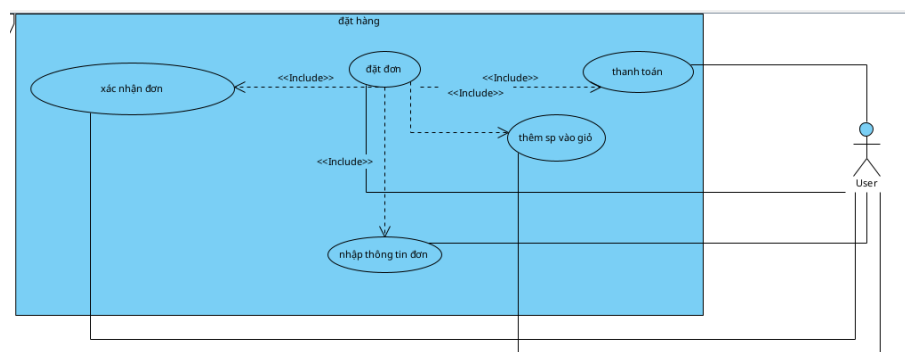
Hình 3.15: Sơ đồ Use Case - Xóa sản phẩm khỏi giỏ hàng

- **Mô tả:** Cho phép khách hàng loại bỏ hoàn toàn một mục sản phẩm ra khỏi giỏ hàng khi không còn ý định mua.

Tên Use Case	Xóa sản phẩm khỏi giỏ hàng
Tác nhân	Khách hàng
Tiền điều kiện	Mục giỏ hàng tồn tại.
Hậu điều kiện	Mục sản phẩm bị xóa khỏi cơ sở dữ liệu giỏ hàng.
Kịch bản chính	- Khách hàng chọn sản phẩm cần xóa và nhấn nút xóa. - Hệ thống tìm mục giỏ hàng và kiểm tra quyền sở hữu. - Hệ thống xóa mục giỏ hàng khỏi database. - Hệ thống trả về kết quả thành công. - Giao diện cập nhật lại danh sách và tổng tiền.
Ngoại lệ	- Không tìm thấy mục giỏ hàng. - Không phải chủ sở hữu.

Bảng 3.12: Kịch bản Use Case Xóa sản phẩm khỏi giỏ hàng

13. Use Case: Đặt hàng



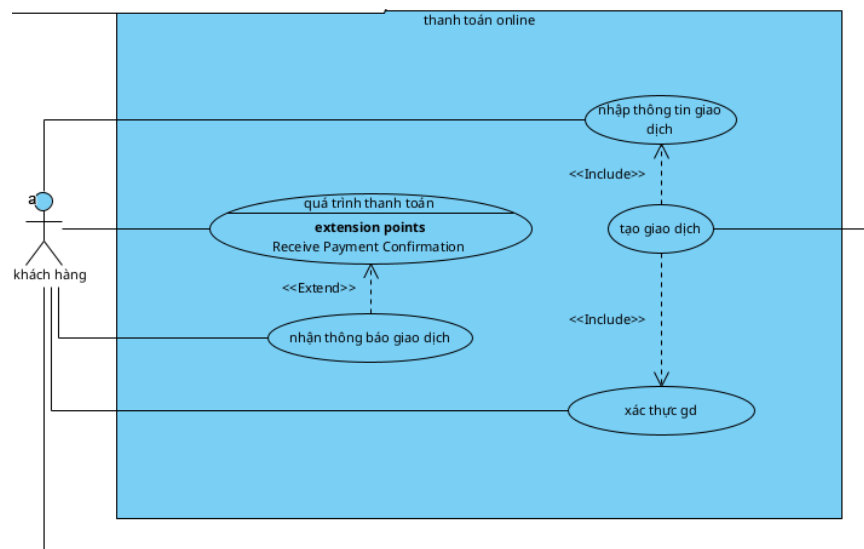
Hình 3.16: Sơ đồ Use Case - Đặt hàng

- Mô tả:** Khách hàng tạo đơn hàng mới từ các sản phẩm trong giỏ. Hệ thống ghi nhận thông tin, tính tổng tiền và chuyển các sản phẩm từ giỏ hàng sang đơn hàng.

Tên Use Case	Đặt hàng
Tác nhân	Khách hàng
Tiền điều kiện	- Giỏ hàng có ít nhất 1 sản phẩm. - Khách hàng đã đăng nhập.
Hậu điều kiện	- Đơn hàng mới được tạo. - Các sản phẩm đã đặt bị xóa khỏi giỏ hàng.
Kịch bản chính	- Khách hàng kiểm tra thông tin và nhấn "Mua hàng". - Hệ thống tạo đơn hàng với thông tin khách hàng cung cấp. - Hệ thống duyệt danh sách sản phẩm, tạo chi tiết đơn hàng. - Hệ thống lấy giá bán hiện tại và tính tổng tiền đơn hàng. - Hệ thống xóa các sản phẩm tương ứng trong giỏ hàng. - Hệ thống lưu đơn hàng và chuyển sang thanh toán.
Ngoại lệ	- Sản phẩm không còn tồn tại. - Lỗi kết nối hệ thống.

Bảng 3.13: Kịch bản Use Case Đặt hàng

14. Use Case: Thanh toán online



Hình 3.17: Sơ đồ Use Case - Thanh toán online

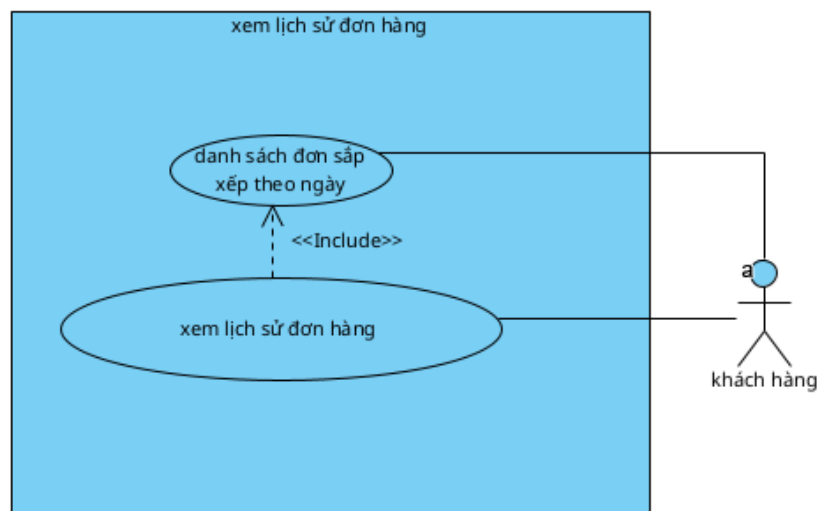
- **Mô tả:** Cho phép khách hàng thanh toán đơn hàng thông qua cổng ZaloPay.

Hệ thống tạo giao dịch, chuyển hướng người dùng và cập nhật trạng thái khi nhận phản hồi.

Tên Use Case	Thanh toán online
Tác nhân	Khách hàng
Tiền điều kiện	- Đơn hàng chưa thanh toán. - Cổng thanh toán hoạt động bình thường.
Hậu điều kiện	Trạng thái đơn hàng cập nhật thành "Đã thanh toán" (nếu thành công).
Kịch bản chính	- Giao diện gửi yêu cầu thanh toán cho đơn hàng. - Hệ thống tạo mã giao dịch và gửi yêu cầu sang ZaloPay. - Khách hàng được chuyển hướng sang trang ZaloPay để thanh toán. - ZaloPay gọi callback báo kết quả về hệ thống. - Hệ thống cập nhật trạng thái đơn hàng (Đã thanh toán/Chờ xử lý). - Giao diện hiển thị kết quả cho khách hàng.
Ngoại lệ	- Đơn hàng đã được thanh toán trước đó. - Giao dịch bị hủy hoặc thất bại.

Bảng 3.14: Kịch bản Use Case Thanh toán online

15. Use Case: Xem lịch sử đơn hàng



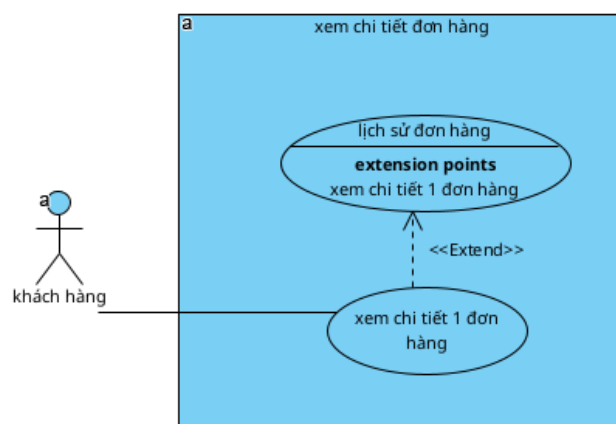
Hình 3.18: Sơ đồ Use Case - Xem lịch sử đơn hàng

- **Mô tả:** Khách hàng xem danh sách toàn bộ các đơn hàng đã tạo (mua hàng hoặc sửa chữa) để theo dõi trạng thái và lịch sử giao dịch.

Tên Use Case	Xem lịch sử đơn hàng
Tác nhân	Khách hàng
Tiền điều kiện	• Khách hàng đã đăng nhập.
Hậu điều kiện	Danh sách đơn hàng được hiển thị.
Kịch bản chính	1. Khách hàng chọn mục "Lịch sử đơn hàng". 2. Hệ thống truy vấn các đơn hàng thuộc về khách hàng. 3. Hệ thống trả về danh sách đơn (mã, ngày đặt, trạng thái, tổng tiền). 4. Giao diện hiển thị danh sách cho khách hàng.
Ngoại lệ	• Không tìm thấy đơn hàng nào.

Bảng 3.15: Kịch bản Use Case Xem lịch sử đơn hàng

16. Use Case: Xem chi tiết đơn hàng



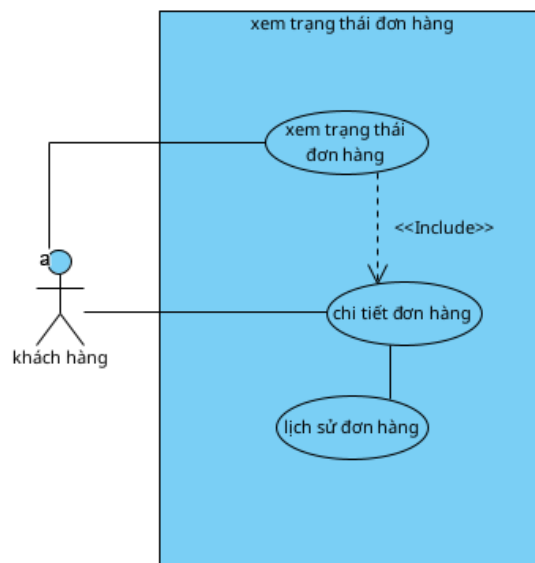
Hình 3.19: Sơ đồ Use Case - Xem chi tiết đơn hàng

- **Mô tả:** Xem thông tin đầy đủ của một đơn hàng cụ thể, bao gồm thông tin người nhận, danh sách sản phẩm chi tiết và trạng thái xử lý.

Tên Use Case	Xem chi tiết đơn hàng
Tác nhân	Khách hàng
Tiền điều kiện	Đơn hàng tồn tại và thuộc về khách hàng.
Hậu điều kiện	Chi tiết đơn hàng được hiển thị.
Kịch bản chính	- Khách hàng chọn xem một đơn hàng cụ thể. - Hệ thống xác thực và truy vấn chi tiết đơn hàng. - Hệ thống trả về thông tin chung, danh sách sản phẩm và ghi chú. - Giao diện hiển thị trang chi tiết đơn hàng.
Ngoại lệ	- Không tìm thấy đơn hàng. - Khách hàng không phải chủ sở hữu.

Bảng 3.16: Kịch bản Use Case Xem chi tiết đơn hàng

17. Use Case: Xem tình trạng đơn hàng



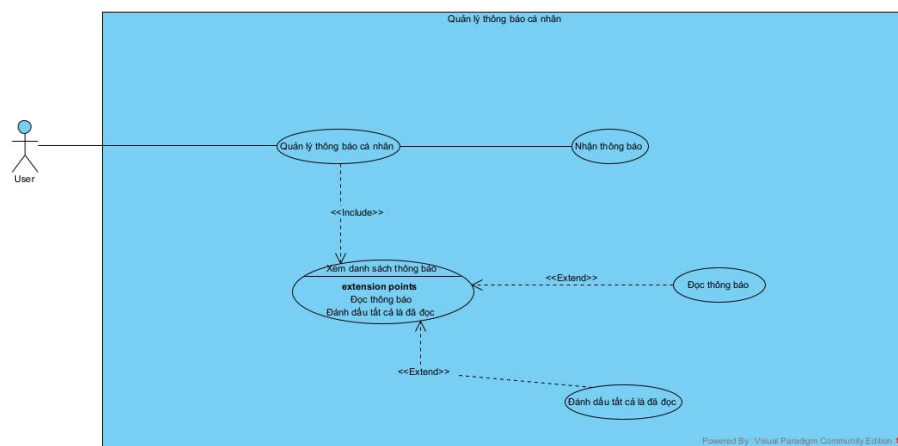
Hình 3.20: Sơ đồ Use Case - Xem tình trạng đơn hàng

- **Mô tả:** Cho phép khách hàng theo dõi tiến trình xử lý đơn hàng (Ví dụ: Đang xử lý, Đang giao hàng) thông qua các trạng thái và ghi chú cập nhật.

Tên Use Case	Xem tình trạng đơn hàng
Tác nhân	Khách hàng
Tiền điều kiện	Đơn hàng thuộc về khách hàng đang đăng nhập.
Hậu điều kiện	Trạng thái hiện tại và lịch sử cập nhật được hiển thị.
Kịch bản chính	- Khách hàng xem danh sách hoặc chi tiết đơn hàng. - Giao diện gửi yêu cầu cập nhật thông tin đơn hàng mới nhất. - Hệ thống trả về trạng thái hiện tại và các ghi chú cập nhật. - Giao diện hiển thị tiến trình xử lý cho khách hàng.
Ngoại lệ	Đơn hàng không tồn tại.

Bảng 3.17: Kịch bản Use Case Xem tình trạng đơn hàng

18. Use Case: Quản lý thông báo cá nhân



Hình 3.21: Sơ đồ Use Case - Quản lý thông báo cá nhân

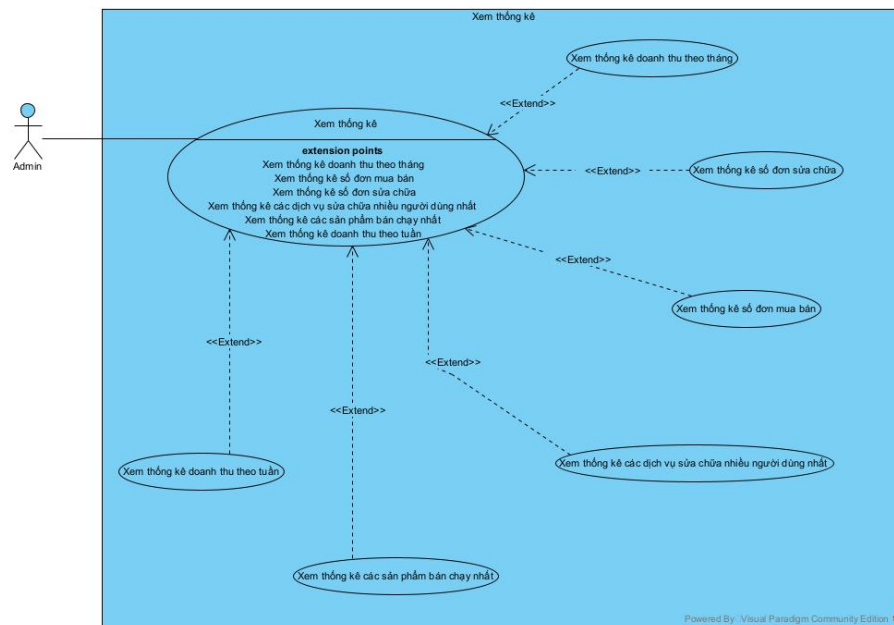
- **Mô tả:** Use case cho phép người dùng quản lý các thông báo cá nhân do hệ thống gửi đến, bao gồm việc nhận thông báo, xem danh sách thông báo, đọc thông báo và đánh dấu tất cả thông báo là đã đọc.

Tên Use Case	Quản lý thông báo cá nhân
Tác nhân	Khách hàng (User)
Tiền điều kiện	- User đã đăng nhập hệ thống và có tài khoản hợp lệ. - Hệ thống hoạt động bình thường.
Hậu điều kiện	- Thông báo được hiển thị cho người dùng. - Trạng thái thông báo được cập nhật (đã đọc / chưa đọc). - Không làm thay đổi các dữ liệu khác trong hệ thống.
Kịch bản chính	- User đăng nhập vào hệ thống. - User chọn chức năng "Thông báo". - Hệ thống tự động thực hiện use case "Xem danh sách thông báo". - Hệ thống hiển thị danh sách các thông báo của user.
Ngoại lệ	- Hệ thống không tìm thấy thông báo nào của user. - Hệ thống hiển thị thông báo "Không có thông báo".

Bảng 3.18: Kịch bản Use Case Quản lý thông báo cá nhân

b, Tác nhân: Admin (Quản trị viên)

1. Use Case: Xem thống kê



Hình 3.22: Sơ đồ Use Case - Xem thống kê

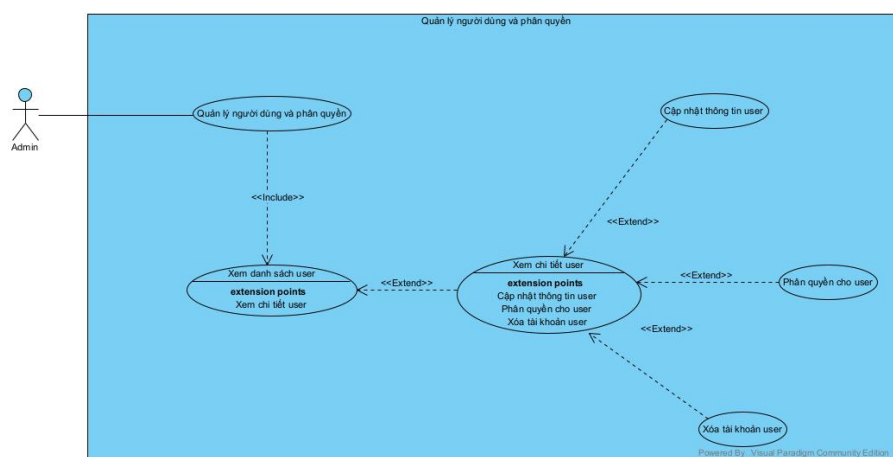
- **Mô tả:** Use case cho phép Admin xem các thống kê tổng hợp liên quan đến hoạt động kinh doanh của cửa hàng, bao gồm doanh thu, số lượng đơn hàng

mua bán, số lượng đơn sửa chữa, các sản phẩm bán chạy và các dịch vụ sửa chữa được sử dụng nhiều nhất.

Tên Use Case	Xem thống kê
Tác nhân	Admin
Tiền điều kiện	- Admin đã đăng nhập hệ thống. - Admin có quyền truy cập chức năng thống kê. - Hệ thống có dữ liệu phát sinh (đơn hàng, hóa đơn, dịch vụ).
Hậu điều kiện	- Thông tin thống kê được hiển thị cho Admin. - Không làm thay đổi dữ liệu trong hệ thống.
Kịch bản chính	- Admin chọn chức năng "Xem thống kê". - Hệ thống hiển thị giao diện thống kê tổng quan. - Hệ thống hiển thị toàn bộ kết quả thống kê.
Ngoại lệ	- Hệ thống không tìm thấy dữ liệu phù hợp. - Hệ thống thông báo không có dữ liệu để thống kê.

Bảng 3.19: Kịch bản Use Case Xem thống kê

2. Use Case: Quản lý người dùng và phân quyền



Hình 3.23: Sơ đồ Use Case - Quản lý người dùng và phân quyền

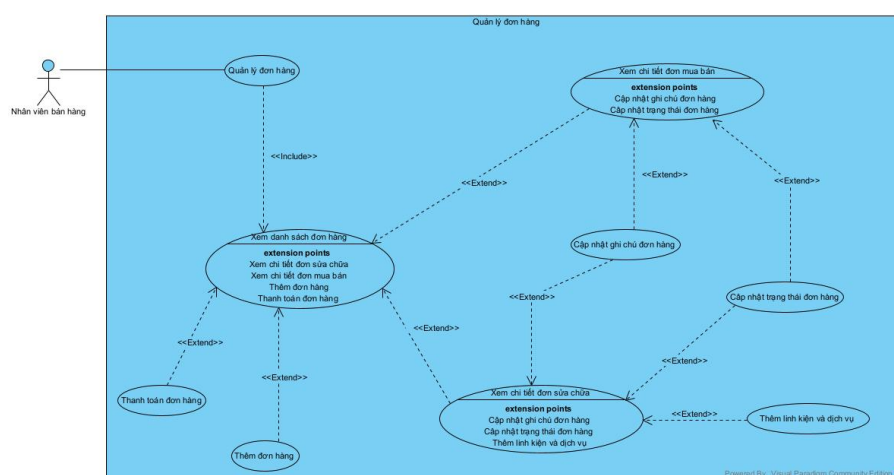
- Mô tả:** Admin chọn chức năng “Quản lý người dùng” trên giao diện, có thể chọn xem danh sách nhân viên và khách hàng. Hệ thống hiển thị danh sách người dùng, cho phép chọn từng người dùng để xem chi tiết hoặc cập nhật thông tin.

Tên Use Case	Quản lý người dùng
Tác nhân	Admin
Tiền điều kiện	- Người dùng cần có quyền ADMIN để truy cập chức năng. - Hệ thống đã lưu dữ liệu về người dùng.
Hậu điều kiện	Danh sách toàn bộ người dùng được hiển thị lên giao diện.
Kịch bản chính	- Admin vào trang quản lý hệ thống. - Admin chọn chức năng quản lý người dùng trên menu. - Giao diện hiển thị các thông tin: - Tất cả bản ghi thông tin và vai trò của người dùng trong hệ thống. - Click vào một người dùng để xem và chỉnh sửa thông tin chi tiết. - Ô tìm kiếm người dùng theo số điện thoại.
Ngoại lệ	- Người dùng không có quyền ADMIN. - Lỗi hệ thống, không thể hiển thị dữ liệu.

Bảng 3.20: Kịch bản Use Case Quản lý người dùng

c, Tác nhân: Nhân viên bán hàng (Sales Staff)

1. Use Case: Quản lý đơn hàng



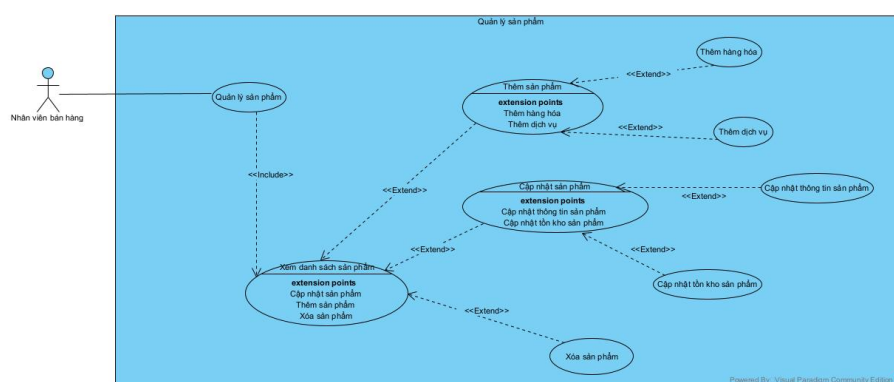
Hình 3.24: Sơ đồ Use Case - Quản lý đơn hàng

- Mô tả:** Nhân viên bán hàng chọn chức năng “Đơn hàng” trên giao diện. Hệ thống hiển thị danh sách tất cả đơn hàng kèm trạng thái, giá tiền và khách hàng.

Tên Use Case	Quản lý đơn hàng
Tác nhân	Nhân viên bán hàng
Tiền điều kiện	- Nhân viên bán hàng đã đăng nhập và có quyền truy cập chức năng. - Hệ thống đã lưu dữ liệu về sản phẩm.
Hậu điều kiện	Danh sách toàn bộ sản phẩm, danh mục được hiển thị trên giao diện.
Kịch bản chính	- Nhân viên bán hàng vào trang chủ hệ thống. - Nhân viên chọn chức năng quản lý đơn hàng trên menu. - Giao diện quản lý đơn hàng hiển thị các thông tin: - Tất cả bản ghi đơn hàng. - Click vào sản phẩm để xem và thực hiện cập nhật trạng thái đơn (có thể cập nhật sản phẩm cho đơn sửa chữa). - Ô tìm kiếm đơn hàng. - Nút lọc đơn hàng theo loại (sửa chữa, mua bán).
Ngoại lệ	- Không có dữ liệu để hiển thị. - Lỗi hệ thống, không thể hiển thị dữ liệu.

Bảng 3.21: Kịch bản Use Case Quản lý đơn hàng

2. Use Case: Quản lý sản phẩm



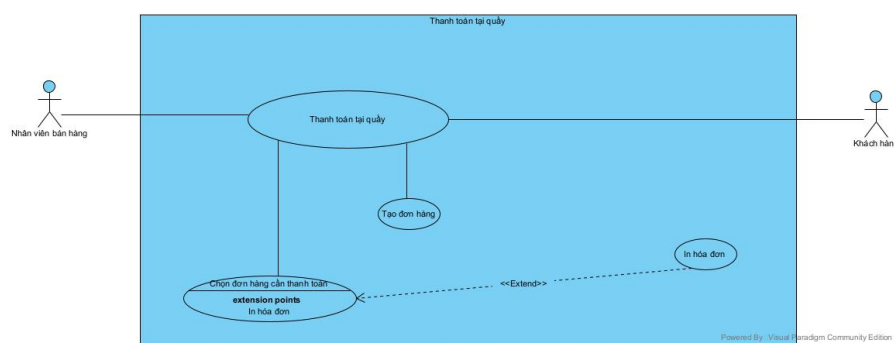
Hình 3.25: Sơ đồ Use Case - Quản lý sản phẩm

- Mô tả:** Nhân viên bán hàng chọn chức năng “Hàng hóa” trên giao diện. Hệ thống hiển thị danh sách tất cả sản phẩm, danh mục và có thể thêm mới, chọn từng sản phẩm để xem chi tiết, cập nhật thông tin.

Tên Use Case	Quản lý sản phẩm
Tác nhân	Nhân viên bán hàng
Tiền điều kiện	- Nhân viên bán hàng đã đăng nhập và có quyền truy cập chức năng. - Hệ thống đã lưu dữ liệu về sản phẩm.
Hậu điều kiện	Danh sách toàn bộ sản phẩm, danh mục được hiển thị trên giao diện.
Kịch bản chính	- Nhân viên bán hàng vào trang chủ hệ thống. - Nhân viên chọn chức năng quản lý sản phẩm trên menu. - Giao diện hiển thị các thông tin: - Tất cả bản ghi thông tin sản phẩm, danh mục. - Click vào sản phẩm để xem và chỉnh sửa thông tin chi tiết. - Ô tìm kiếm sản phẩm. - Nút thêm sản phẩm mới.
Ngoại lệ	- Nhân viên không có quyền truy cập chức năng. - Lỗi hệ thống, không thể hiển thị dữ liệu.

Bảng 3.22: Kịch bản Use Case Quản lý sản phẩm

3. Use Case: Thanh toán tại quầy



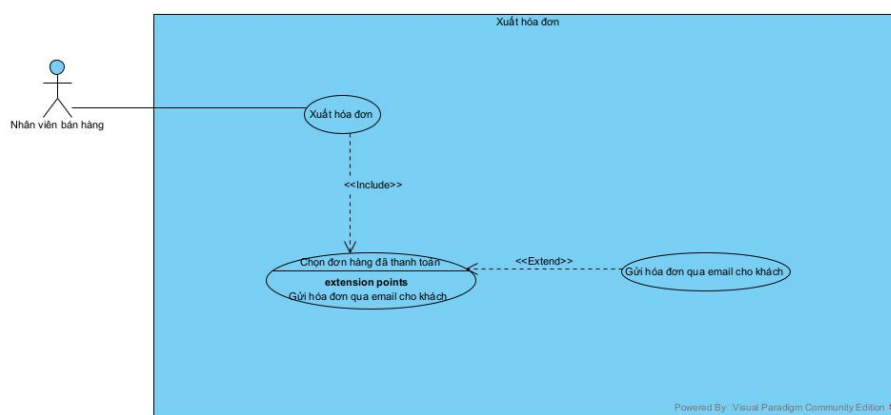
Hình 3.26: Sơ đồ Use Case - Thanh toán tại quầy

- **Mô tả:** Quy trình nhân viên bán hàng thực hiện thanh toán đơn hàng trực tiếp tại quầy cho khách hàng, bao gồm việc tạo đơn hàng, chọn phương thức thanh toán và in hóa đơn cho khách hàng khi cần.

Tên Use Case	Thanh toán tại quầy
Tác nhân	Nhân viên bán hàng, Khách hàng
Tiền điều kiện	- Nhân viên đã đăng nhập hệ thống. - Khách hàng có mặt tại quầy. - Hệ thống hoạt động bình thường.
Hậu điều kiện	- Đơn hàng được thanh toán thành công. - Thông tin đơn hàng được lưu vào hệ thống. - Hóa đơn được in (nếu có yêu cầu).
Kịch bản chính	- Nhân viên bán hàng chọn chức năng "Đơn hàng". - Hệ thống hiển thị giao diện đơn hàng. - Nhân viên chọn đơn hàng cần thanh toán. - Nhân viên nhập thông tin thanh toán và xác nhận. - Hệ thống xử lý thanh toán và lưu thông tin đơn hàng. - Hệ thống thông báo "thanh toán thành công". - Nhân viên in hóa đơn cho khách nếu được yêu cầu.
Ngoại lệ	Khách hàng không có đơn hàng: Nhân viên tạo đơn hàng cho khách hàng.

Bảng 3.23: Kịch bản Use Case Thanh toán tại quầy

4. Use Case: Xuất hóa đơn



Hình 3.27: Sơ đồ Use Case - Xuất hóa đơn

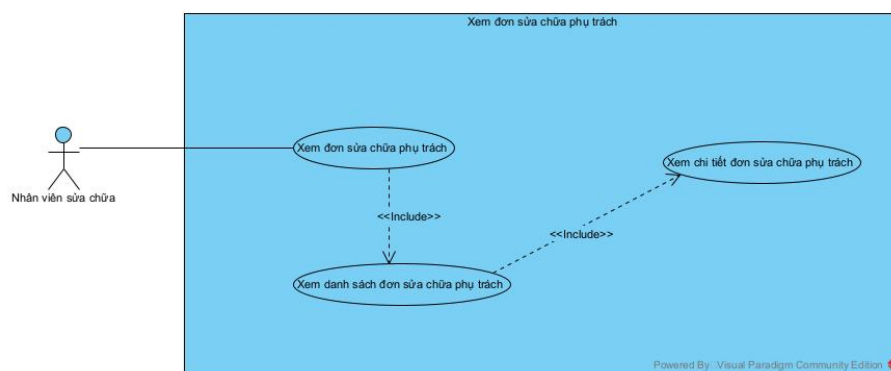
- **Mô tả:** Use case cho phép nhân viên bán hàng xuất hóa đơn cho các đơn hàng đã được thanh toán. Trong quá trình xuất hóa đơn, nhân viên có thể lựa chọn gửi hóa đơn qua email cho khách hàng nếu có nhu cầu.

Tên Use Case	Xuất hóa đơn
Tác nhân	Nhân viên bán hàng
Tiền điều kiện	- Nhân viên bán hàng đã đăng nhập hệ thống. - Đơn hàng đã được thanh toán thành công. - Hệ thống hoạt động bình thường.
Hậu điều kiện	Hóa đơn được gửi email cho khách hàng.
Kịch bản chính	- Nhân viên bán hàng chọn chức năng "Đơn hàng". - Hệ thống hiển thị giao diện đơn hàng. - Nhân viên chọn đơn hàng cần xuất hóa đơn. - Nhân viên chọn xuất hóa đơn. - Hệ thống xử lý và xuất hóa đơn gửi hóa đơn qua email cho khách. - Hệ thống thông báo xuất hóa đơn thành công.
Ngoại lệ	Hệ thống không gửi được email do lỗi kết nối hoặc email không hợp lệ.

Bảng 3.24: Kịch bản Use Case Xuất hóa đơn

d, Tác nhân: Kỹ thuật viên (Technician)

1. Use Case: Xem đơn sửa chữa phụ trách



Hình 3.28: Sơ đồ Use Case - Xem đơn sửa chữa phụ trách

- Mô tả:** Nhân viên sửa chữa chọn chức năng “Đơn sửa chữa phụ trách” trên giao diện. Hệ thống hiển thị danh sách tất cả các đơn hàng của nhân viên sửa chữa đó phụ trách.

Kịch bản chi tiết: Xem đơn sửa chữa phụ trách

Tên Use Case	Xem đơn sửa chữa phụ trách
Tác nhân	Nhân viên sửa chữa
Tiền điều kiện	Nhân viên sửa chữa đã đăng nhập và có quyền truy cập chức năng “Xem đơn sửa chữa phụ trách”
Hậu điều kiện	Danh sách toàn bộ sản phẩm, danh mục được hiển thị trên giao diện
Kịch bản chính	1. Nhân viên sửa chữa vào trang chủ hệ thống. 2. Nhân viên chọn chức năng xem đơn sửa chữa phụ trách trên menu. 3. Giao diện xem đơn sửa chữa phụ trách hiển thị các thông tin: • Tất cả bản ghi đơn hàng mà nhân viên sửa chữa đó phụ trách. • Click vào sản phẩm để xem chi tiết đơn hàng. • Ô tìm kiếm đơn hàng.
Ngoại lệ	• Không có dữ liệu để hiển thị. • Lỗi hệ thống, không thể hiển thị dữ liệu.

Bảng 3.25: Kịch bản Use Case Xem đơn sửa chữa phụ trách

3.4 Thiết kế Cơ sở dữ liệu

3.4.1 Mô hình thực thể liên kết (ERD)

Hệ thống được thiết kế với các module chính và các quan hệ giữa chúng:

a, Identity Module (Quản lý danh tính)

- **User (1) (N) Orders:** Một user có thể có nhiều đơn hàng
- **User (M) (N) Role:** Nhiều-many relationship thông qua bảng `user_roles`
- **Role (M) (N) Permission:** Nhiều-many relationship thông qua bảng `role_permissions`
- **User (1) (N) CartItem:** Một user có thể có nhiều items trong giỏ hàng
- **User (1) (N) Notification:** Một user có thể nhận nhiều thông báo
- **User (1) (N) FcmDeviceToken:** Một user có thể có nhiều FCM tokens (nhiều thiết bị)
- **User (M) (N) WorkSchedule:** Nhiều-many relationship thông qua bảng `work_schedule_u`

b, Product Module (Quản lý sản phẩm)

- **Category (1) (N) Product:** Một category có thể chứa nhiều products
- **Product (1) (N) ProductVariant:** Một product có thể có nhiều variants (màu

sắc khác nhau)

- **Product (1) (N) ProductImage:** Một product có nhiều hình ảnh giới thiệu (introImages)
- **ProductVariant (1) (N) ProductAttribute:** Một variant có nhiều attributes (kích thước, dung lượng, giá cả khác nhau)
- **ProductVariant (1) (N) ProductImage:** Một variant có nhiều hình ảnh chi tiết (detailImages)
- **ProductAttribute (1) (N) OrderItem:** Một attribute có thể xuất hiện trong nhiều order items
- **ProductAttribute (1) (N) CartItem:** Một attribute có thể xuất hiện trong nhiều cart items

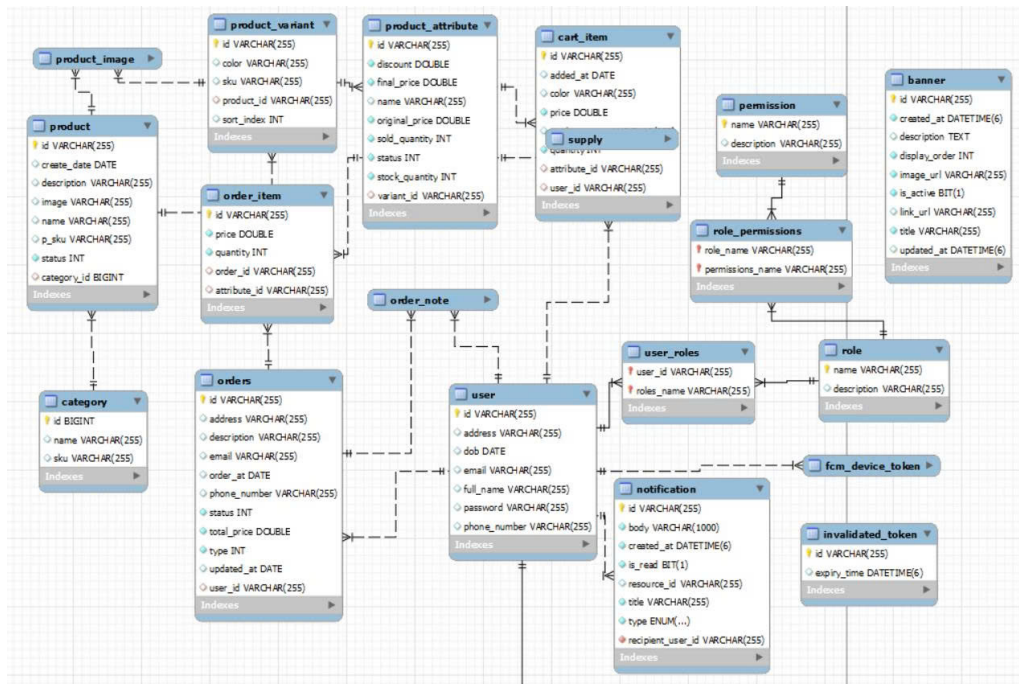
c, **Cart & Order Module (Giỏ hàng và Đơn hàng)**

- **Orders (1) (N) OrderItem:** Một đơn hàng có nhiều order items
- **Orders (1) (N) OrderNote:** Một đơn hàng có thể có nhiều ghi chú (đặc biệt cho đơn sửa chữa)
- **OrderItem (N) (1) ProductAttribute:** Một order item thuộc về một product attribute cụ thể

d, **Work Schedule Module (Lịch làm việc)**

- **WorkSchedule (N) (1) Shift:** Một work schedule thuộc về một ca làm việc
- **WorkSchedule (M) (N) User:** Nhiều-many relationship thông qua bảng work_schedule_u
- **Shift (1) (N) Task:** Một ca làm việc có nhiều nhiệm vụ (có thể được mở rộng trong tương lai)

3.4.2 Thiết kế danh sách các bảng



Hình 3.29: Sơ đồ cơ sở dữ liệu

a, Khái quát các nhóm bảng chính

Cơ sở dữ liệu được chia thành các nhóm bảng theo chức năng:

- **Nhóm sản phẩm – danh mục:** gồm category, product, product_image, product_variant, product_attribute. Nhóm này quản lý thông tin danh mục và sản phẩm, đồng thời hỗ trợ sản phẩm có nhiều biến thể (ví dụ: màu sắc/sku) và thuộc tính bán hàng (giá, số lượng tồn, số lượng đã bán...).
- **Nhóm giỏ hàng:** gồm cart_item dùng để lưu các mặt hàng người dùng đã thêm vào giỏ, phục vụ quá trình đặt hàng.
- **Nhóm đơn hàng:** gồm orders, order_item, order_note. orders lưu thông tin tổng quan của đơn hàng; order_item lưu danh sách sản phẩm trong đơn; order_note lưu ghi chú liên quan đến đơn hàng.
- **Nhóm người dùng – phân quyền:** gồm user, role, permission, user_roles, role_permissions. Nhóm này quản lý tài khoản người dùng và mô hình phân quyền theo vai trò (RBAC), cho phép mỗi người dùng có thể có nhiều vai trò và mỗi vai trò có thể có nhiều quyền.
- **Nhóm thông báo và xác thực:** gồm notification, fcm_device_token, invalidated_token. notification lưu nội dung thông báo gửi đến người dùng; fcm_device_token quản lý thiết bị nhận thông báo (FCM); invalidated_token

phục vụ cơ chế vô hiệu hóa token (đăng xuất/hết hạn/thu hồi).

b, Khái quát quan hệ giữa các bảng

Các bảng trong hệ thống liên kết với nhau theo các quan hệ chính sau:

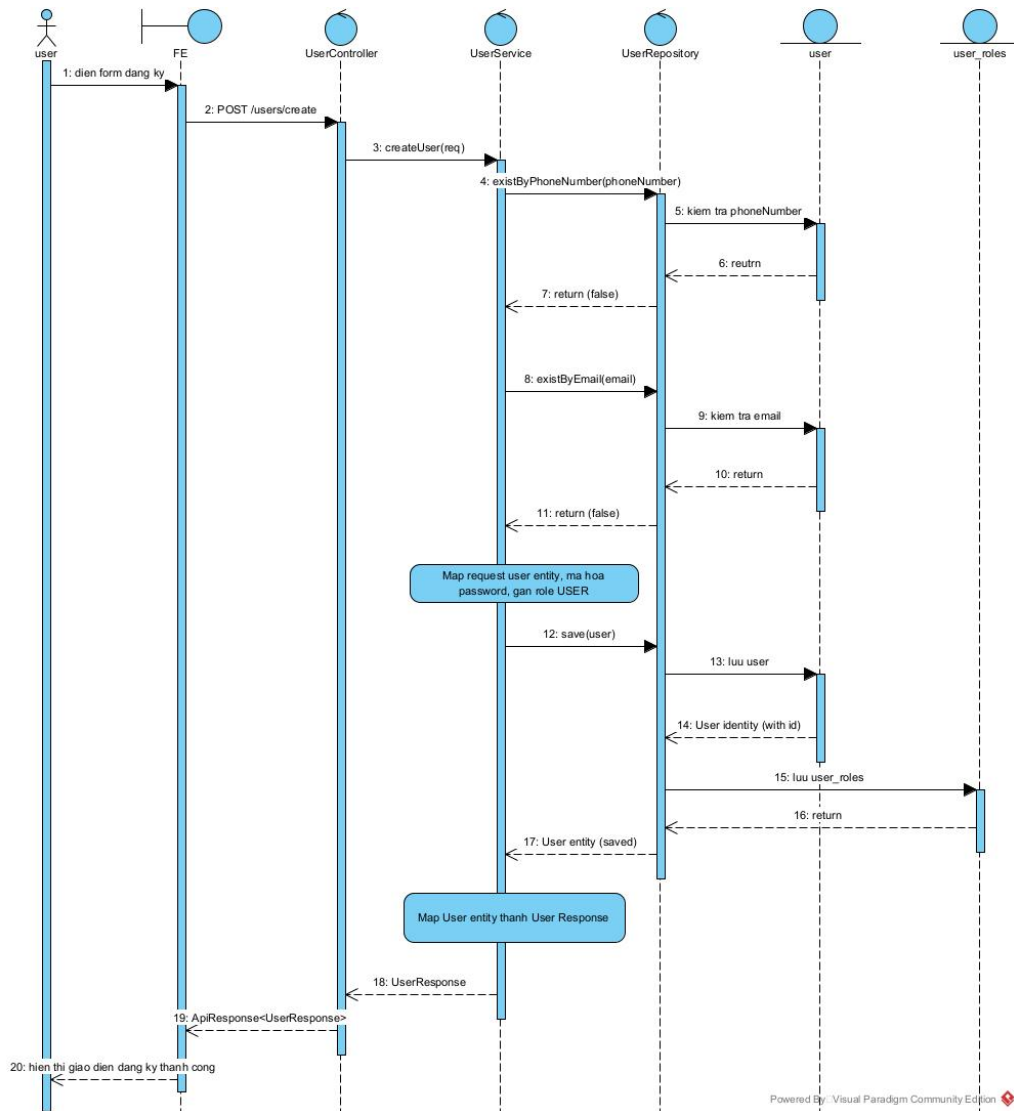
- **Danh mục – Sản phẩm (1–N):** Một category có thể chứa nhiều product; mỗi product thuộc về một category.
- **Sản phẩm – Ảnh sản phẩm (1–N):** Một product có thể có nhiều bản ghi trong product_image để lưu nhiều hình ảnh minh họa.
- **Sản phẩm – Biến thể (1–N):** Một product có thể có nhiều product_variant (ví dụ: theo màu/sku).
- **Biến thể – Thuộc tính bán hàng (1–N hoặc 1–1 tùy thiết kế):** product_attribute lưu thông tin bán hàng như giá, tồn kho, số lượng đã bán và thường liên kết với product_variant để quản lý theo từng biến thể.
- **Người dùng – Đơn hàng (1–N):** Một user có thể tạo nhiều orders; mỗi orders thuộc về một user.
- **Đơn hàng – Chi tiết đơn hàng (1–N):** Một orders có nhiều order_item; mỗi order_item gắn với một đơn hàng và tham chiếu tới sản phẩm/biến thể tương ứng.
- **Người dùng – Giỏ hàng (1–N theo từng item):** cart_item liên kết với người dùng (trực tiếp hoặc gián tiếp tùy triển khai), giúp lưu danh sách sản phẩm người dùng đang chuẩn bị mua.
- **Người dùng – Vai trò (N–N):** Quan hệ nhiều–nhiều được triển khai qua bảng trung gian user_roles, cho phép một người dùng có nhiều vai trò và một vai trò có thể gán cho nhiều người dùng.
- **Vai trò – Quyền (N–N):** Quan hệ nhiều–nhiều được triển khai qua bảng trung gian role_permissions, cho phép một vai trò có nhiều quyền và một quyền có thể được dùng bởi nhiều vai trò.
- **Người dùng – Thông báo (1–N):** Một user có thể nhận nhiều bản ghi trong notification.
- **Người dùng – Thiết bị nhận thông báo (1–N):** Một user có thể có nhiều fcm_device_token tương ứng với nhiều thiết bị đăng nhập.

3.5 Thiết kế hệ thống chi tiết

3.5.1 Biểu đồ trình tự (Sequence Diagram)

a, Tác nhân A: Khách hàng (Customer)

1. Luồng Đăng ký tài khoản (Register)



Hình 3.30: Sequence Diagram - Đăng ký tài khoản

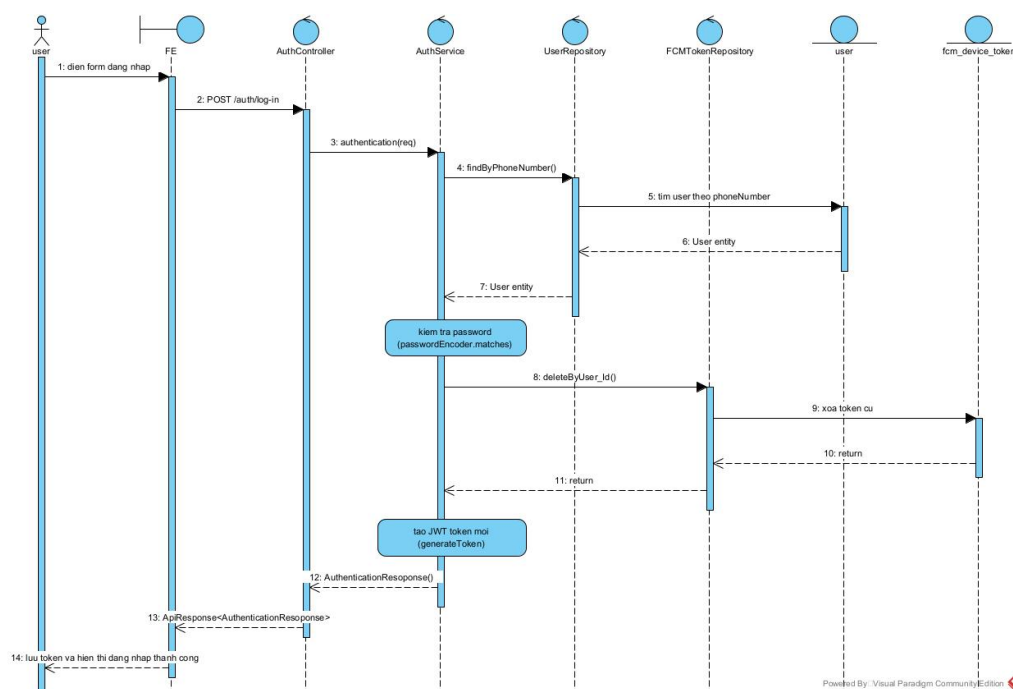
Mô tả các bước tương tác:

1. **Người dùng** điền form đăng ký (số điện thoại, mật khẩu, họ tên, email) và nhấn nút đăng ký trên Client.
2. **Client** gửi HTTP POST request đến endpoint /users/create với dữ liệu UserCreationRequest tới **UserController**.
3. **Controller** gọi hàm createUser(request) trong **UserService**.
4. **Service** gọi existsByPhoneNumber() và existsByEmail() trong **UserRepos-**

itory để kiểm tra sự tồn tại của tài khoản.

5. **Repository** trả về kết quả (nếu chưa tồn tại, tiếp tục xử lý).
6. **Service** map request thành User entity, mã hóa mật khẩu bằng PasswordEncoder, và gán role "USER".
7. **Service** gọi save(user) trong **UserRepository**.
8. **Repository** lưu thông tin user và quan hệ user-role vào database, sau đó trả về entity đã lưu.
9. **Service** trả về UserResponse (đã ẩn mật khẩu) cho **Controller**.
10. **Controller** trả về ApiResponse cho **Client** để hiển thị thông báo thành công.

2. Luồng Đăng nhập (Login)



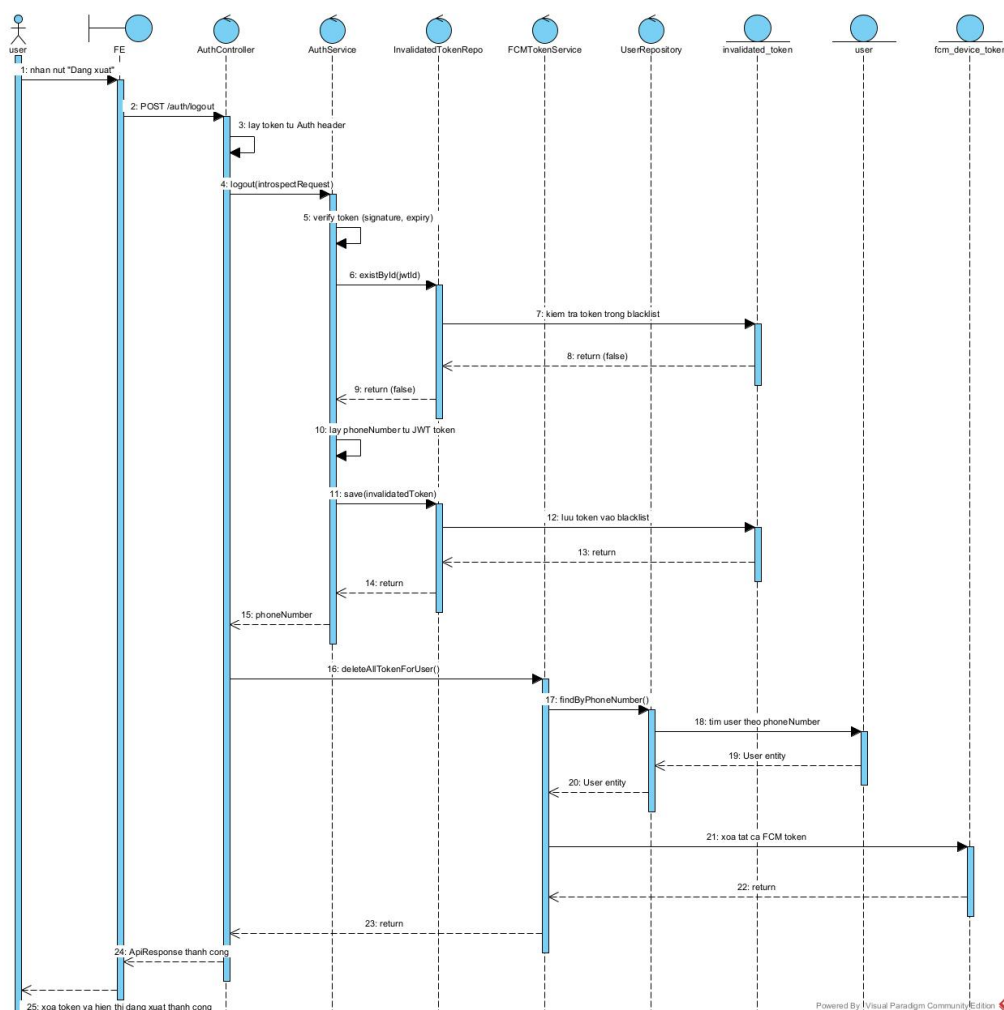
Hình 3.31: Sequence Diagram - Đăng nhập

Mô tả các bước tương tác:

1. **Người dùng** nhập số điện thoại, mật khẩu và nhấn đăng nhập.
2. **Client** gửi HTTP POST request đến /auth/login tới **AuthenticationController**.
3. **Controller** gọi hàm authenticate() trong **AuthenticationService**.
4. **Service** gọi findByPhoneNumber() trong **UserRepository** để lấy thông tin user.
5. **Service** kiểm tra mật khẩu bằng passwordEncoder.matches().

6. **Service** gọi `deleteByUser_Id()` trong **FcmDeviceTokenRepository** để xóa token thiết bị cũ (đảm bảo tính duy nhất).
7. **Service** tạo JWT token mới (chứa thông tin subject, roles, expiry) và trả về `AuthenticationResponse`.
8. **Controller** trả về response chứa token cho **Client**.
9. **Client** lưu token và chuyển hướng người dùng vào hệ thống.

3. Luồng Đăng xuất (Logout)



Hình 3.32: Sequence Diagram - Đăng xuất

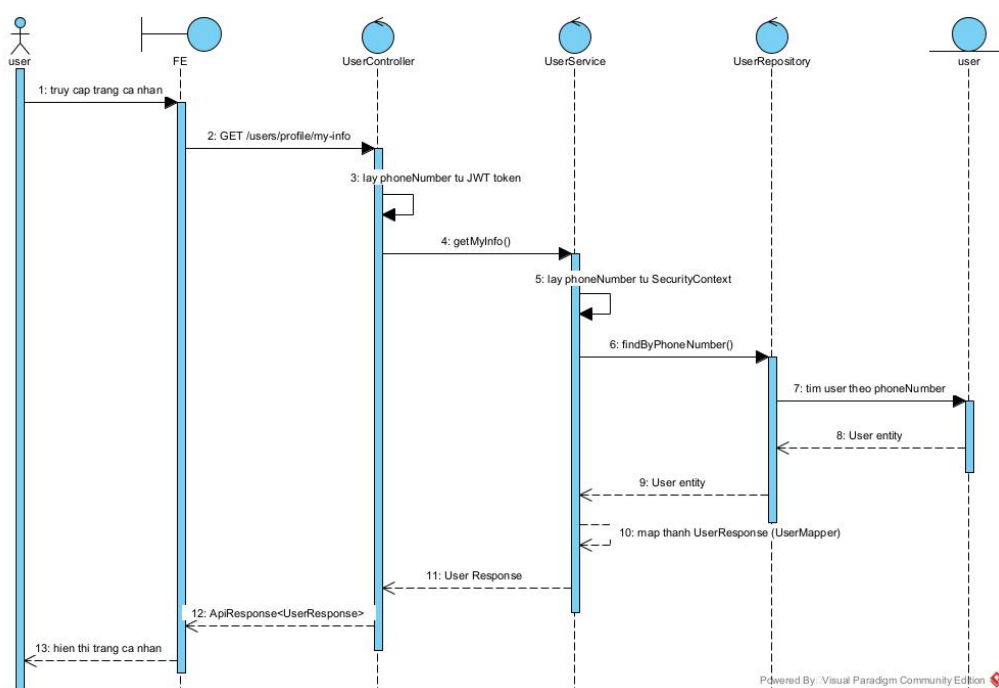
Mô tả các bước tương tác:

1. **Người dùng** nhấn nút đăng xuất trên **Client**.
2. **Client** gửi HTTP POST request đến `/auth/logout` kèm JWT token trong header tới **AuthenticationController**.
3. **Controller** gọi hàm `logout()` trong **AuthenticationService**.
4. **Service** verify token và gọi `save()` trong **InvalidatedTokenRepository** để

đưa token vào blacklist.

5. **Service** gọi `deleteAllTokensForUser()` trong **FcmDeviceTokenService** để xóa token FCM (ngắt thông báo đẩy).
6. **Repository** thực hiện xóa dữ liệu trong database và trả kết quả.
7. **Controller** trả về thông báo thành công cho **Client**.
8. **Client** xóa token khỏi bộ nhớ trình duyệt và hiển thị màn hình đăng nhập.

4. Luồng Xem thông tin cá nhân (View Profile)

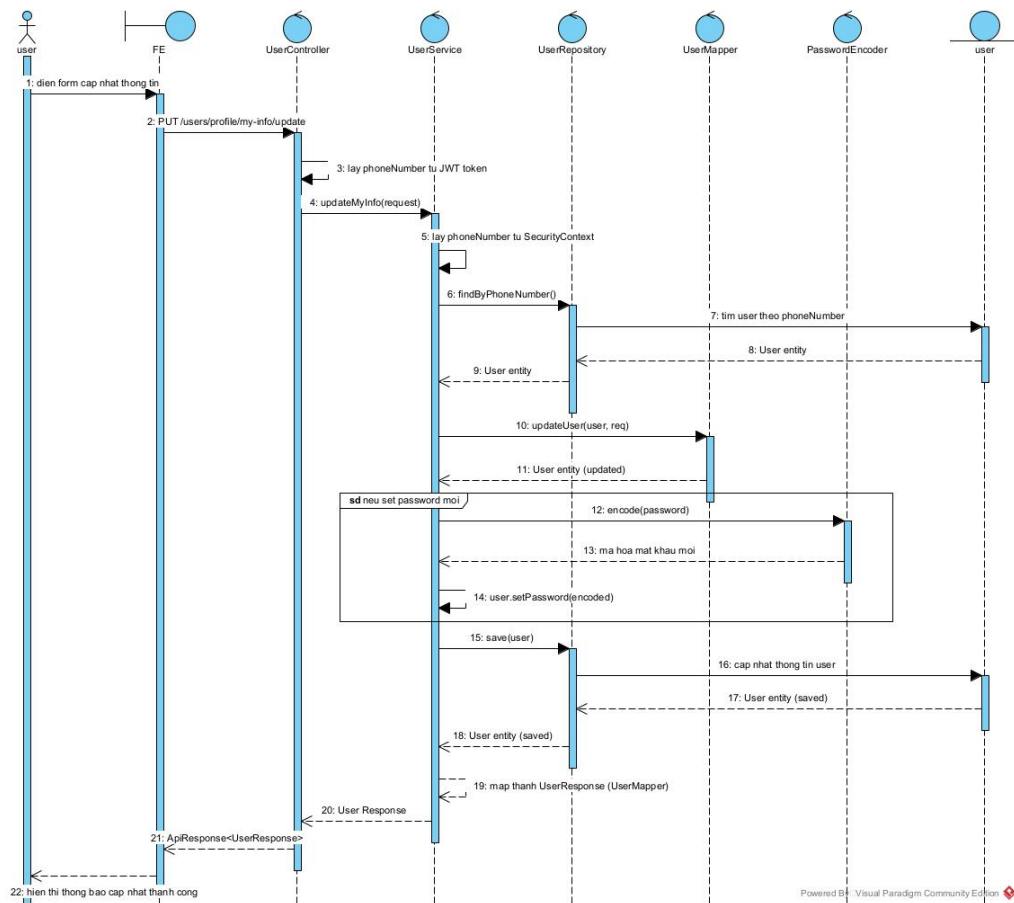


Hình 3.33: Sequence Diagram - Xem thông tin cá nhân

Mô tả các bước tương tác:

1. **Client** gửi HTTP GET request đến `/users/profile/my-info` kèm token tới **UserController**.
2. **Controller** lấy số điện thoại từ `SecurityContext` và gọi `getMyInfo()` trong **UserService**.
3. **Service** gọi `findByPhoneNumber()` trong **UserRepository**.
4. **Repository** trả về `User entity`.
5. **Service** map entity sang `UserResponse` và trả về cho **Controller**.
6. **Controller** trả dữ liệu JSON cho **Client** hiển thị thông tin người dùng.

5. Luồng Xem danh sách sản phẩm (View Products)

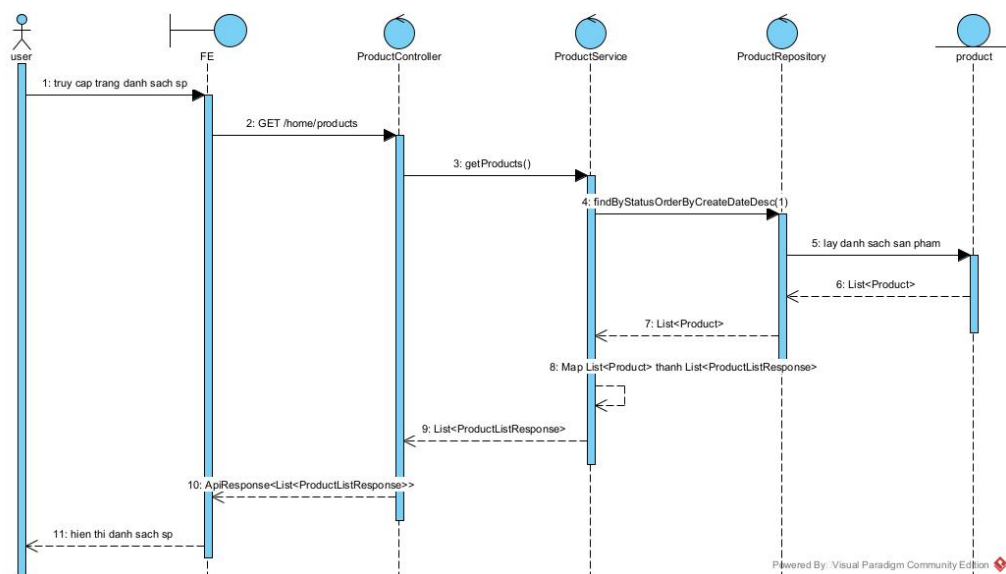


Hình 3.34: Sequence Diagram - Xem danh sách sản phẩm

Mô tả các bước tương tác:

1. **Client** gửi HTTP GET request đến /home/products tới **ProductController**.
2. **Controller** gọi hàm `getProducts()` trong **ProductService**.
3. **Service** gọi `findByStatusOrderByCreateDateDesc(1)` trong **ProductRepository** để lấy sản phẩm đang hoạt động.
4. **Repository** trả về danh sách Product entities.
5. **Service** map danh sách này sang `List<ProductListResponse>` (gồm id, tên, giá, ảnh).
6. **Controller** trả dữ liệu JSON cho **Client** hiển thị danh sách sản phẩm.

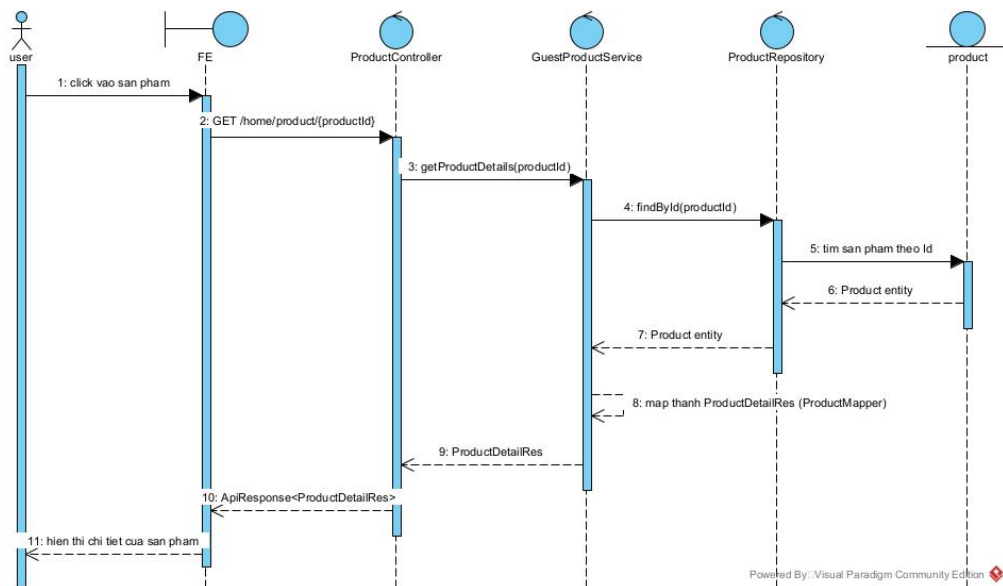
6. Luồng Cập nhật thông tin cá nhân (Update Profile)



Hình 3.35: Sequence Diagram - Cập nhật thông tin cá nhân

Mô tả các bước tương tác:

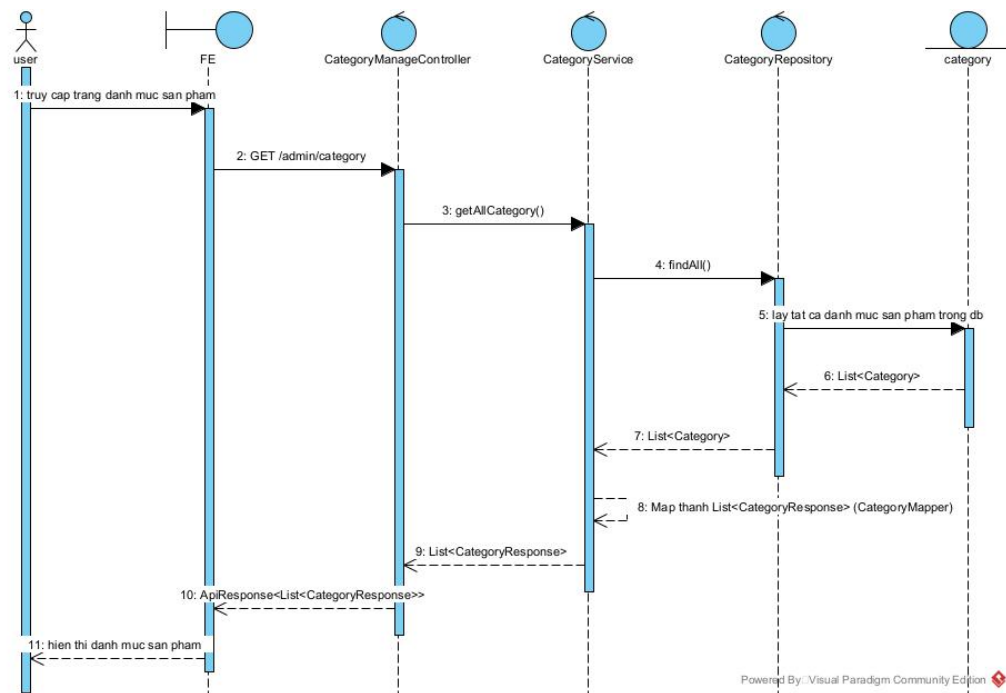
1. **Người dùng** điền thông tin mới và nhấn lưu.
 2. **Client** gửi HTTP PUT request đến `/users/profile/my-info/update` tới **UserController**.
 3. **Controller** gọi `updateMyInfo(request)` trong **UserService**.
 4. **Service** lấy user hiện tại, gọi `UserMapper.updateUser()` để cập nhật các trường thay đổi.
 5. Nếu có đổi mật khẩu, **Service** gọi `PasswordEncoder.encode()` để mã hóa mật khẩu mới.
 6. **Service** gọi `save()` trong **UserRepository** để lưu thông tin.
 7. **Repository** cập nhật DB và trả về entity mới.
 8. **Controller** trả về `UserResponse` mới nhất cho **Client**.
- 7. Luồng Xem chi tiết sản phẩm (View Product Details)**



Hình 3.36: Sequence Diagram - Xem chi tiết sản phẩm

Mô tả các bước tương tác:

1. **Client** gửi HTTP GET request đến /home/product/{id} tới **ProductController**.
2. **Controller** gọi getProductDetails(id) trong **GuestProductService**.
3. **Service** gọi findById(id) trong **ProductRepository**.
4. **Repository** trả về Product entity đầy đủ thông tin.
5. **Service** map entity sang ProductDetailRes (bao gồm ảnh intro, variants, attributes, detail images).
6. **Controller** trả dữ liệu JSON cho **Client** hiển thị.
8. **Luồng Xem danh mục sản phẩm (View Categories)**

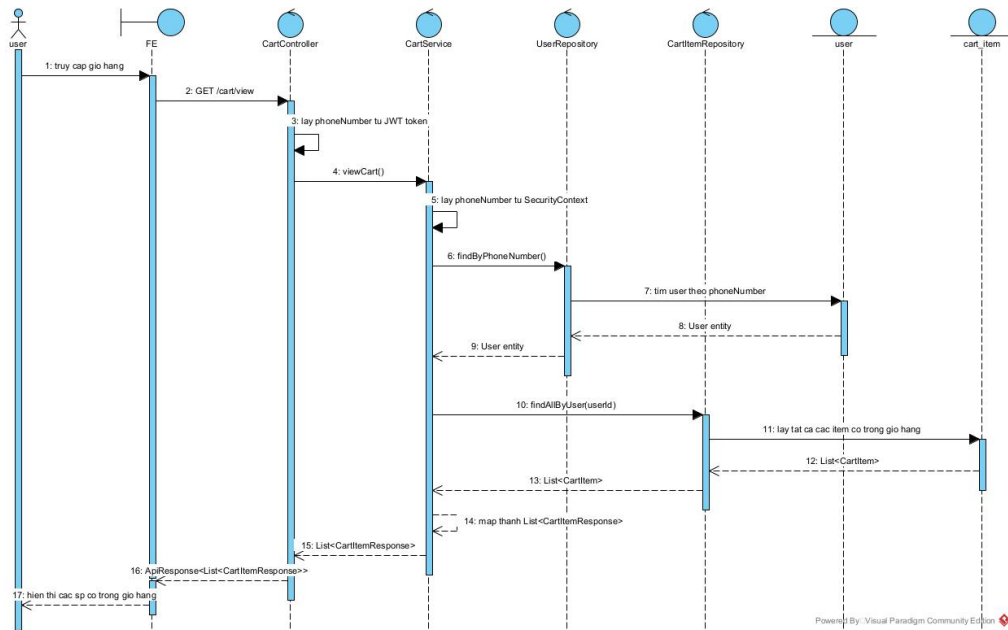


Hình 3.37: Sequence Diagram - Xem danh mục sản phẩm

Mô tả các bước tương tác:

1. **Client** gửi HTTP GET request đến `/admin/category` tới **CategoryManage-Controller**.
2. **Controller** gọi `getAllCategory()` trong **CategoryService**.
3. **Service** gọi `findAll()` trong **CategoryRepository**.
4. **Repository** trả về danh sách tất cả danh mục.
5. **Service** map sang `List<CategoryResponse>` và trả về cho **Controller**.
6. **Controller** gửi response JSON cho **Client**.

9. Luồng Xem giỏ hàng (View Cart)

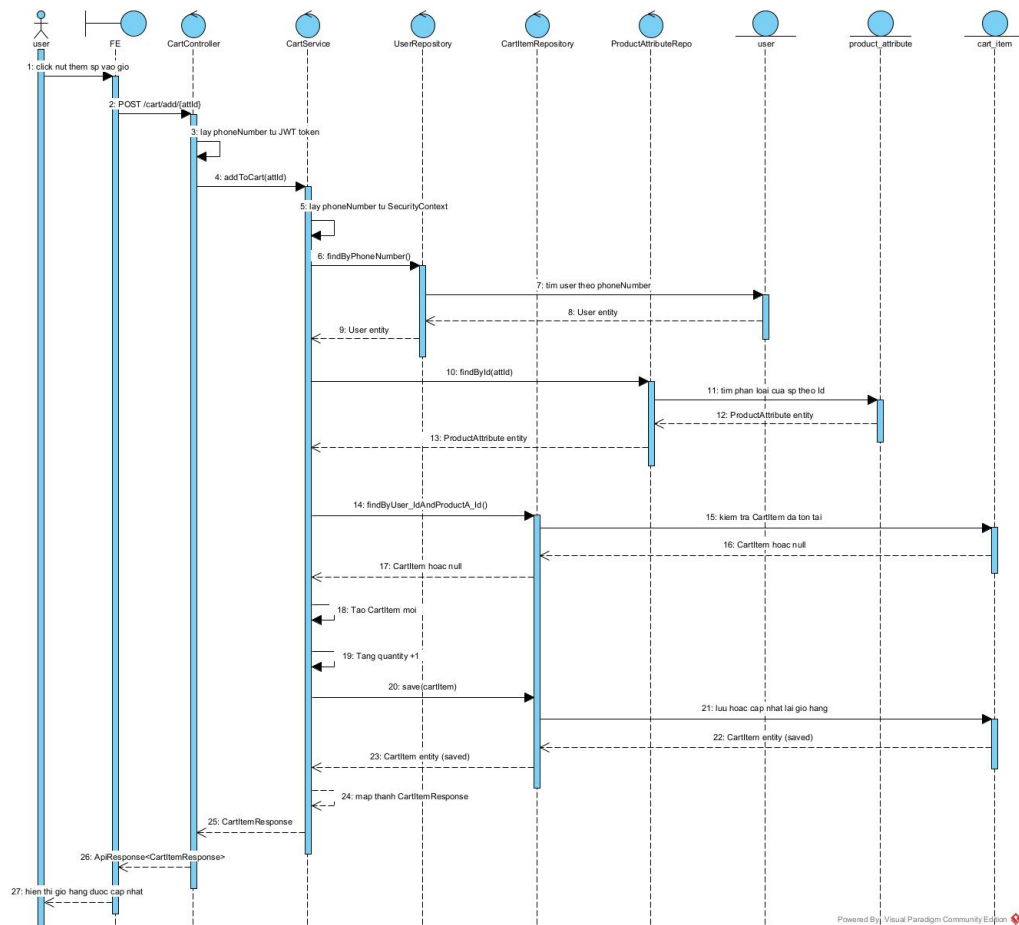


Hình 3.38: Sequence Diagram - Xem giỏ hàng

Mô tả các bước tương tác:

1. **Client** gửi HTTP GET request đến `/cart/view` kèm token tới **CartController**.
2. **Controller** xác định user và gọi `viewCart()` trong **CartService**.
3. **Service** gọi `findAllByUserId()` trong **CartItemRepository**.
4. **Repository** trả về danh sách các items trong giỏ.
5. **Service** map sang `CartItemResponse` (tên SP, màu, giá, số lượng, tổng tiền).
6. **Controller** trả dữ liệu JSON cho **Client** hiển thị.

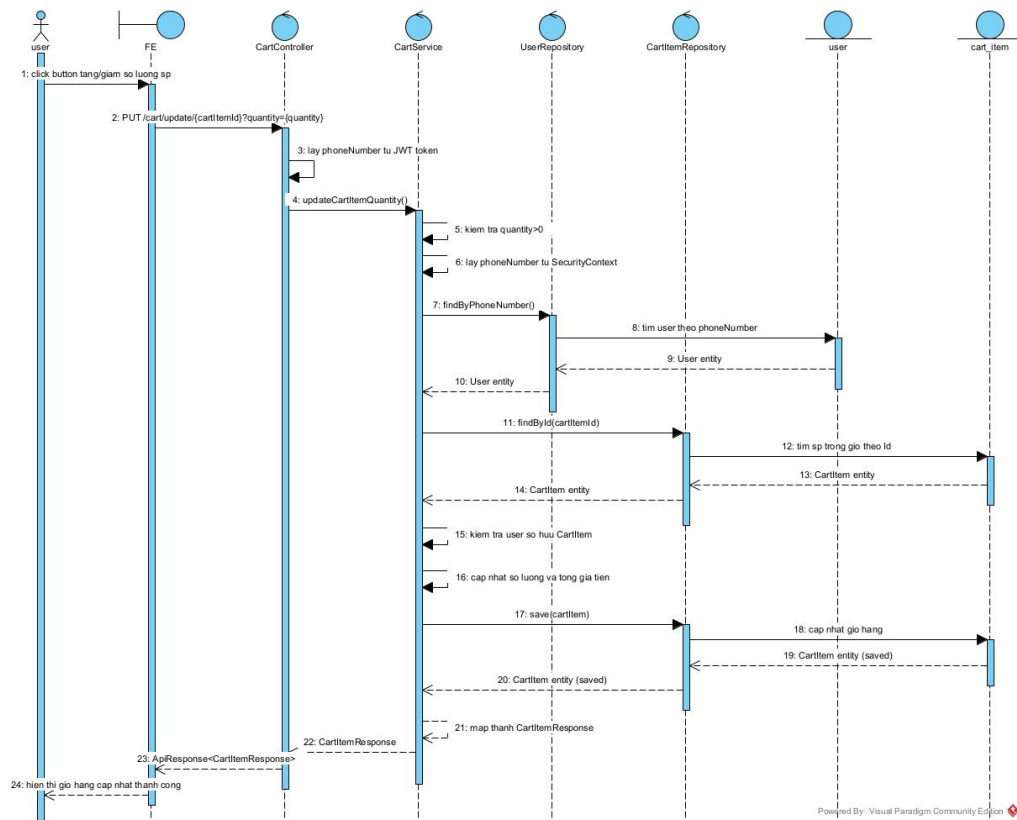
10. Luồng Thêm sản phẩm vào giỏ hàng (Add to Cart)



Hình 3.39: Sequence Diagram - Thêm sản phẩm vào giỏ

Mô tả các bước tương tác:

1. Người dùng chọn sản phẩm và nhấn "Thêm vào giỏ".
2. Client gửi HTTP POST request đến /cart/add/{attId} tới **CartController**.
3. Controller gọi addToCart() trong **CartService**.
4. Service kiểm tra user và gọi findById() trong **ProductAttributeRepository**.
5. Service kiểm tra item đã tồn tại trong giỏ chưa qua **CartItemRepository**.
6. Nếu chưa tồn tại, tạo mới; nếu đã có, tăng số lượng lên 1.
7. Service gọi save() trong **CartItemRepository**.
8. Repository lưu dữ liệu và trả về item đã lưu.
9. Controller trả kết quả thành công cho Client.
11. Luồng Cập nhật số lượng giỏ hàng (Update Cart Quantity)

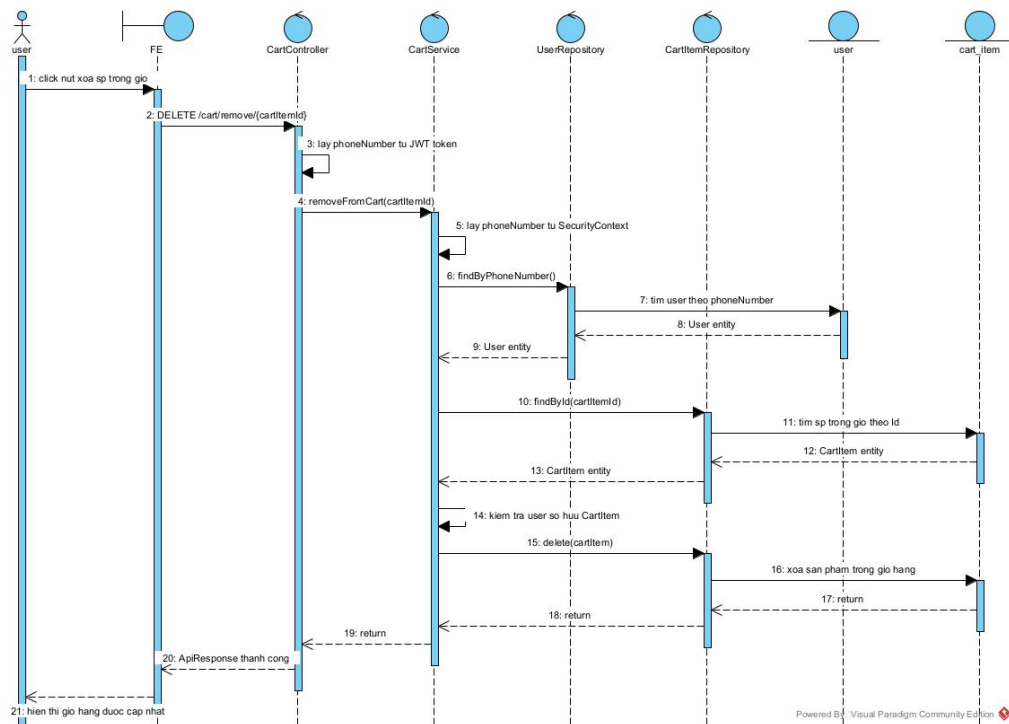


Hình 3.40: Sequence Diagram - Cập nhật số lượng giỏ hàng

Mô tả các bước tương tác:

1. **Client** gửi HTTP PUT request đến `/cart/update/{id}` với số lượng mới tới **CartController**.
2. **Controller** gọi `updateCartItemQuantity()` trong **CartService**.
3. **Service** kiểm tra số lượng hợp lệ và quyền sở hữu item của user.
4. **Service** cập nhật số lượng và tính lại giá (theo giá hiện tại của sản phẩm).
5. **Service** gọi `save()` trong **CartItemRepository**.
6. **Repository** cập nhật DB và trả về kết quả.
7. **Controller** trả dữ liệu item đã cập nhật cho **Client**.

12. Luồng Xóa sản phẩm khỏi giỏ hàng (Remove from Cart)

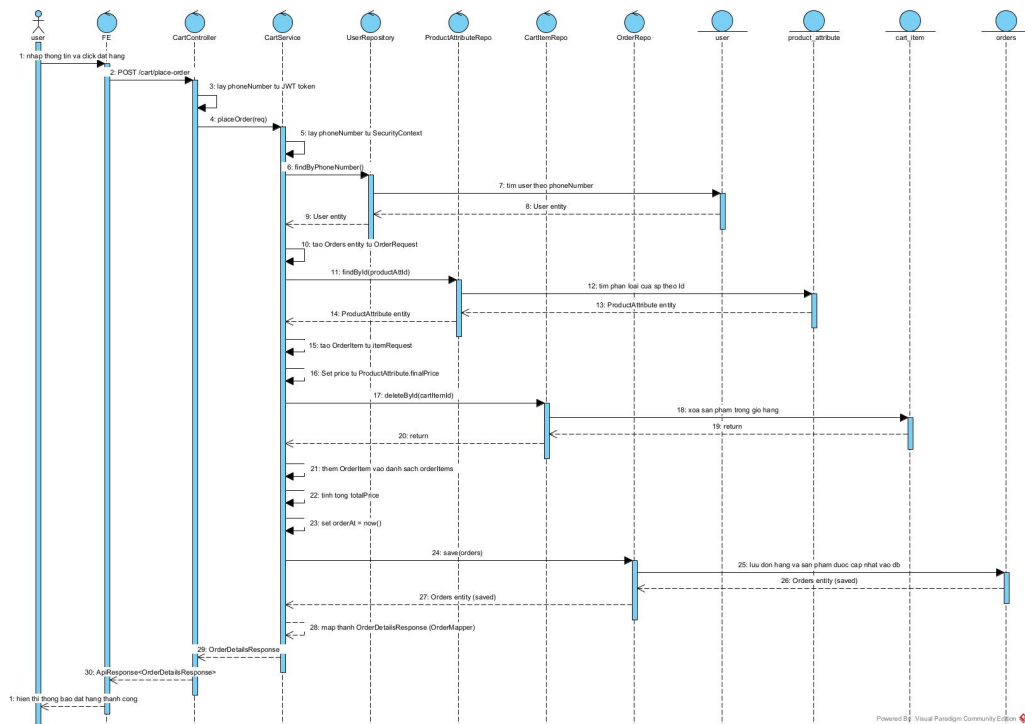


Hình 3.41: Sequence Diagram - Xóa sản phẩm khỏi giỏ hàng

Mô tả các bước tương tác:

1. **Client** gửi HTTP DELETE request đến `/cart/remove/{id}` tới **CartController**.
2. **Controller** gọi `removeFromCart()` trong **CartService**.
3. **Service** tìm item và kiểm tra quyền sở hữu.
4. **Service** gọi `delete()` trong **CartItemRepository**.
5. **Repository** xóa bản ghi khỏi database.
6. **Controller** trả thông báo thành công cho **Client**.

13. Luồng Đặt hàng (Place Order)

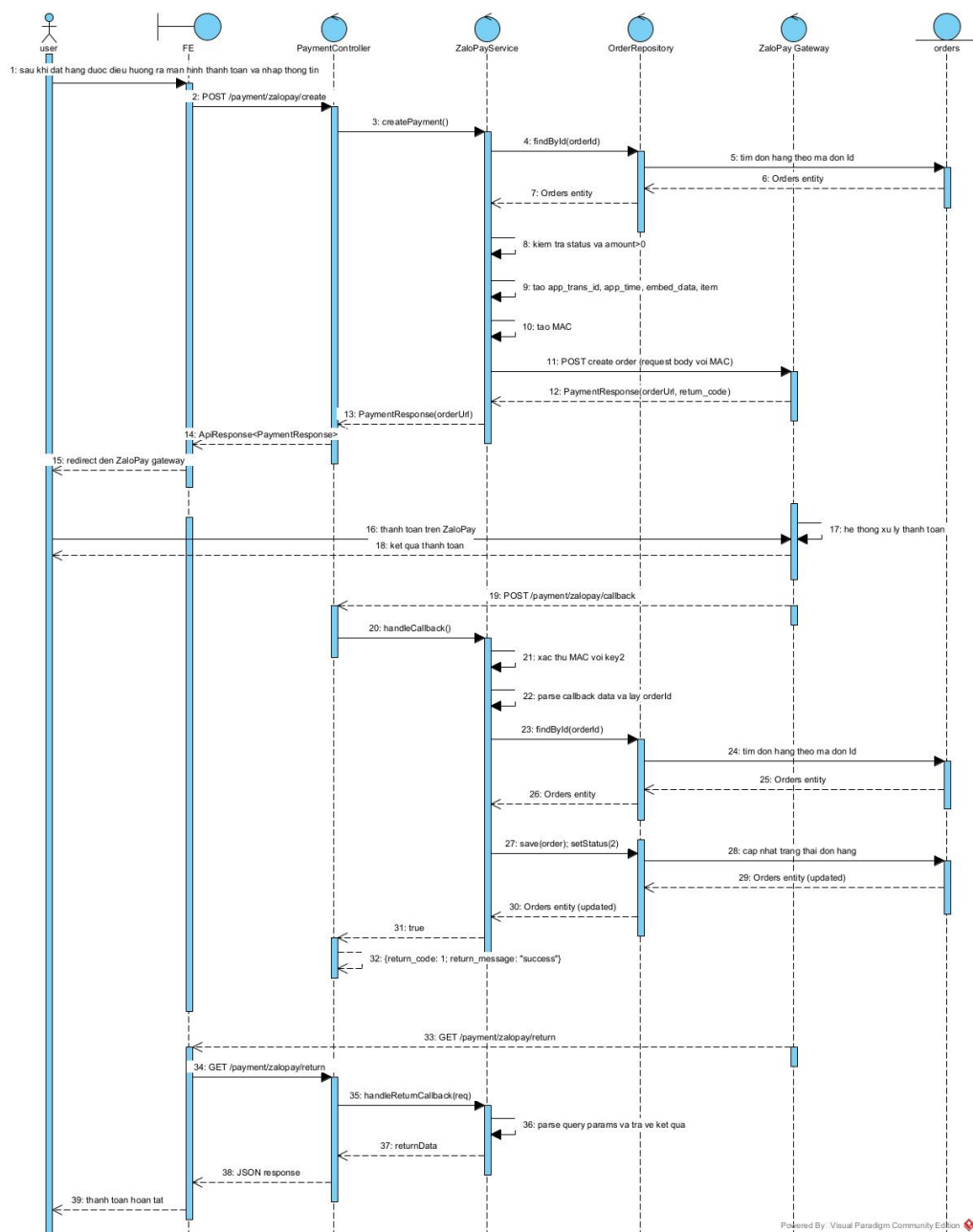


Hình 3.42: Sequence Diagram - Đặt hàng

Mô tả các bước tương tác:

1. **Client** gửi HTTP POST request đến `/cart/place-order` với thông tin đơn hàng tới **CartController**.
2. **Controller** gọi `placeOrder()` trong **CartService**.
3. **Service** tạo entity **Orders**. Với mỗi sản phẩm, tìm `ProductAttribute`, tạo `OrderItem`, set giá, và xóa `CartItem` tương ứng.
4. **Service** tính tổng tiền và gọi `save()` trong **OrderRepository** (cascade save `OrderItems`).
5. **Repository** lưu đơn hàng vào DB và trả về entity.
6. **Service** map sang `OrderDetailsResponse`.
7. **Controller** trả thông tin đơn hàng đã tạo cho **Client**.

14. Lũồng Thanh toán Online (Payment)



Hình 3.43: Sequence Diagram - Thanh toán Online (ZaloPay)

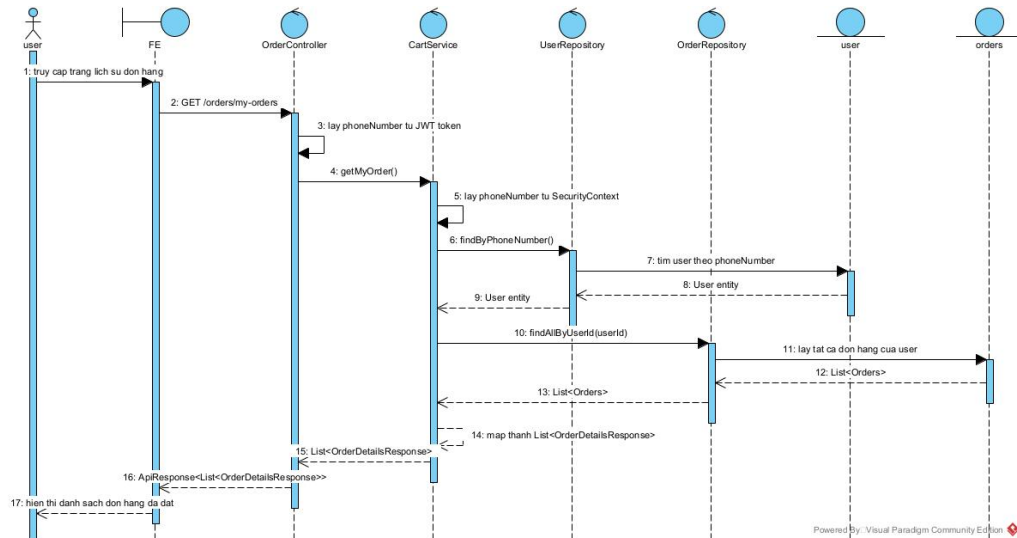
Mô tả các bước tương tác:

1. **Client** gọi `/payment/zalopay/create` tới **PaymentController**.
2. **ZaloPayService** tạo `app_trans_id`, tính HMAC SHA256 và gọi API của ZaloPay Gateway.
3. ZaloPay trả về `orderId`, **Controller** trả về cho **Client** để redirect người dùng.
4. Sau khi thanh toán, ZaloPay gọi callback đến `/payment/zalopay/callback`.
5. **ZaloPayService** verify MAC, tìm đơn hàng qua **OrderRepository** và cập nhật

trạng thái `status = 2` (Đã thanh toán).

- Người dùng được redirect về trang kết quả, **Client** hiển thị trạng thái thành công/thất bại.

15. Luồng Xem lịch sử đơn hàng (Order History)

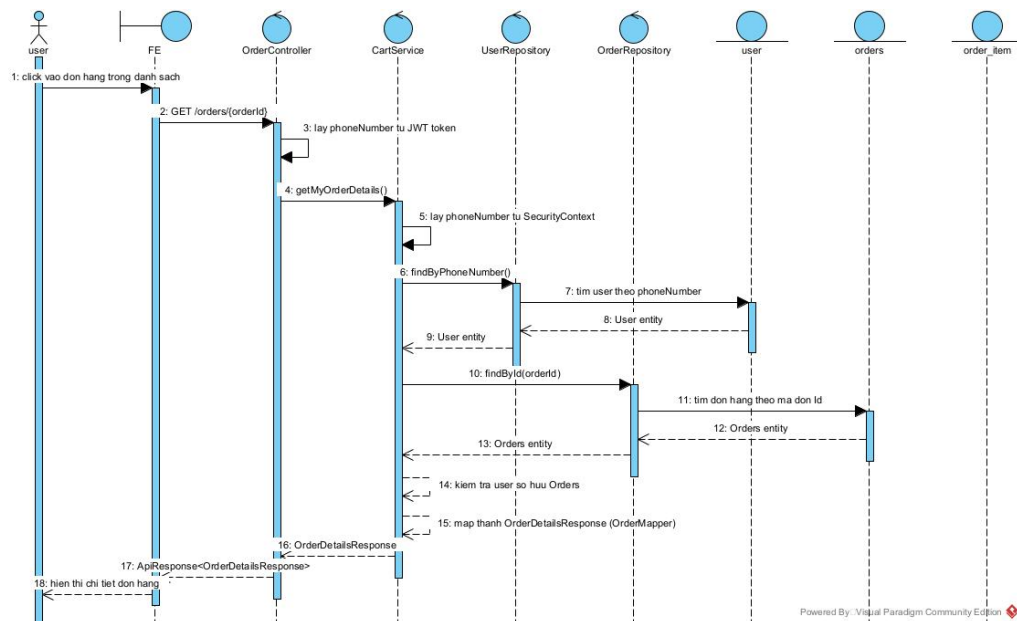


Hình 3.44: Sequence Diagram - Xem lịch sử đơn hàng

Mô tả các bước tương tác:

- Client** gửi HTTP GET request đến `/orders/my-orders` tới **OrderController**.
- Controller** gọi `getMyOrder()` trong **CartService**.
- Service** gọi `findAllByUserId()` trong **OrderRepository**.
- Repository** trả về danh sách đơn hàng.
- Service** map sang `List<OrderDetailsResponse>` (tóm tắt).
- Controller** trả dữ liệu JSON cho **Client**.

16. Luồng Xem chi tiết đơn hàng (Order Details)

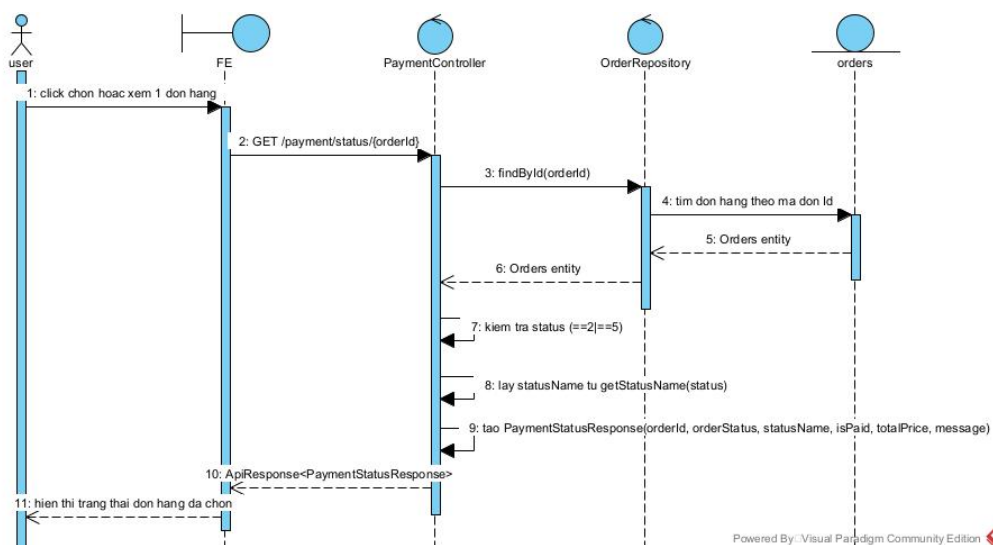


Hình 3.45: Sequence Diagram - Xem chi tiết đơn hàng

Mô tả các bước tương tác:

1. **Client** gửi HTTP GET request đến `/orders/{id}` tới **OrderController**.
2. **Controller** gọi `getMyOrderDetails()` trong **CartService**.
3. **Service** gọi `findById()` trong **OrderRepository**.
4. **Service** kiểm tra quyền sở hữu đơn hàng của user.
5. **Service** map đơn hàng và danh sách items sang `OrderDetailsResponse`.
6. **Controller** trả dữ liệu chi tiết cho **Client**.

17. Luồng Xem trạng thái đơn hàng (Order Status)



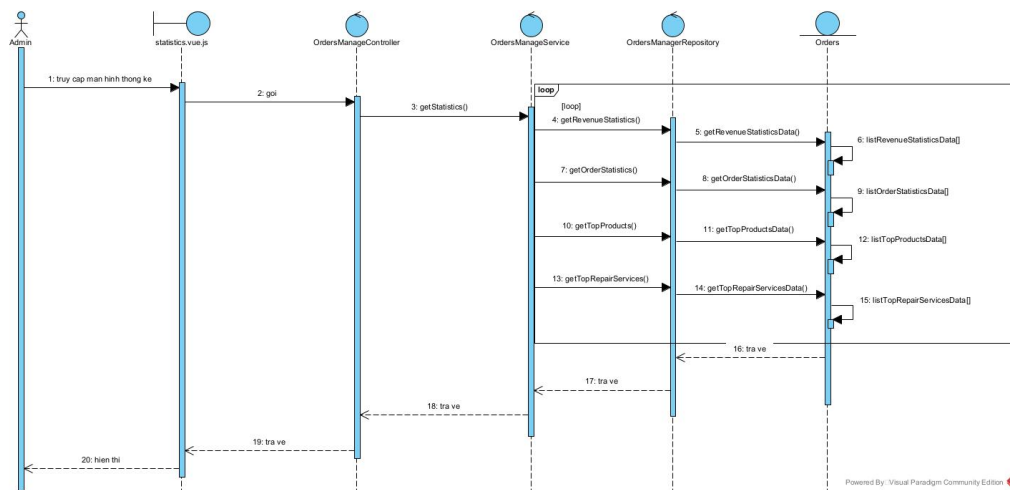
Hình 3.46: Sequence Diagram - Xem trạng thái đơn hàng

Mô tả các bước tương tác:

1. **Client** gửi HTTP GET request đến `/payment/status/{id}` tới **Payment-Controller**.
2. **Controller** tìm đơn hàng qua **OrderRepository**.
3. **Controller** kiểm tra status code để xác định trạng thái thanh toán (isPaid) và tên trạng thái.
4. **Controller** tạo `PaymentStatusResponse` chứa thông điệp và trạng thái.
5. **Controller** trả dữ liệu cho **Client** hiển thị (ví dụ: nhãn "Đã thanh toán").

b, Tác nhân B: Quản trị viên (Admin)

1. Luồng Xem thống kê (View Statistics)



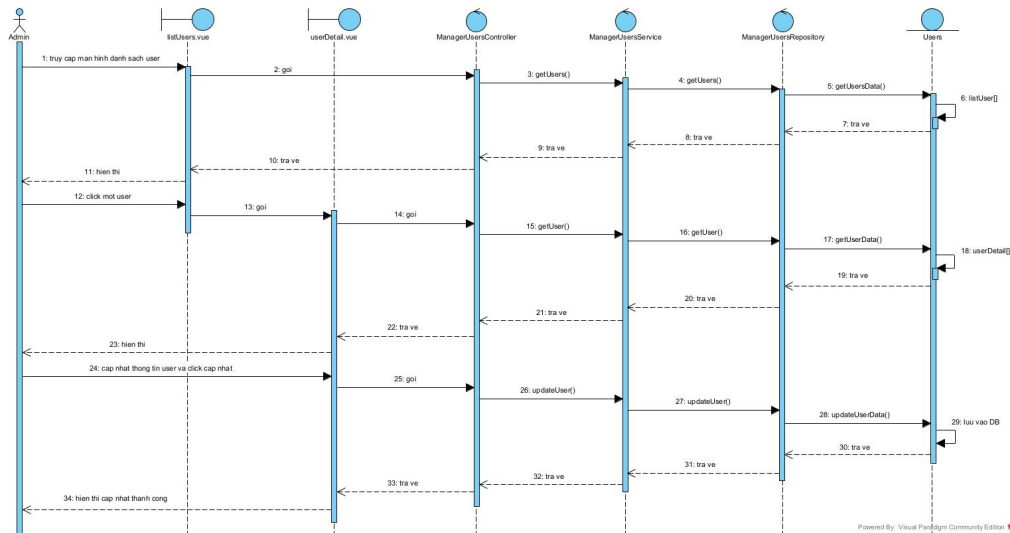
Hình 3.47: Sequence Diagram - Xem thống kê

Mô tả các bước tương tác:

1. **Admin** truy cập màn hình thống kê **statistics.vue**.
2. Hệ thống gửi yêu cầu lấy thông tin thống kê kèm thông tin tìm kiếm tới **OrdersManageController**.
3. Trong **OrdersManageController**, gọi đến các hàm `getRevenueStatistic()`, `getOrderStatistic()`, `getTopProducts()`, `getTopRepairService()` trong **OrderManageService**.
4. **OrderManageService** tiếp tục gọi đến các hàm `getData` tương ứng trong **OrderRepository**.
5. **OrderRepository** thực hiện các truy vấn SQL, nhận về thông tin các thống kê, chuyển đổi thành các đối tượng và gửi trả lại **Service**.

6. **Service** map các bản ghi dữ liệu thành các response phù hợp với từng thống kê, gửi trả lại **Controller**.
7. **Controller** xử lí và trả dữ liệu dưới dạng JSON, **statistics.vue** nhận được dữ liệu và hiển thị các thống kê.

2. Luồng Quản lý người dùng và phân quyền (User Management)



Hình 3.48: Sequence Diagram - Quản lý người dùng và phân quyền

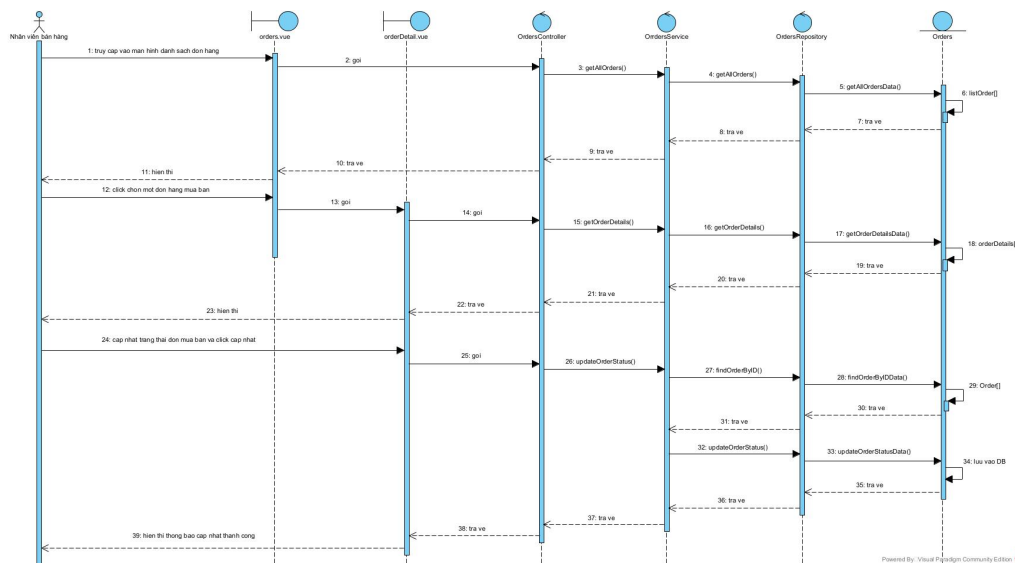
Mô tả các bước tương tác:

1. **Admin** đăng nhập thành công và truy cập vào giao diện **list-user.vue**.
2. Hệ thống gửi yêu cầu lấy thông tin tất cả người dùng tới **UserManageController**.
3. Trong **UserManageController**, gọi đến hàm `getAllUser()` trong **UserService**.
4. **UserService** gọi đến `findAll()` trong **UserRepository**.
5. **UserRepository** thực hiện truy vấn SQL, nhận về các bản ghi người dùng, chuyển đổi thành đối tượng **User** và gửi trả về **Service**.
6. **Service** map dữ liệu thành list **UserResponse** và trả về cho **Controller**.
7. **Controller** trả dữ liệu JSON, **list-user.vue** hiển thị danh sách cho Admin.
8. **Admin** chọn một người dùng để xem chi tiết, hệ thống gửi yêu cầu tới **UserManageController**.
9. **UserManageController** gọi đến `getUserDetails()` trong **UserService**.
10. **Service** gọi tới `findById(userId)` trong **UserRepository**.

11. **UserRepository** trả về bản ghi chi tiết (Entity) cho **Service**, **Service** map thành **UserResponse** gửi về **Controller**.
12. **Controller** trả dữ liệu JSON, **user-detail.vue** hiển thị thông tin chi tiết.
13. **Admin** nhập thông tin cần cập nhật (hoặc cấp quyền mới) và chọn cập nhật. Hệ thống gửi yêu cầu tới **UserManageController**.
14. **Controller** gọi đến **updateUser** trong **UserService**.
15. **Service** gọi **findById()** để lấy bản ghi cũ, sau đó thực hiện cập nhật thông tin và gọi hàm **save()**.
16. **Repository** lưu thông tin vào DB và trả về dữ liệu đã cập nhật cho **Service**.
17. **Service** map dữ liệu thành **UserResponse** và gửi về **Controller**.
18. **Controller** trả dữ liệu JSON, **user-details.vue** hiển thị thông tin đã được cập nhật thành công.

c, Tác nhân C: Nhân viên bán hàng (Sales Staff)

1. Luồng Cập nhật trạng thái đơn hàng



Hình 3.49: Sequence Diagram - Cập nhật trạng thái đơn hàng

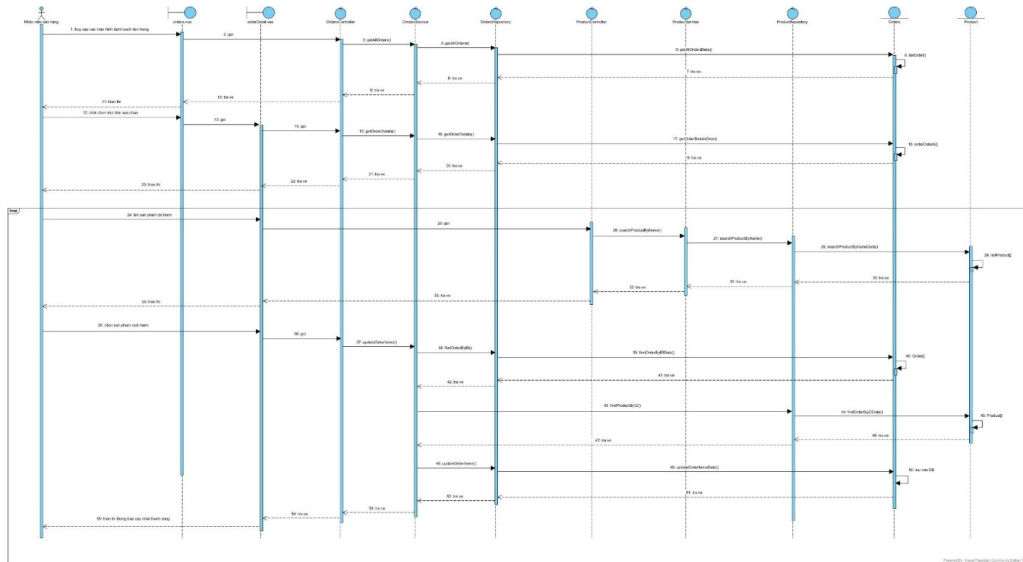
Mô tả các bước tương tác:

1. **Nhân viên** đăng nhập thành công và truy cập vào giao diện **Order.vue**.
2. Hệ thống gửi yêu cầu lấy thông tin tất cả đơn hàng kèm thông tin tìm kiếm tới **OrdersController**.
3. Trong **OrdersController**, gọi đến hàm **getAllOrders()** trong **OrderService**.

4. Hàm `getAllOrders()` (hoặc `getOrdersByType`) gọi đến `findAllByOrderByOrderAtDes` trong **OrderRepository** để xử lý thông tin tìm kiếm.
5. **OrderRepository** thực hiện các truy vấn SQL, nhận về các bản ghi dữ liệu và chuyển đổi chúng thành list đối tượng (Entity), hoàn tất đóng gói dữ liệu và trả về kết quả cho **Service**.
6. **OrderService** map dữ liệu nhận được thành list `OrderResponse` và trả danh sách response đó về cho **Controller**.
7. **Controller** trả dữ liệu này về dưới dạng JSON, **Orders.vue** nhận được và hiển thị cho người dùng.
8. **Nhân viên** chọn một đơn hàng từ danh sách để xem chi tiết, hệ thống gửi yêu cầu lấy thông tin chi tiết đơn hàng tới **OrderController**.
9. **OrderController** gọi đến hàm `getOrderDetails()` trong **OrderService**.
10. Hàm `getOrderDetails()` gọi đến `findById(orderId)` trong **OrderRepository** để tìm kiếm đơn hàng.
11. **OrderRepository** thực hiện truy vấn SQL, nhận về bản ghi chi tiết đơn hàng và chuyển đổi thành đối tượng (Entity), trả về kết quả cho **Service**.
12. **OrderService** map dữ liệu nhận được thành `OrderDetailResponse` và trả về cho **Controller**.
13. **Controller** trả dữ liệu JSON, **Order.vue** nhận được và hiển thị chi tiết đơn hàng.
14. **Nhân viên** nhập ghi chú và trạng thái mới của đơn hàng, chọn nút “Cập nhật trạng thái”.
15. Hệ thống gửi yêu cầu cập nhật trạng thái kèm request (ghi chú, trạng thái mới) tới **OrderController**.
16. **OrderController** gọi tới hàm `updateOrderStatus()` trong **OrderService**.
17. Hàm `updateOrderStatus()` gọi đến `findById(orderId)` trong **OrderRepository** để tìm đơn hàng cần cập nhật.
18. **OrderRepository** trả về entity `Order` cho **Service**.
19. **OrderService** cập nhật thông tin status và note, sau đó gọi hàm `save(order)` trong **OrderRepository**.
20. Hàm `save(order)` lưu thông tin mới vào database và trả về bản ghi đã cập nhật cho **Service**.

21. **OrderService** map dữ liệu thành `OrderStatusUpdateResponse` và trả kết quả về cho **Controller**.
22. **Controller** trả dữ liệu JSON, **Order.vue** hiển thị thông báo cập nhật thành công.

2. Luồng Cập nhật sản phẩm trong đơn sửa chữa



Hình 3.50: Sequence Diagram - Cập nhật sản phẩm trong đơn sửa chữa

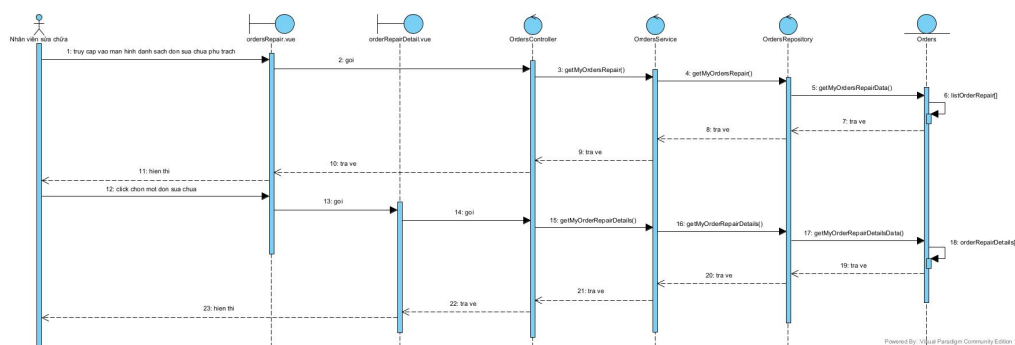
Mô tả các bước tương tác:

1. **Nhân viên** đăng nhập thành công và truy cập vào giao diện **Order.vue**.
2. Hệ thống gửi yêu cầu lấy thông tin đơn hàng tới **OrdersController**.
3. **OrdersController** gọi đến hàm `getAllOrders()` trong **OrderService**.
4. Hàm `getOrdersByType()` gọi đến `findDistinctByType()` trong **OrderRepository** để lọc các đơn hàng sửa chữa.
5. **OrderRepository** thực hiện truy vấn, trả về danh sách các đơn hàng sửa chữa (Entity) cho **Service**.
6. **OrderService** map dữ liệu thành list `OrderResponse`, trả về cho **Controller**.
7. **Controller** trả dữ liệu JSON, **Orders.vue** hiển thị danh sách đơn sửa chữa.
8. **Nhân viên** chọn một đơn hàng để xem chi tiết, hệ thống gửi yêu cầu tới **OrderController**.
9. **OrderController** gọi `getOrderDetails()` trong **OrderService**.
10. **OrderService** gọi `findById(orderId)` trong **OrderRepository**, nhận về chi tiết đơn hàng và trả về `OrderDetailResponse` cho giao diện.

11. **Nhân viên** nhập tên sản phẩm vào ô tìm kiếm.
12. Hệ thống gửi yêu cầu tìm kiếm sản phẩm tới **ProductController** kèm từ khóa.
13. **ProductController** gọi hàm `searchProductByName()` trong **ProductService**.
14. **ProductService** gọi `findByName(name)` trong **ProductRepository**.
15. **ProductRepository** truy vấn và trả về danh sách sản phẩm phù hợp cho **Service**.
16. **ProductService** map thành `ProductResponse` và trả về cho **Controller**, hiển thị lên form cập nhật.
17. **Nhân viên** chọn sản phẩm cần thêm và nhấn nút thêm.
18. Hệ thống gửi yêu cầu cập nhật sản phẩm (kèm ID) tới **OrderController**.
19. **OrderController** gọi hàm `updateOrderItems()` trong **OrderService**.
20. **OrderService** gọi `findById(orderId)` trong **OrderRepository** để lấy đơn hàng, và gọi `findById(productId)` trong **ProductRepository** để lấy sản phẩm.
21. **OrderService** cập nhật danh sách sản phẩm trong đơn hàng, sau đó gọi `save(order)` để lưu vào database.
22. **OrderRepository** lưu và trả về dữ liệu đơn hàng đã cập nhật.
23. **OrderService** map thành `OrderStatusUpdateResponse` và trả về cho **Controller**.
24. **Controller** trả kết quả JSON, giao diện hiển thị thông báo cập nhật thành công.

d, Tác nhân D: Nhân viên sửa chữa (Technician)

1. Luồng Xem chi tiết đơn sửa chữa phụ trách

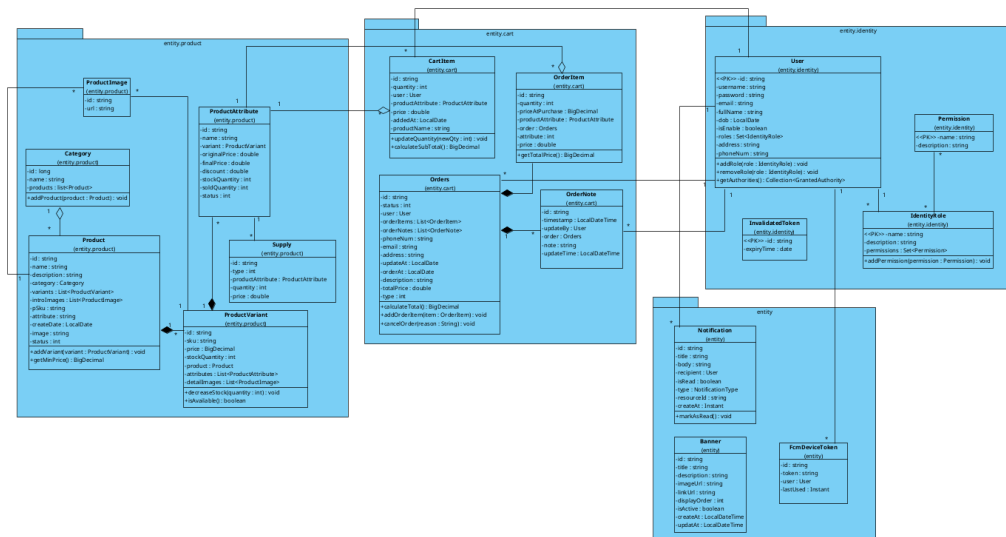


Hình 3.51: Sequence Diagram - Xem chi tiết đơn sửa chữa phụ trách

Mô tả các bước tương tác:

1. **Nhân viên** đăng nhập thành công và truy cập vào giao diện **Order.vue**, chọn ô hiển thị đơn sửa chữa.
2. Hệ thống gửi yêu cầu lấy thông tin tất cả đơn hàng kèm thông tin tìm kiếm tới **OrdersController**.
3. Trong **OrdersController**, gọi đến hàm `getMyOrderRepair()` trong **OrderService**.
4. Hàm `getMyOrderRepair` gọi đến `getMyOrderRepairData90` trong **OrderRepository** để xử lý thông tin tìm kiếm.
5. **Repository** thực hiện các truy vấn SQL, nhận về dữ liệu gồm các đơn sửa chữa theo người dùng, đóng gói dữ liệu và trả về **Service**.
6. **OrderService** map dữ liệu nhận được thành list `OrderResponse` và trả danh sách response đó về cho **Controller**.
7. **Controller** trả dữ liệu này về dưới dạng JSON, **Orders.vue** nhận được và hiển thị cho người dùng.
8. **Nhân viên** chọn một đơn hàng từ danh sách để xem chi tiết, hệ thống gửi yêu cầu lấy thông tin chi tiết đơn hàng được chọn tới **OrdersController**.
9. **OrdersController** gọi đến hàm `getOrderDetails()` trong **OrderService()**.
10. Hàm `getOrderDetails()` gọi đến `findById(orderId)` trong **OrderRepository** để xử lý thông tin tìm kiếm.
11. Hàm `findById(orderId)` thực hiện truy vấn SQL, nhận về bản ghi chi tiết đơn hàng và chuyển đổi thành đối tượng (Entity), hoàn tất đóng gói dữ liệu và trả về kết quả cho **Service**.
12. **OrderService** map dữ liệu nhận được thành `OrderDetailResponse` và trả về cho **Controller**.
13. **Controller** trả dữ liệu này về dưới dạng JSON, **Order.vue** nhận được và hiển thị chi tiết đơn hàng cho nhân viên.

3.5.2 Biểu đồ lớp (Class Diagram)



Hình 3.52: Class Diagram - Biểu đồ lớp (Domain Model)

a, Mục tiêu và phạm vi

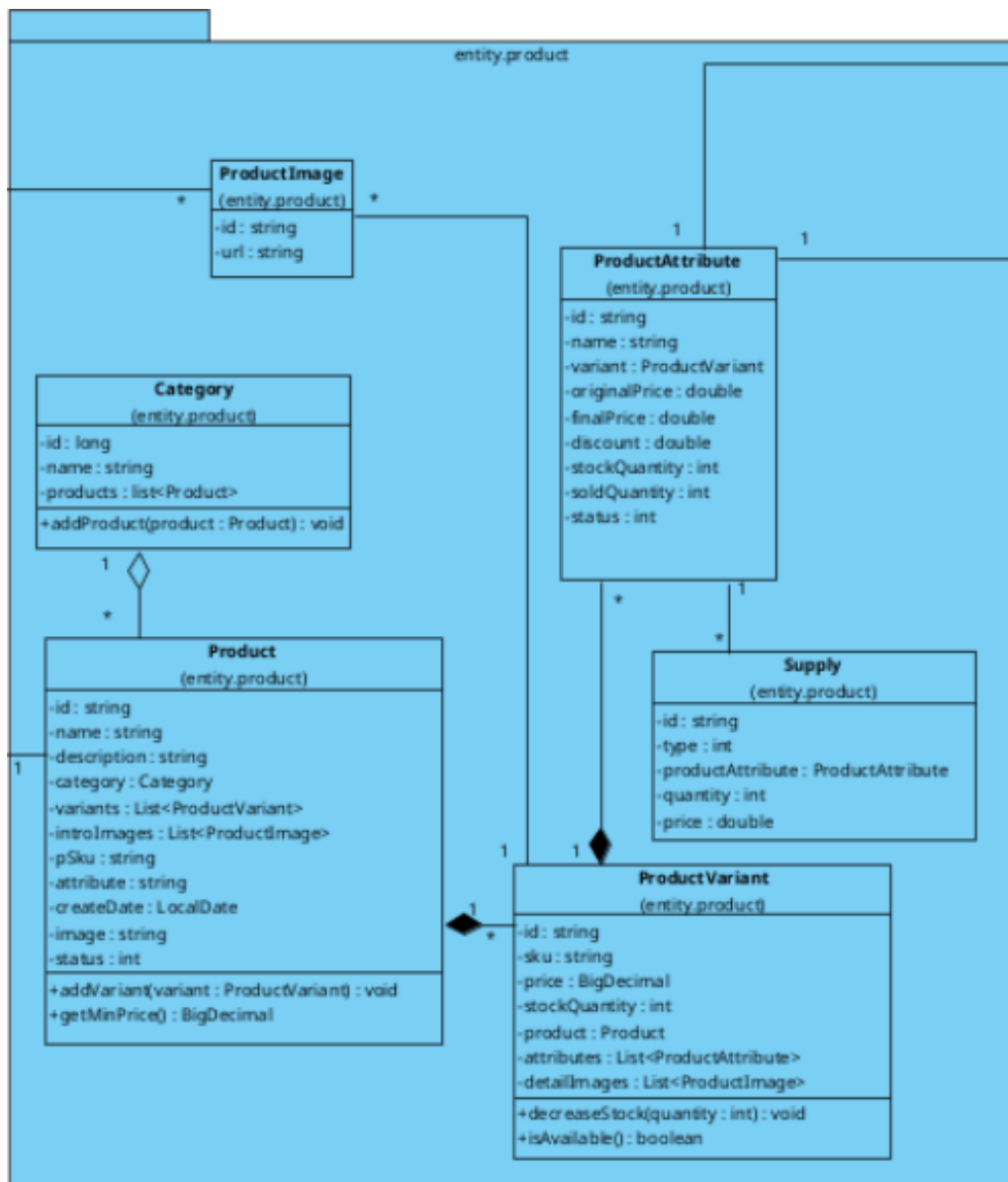
Class diagram mô tả cấu trúc dữ liệu lõi (domain model) của hệ thống backend, tập trung vào các module chính:

- **Sản phẩm (Product):** Danh mục, sản phẩm, biến thể, thuộc tính, ảnh, nhập/xuất kho.
- **Giỏ hàng & Đơn hàng (Cart/Order):** Giỏ hàng, đơn hàng, chi tiết đơn hàng, ghi chú đơn hàng.
- **Định danh & Phân quyền (Identity):** Người dùng, vai trò, quyền, token blacklist.
- **Thông báo (Notification):** Thông báo hệ thống, token thiết bị (FCM), banner.

Sơ đồ được chia thành các package tương ứng (entity.product, entity.cart, entity.identity, entity) giúp tách bạch trách nhiệm và dễ theo dõi quan hệ.

b, Tổng quan kiến trúc domain theo package

1. entity.product – Quản lý sản phẩm



Hình 3.53: Class Diagram - Biểu đồ lớp về sản phẩm (Domain Model)

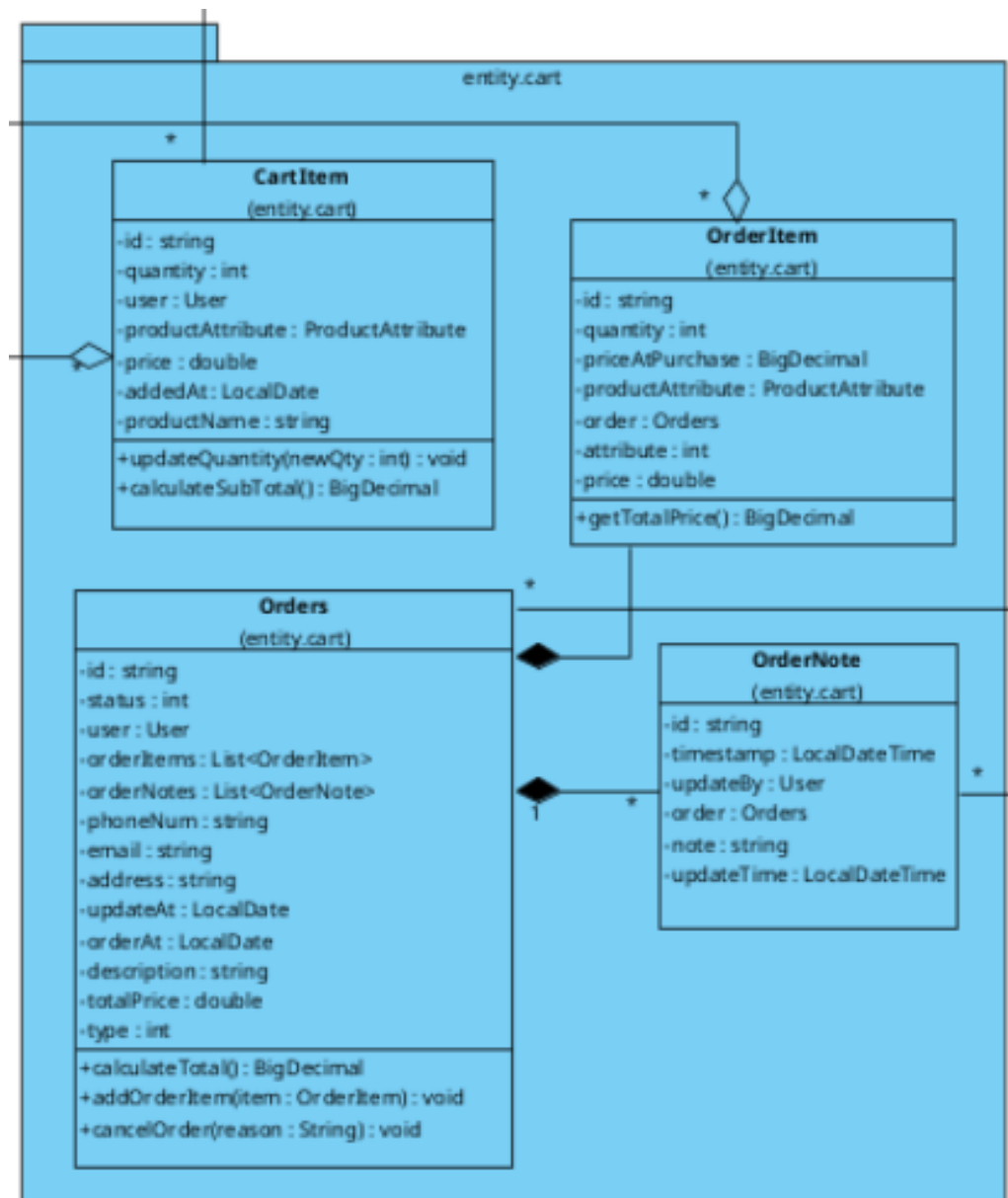
Package này mô hình hoá cấu trúc sản phẩm theo dạng phân cấp:

- **Category:** Danh mục sản phẩm.
- **Product:** Sản phẩm tổng (chứa thông tin cơ bản).
- **ProductVariant:** Biến thể theo màu sắc hoặc SKU.
- **ProductAttribute:** Thuộc tính/phần nhỏ hơn (ví dụ: dung lượng, cấu hình), mang thông tin về giá, giảm giá, tồn kho, số lượng đã bán.
- **ProductImage:** Ảnh giới thiệu (introImages) hoặc ảnh chi tiết (detailImages).
- **Supply:** Ghi nhận lịch sử nhập/xuất kho gắn liền với ProductAttribute.

Ý nghĩa nghiệp vụ: Một Category chứa nhiều Product. Một Product có thể có nhiều Variant. Mỗi Variant có nhiều Attribute (các option cụ thể để bán). Quản trị kho

được thực hiện ở cấp Attribute thông qua Supply.

2. entity.cart – Giỏ hàng và đơn hàng



Hình 3.54: Class Diagram - Biểu đồ lớp về giỏ và đơn hàng (Domain Model)

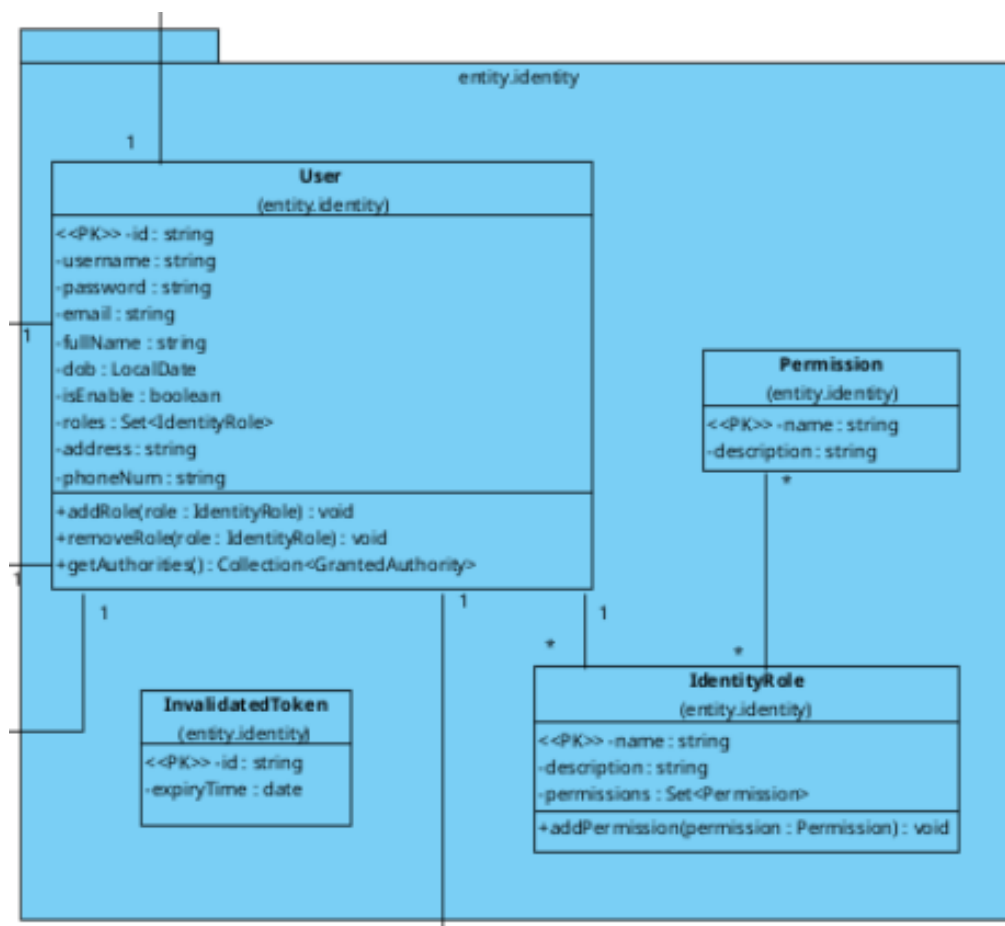
Gồm 4 lớp chính:

- **CartItem:** Item trong giỏ hàng của một user, tham chiếu tới ProductAttribute và lưu trữ số lượng (quantity), giá (price).
- **Orders:** Đơn hàng tổng, chứa thông tin người nhận, tổng tiền, trạng thái, loại đơn.
- **OrderItem:** Từng dòng sản phẩm trong đơn hàng, tham chiếu tới ProductAttribute.

- **OrderNote:** Ghi chú cập nhật đơn hàng, lưu thông tin người cập nhật (User) và thời gian.

Ý nghĩa nghiệp vụ: Giỏ hàng là bước đệm trước khi tạo đơn; khi đặt hàng, các CartItem được chuyển đổi thành OrderItem. Orders đóng vai trò "thực thể cha", quản lý danh sách dòng hàng và các ghi chú.

3. entity.identity – Người dùng & Phân quyền



Hình 3.55: Class Diagram - Biểu đồ lớp về xác thực và phân quyền (Domain Model)

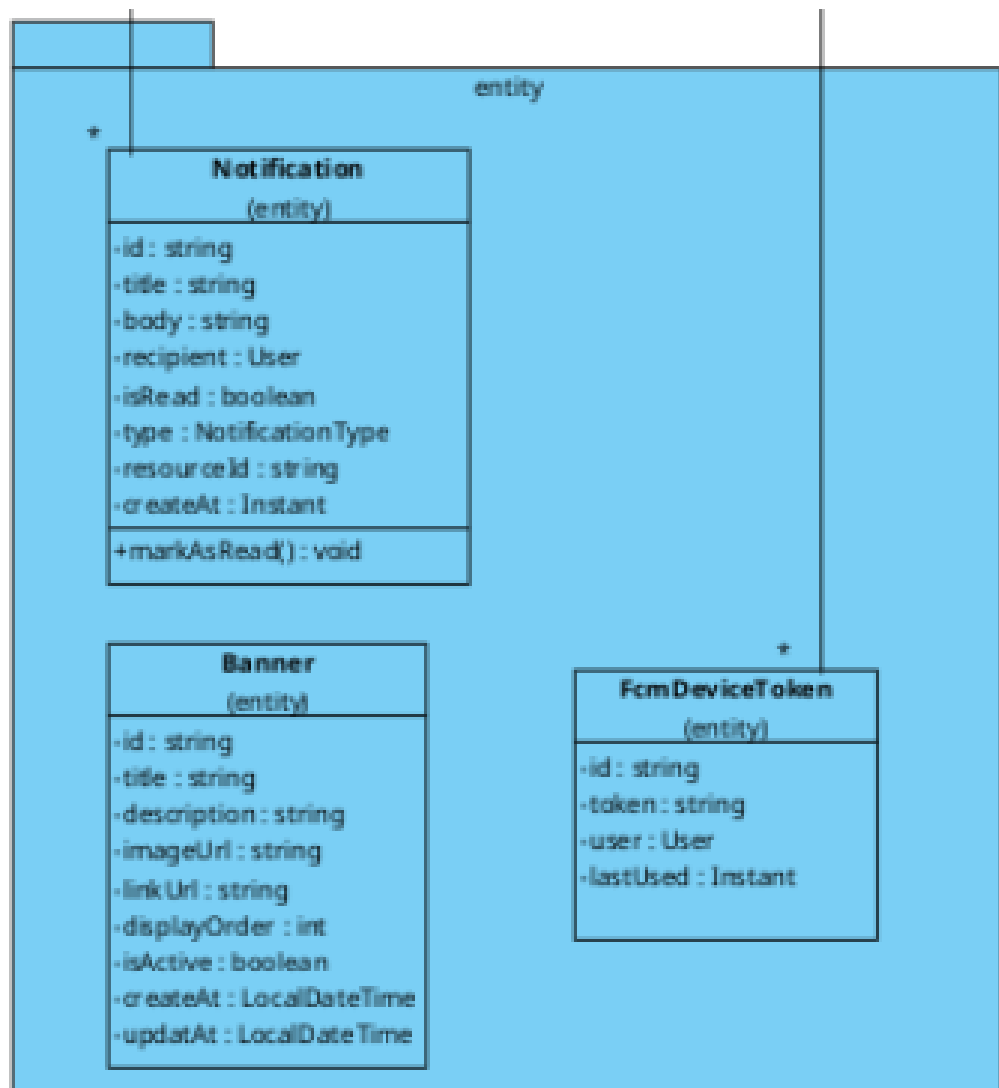
Nhóm các lớp dùng cho xác thực và phân quyền:

- **User:** Thông tin người dùng, tài khoản đăng nhập, thông tin liên hệ và tập hợp các Roles.
- **IdentityRole:** Vai trò (Role) – tập hợp các quyền hạn.
- **Permission:** Quyền cụ thể (Permission).
- **InvalidatedToken:** Lưu trữ các token đã bị vô hiệu hoá (blacklist) phục vụ chức năng đăng xuất hoặc thu hồi JWT.

Ý nghĩa nghiệp vụ: Quan hệ giữa User và Role là quan hệ nhiều-nhiều. Role gom

nhóm các Permission để phân quyền theo chức năng.

4. entity – Notification / Banner / Device Token



Hình 3.56: Class Diagram - Biểu đồ lớp còn lại (Domain Model)

Gồm các lớp hỗ trợ tương tác hệ thống:

- **Notification:** Thông báo gửi cho user (recipient), bao gồm trạng thái đã đọc, loại thông báo (NotificationType) và metadata (như resourceId).
- **FcmDeviceToken:** Token thiết bị để gửi Push Notification (qua Firebase), gắn với User và có thời gian sử dụng gần nhất.
- **Banner:** Banner hiển thị trên giao diện (tiêu đề, ảnh, liên kết, thứ tự hiển thị...).

c, Phân tích quan hệ UML chính trong sơ đồ

1. Composition (Quan hệ cấu thành -)

Các cụm quan hệ "cha-con" thể hiện cấu trúc dữ liệu chặt chẽ, trong đó vòng đời của đối tượng con phụ thuộc vào đối tượng cha:

- Orders — OrderItem và Orders — OrderNote: Một đơn hàng chứa danh sách dòng hàng và ghi chú; khi đơn hàng bị xóa, các thành phần con cũng mất đi ý nghĩa tồn tại độc lập.
- Product — ProductVariant — ProductAttribute: Biến thể là thành phần cấu thành sản phẩm; thuộc tính (giá/cấu hình) lại phụ thuộc vào biến thể.
- Product/ProductVariant — ProductImage: Ảnh đi kèm thuộc về sản phẩm hoặc biến thể cụ thể.

2. Association (Quan hệ liên kết - —)

Các liên kết tham chiếu dữ liệu giữa các thực thể độc lập:

- User — Orders: Một user có thể có nhiều đơn hàng (1-n).
- OrderItem — ProductAttribute: Mỗi dòng hàng trỏ tới một thuộc tính sản phẩm cụ thể.
- CartItem — User và CartItem — ProductAttribute: Giỏ hàng thuộc về user và chứa tham chiếu đến sản phẩm.
- FcmDeviceToken — User: Một user có thể có nhiều token thiết bị (đa nền tảng).

d, Luồng nghiệp vụ được thể hiện qua Class Diagram

- **Luồng đặt hàng:** User chọn sản phẩm theo cấu trúc *Category* → *Product* → *ProductVariant* → *ProductAttribute*. Khi thêm vào giỏ, hệ thống tạo *CartItem*. Khi đặt hàng, hệ thống tạo *Orders* và chuyển đổi *CartItem* thành *OrderItem*. Quá trình xử lý đơn hàng sẽ sinh ra các *OrderNote*.
- **Luồng quản lý kho:** Thực thể *Supply* gắn trực tiếp với *ProductAttribute*, cho phép theo dõi lịch sử nhập/xuất và điều chỉnh số lượng tồn kho ở cấp độ chi tiết nhất (phiên bản/cấu hình).
- **Luồng thông báo:** *Notification* lưu lịch sử thông báo, trong khi *FcmDeviceToken* hỗ trợ việc gửi thông báo đẩy tới thiết bị di động của người dùng.

e, Kết luận

Class diagram cho thấy hệ thống được thiết kế theo hướng **Domain-Driven Design** với sự phân chia rõ ràng:

- Nhóm **Product**: Mô hình hoá sản phẩm phân cấp chi tiết.
- Nhóm **Cart/Order**: Biểu diễn đầy đủ vòng đời mua hàng và xử lý đơn hàng.
- Nhóm **Identity**: Quản lý định danh và phân quyền linh hoạt.
- Nhóm **Notification**: Hỗ trợ tương tác người dùng đa kênh.

Cấu trúc này giúp tách biệt trách nhiệm giữa các module, đồng thời thể hiện rõ ràng các mối quan hệ nghiệp vụ phức tạp của hệ thống cửa hàng bán điện thoại.

3.6 Kết luận chương

Chương 3 đã hoàn thành việc phân tích và thiết kế toàn diện hệ thống. Thông qua việc xác định rõ ràng các yêu cầu chức năng và phi chức năng, mô hình hóa quy trình nghiệp vụ bằng Use Case, và thiết kế chi tiết cơ sở dữ liệu cũng như kiến trúc hệ thống, chúng ta đã có được một bản thiết kế kỹ thuật hoàn chỉnh.

Các biểu đồ UML (Use Case, Sequence, Class) và thiết kế cơ sở dữ liệu trong chương này đóng vai trò là "bản vẽ kỹ thuật" chính xác, đảm bảo tính nhất quán và logic cho quá trình thi công phần mềm. Chương tiếp theo sẽ trình bày kết quả hiện thực hóa các thiết kế này thành sản phẩm phần mềm thực tế, cùng với các giao diện người dùng và kết quả kiểm thử hệ thống.

CHƯƠNG 4. TRIỂN KHAI VÀ ĐÁNH GIÁ

4.1 Giới thiệu chương

Tiếp nối quá trình phân tích và thiết kế hệ thống ở Chương 3, Chương 4 sẽ tập trung vào giai đoạn hiện thực hóa và triển khai sản phẩm. Đây là bước chuyển đổi các mô hình lý thuyết và bản vẽ kỹ thuật thành một phần mềm thực tế, có khả năng hoạt động và tương tác với người dùng.

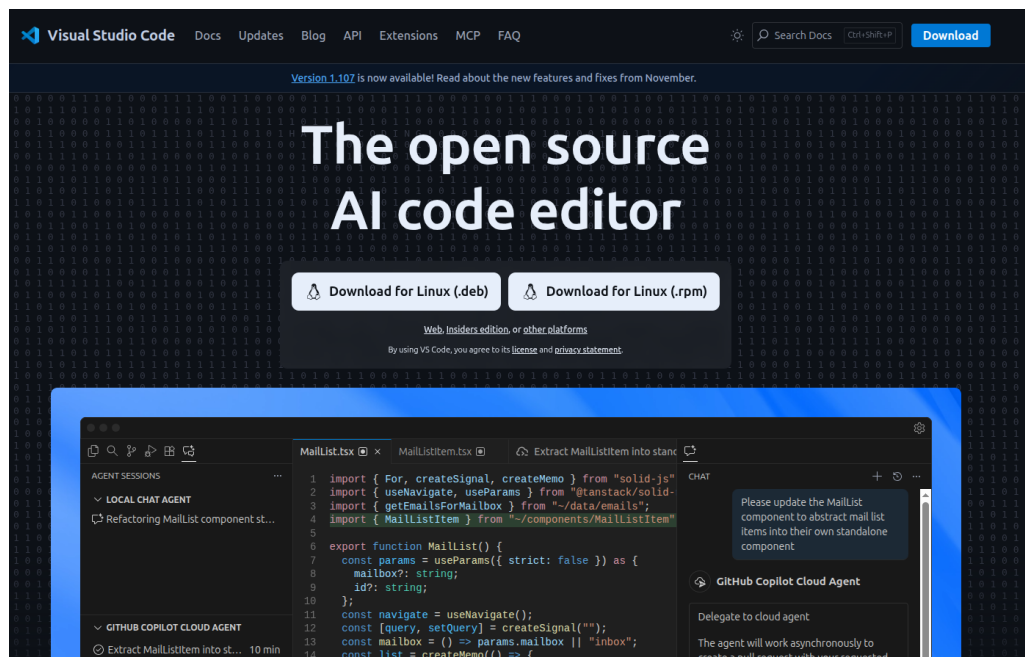
Nội dung chương sẽ trình bày chi tiết về môi trường phát triển (IDE), các công cụ hỗ trợ lập trình, quy trình cài đặt và cấu hình hệ thống cho cả ba thành phần: Backend (Spring Boot), Web Client (Vue.js) và Mobile App (Android). Tiếp đó, chương sẽ giới thiệu các giao diện chức năng chính của chương trình sau khi hoàn thiện và trình bày kết quả kiểm thử để đánh giá mức độ đáp ứng yêu cầu của hệ thống.

4.2 IDE và Công cụ phát triển

4.2.1 Visual Studio Code

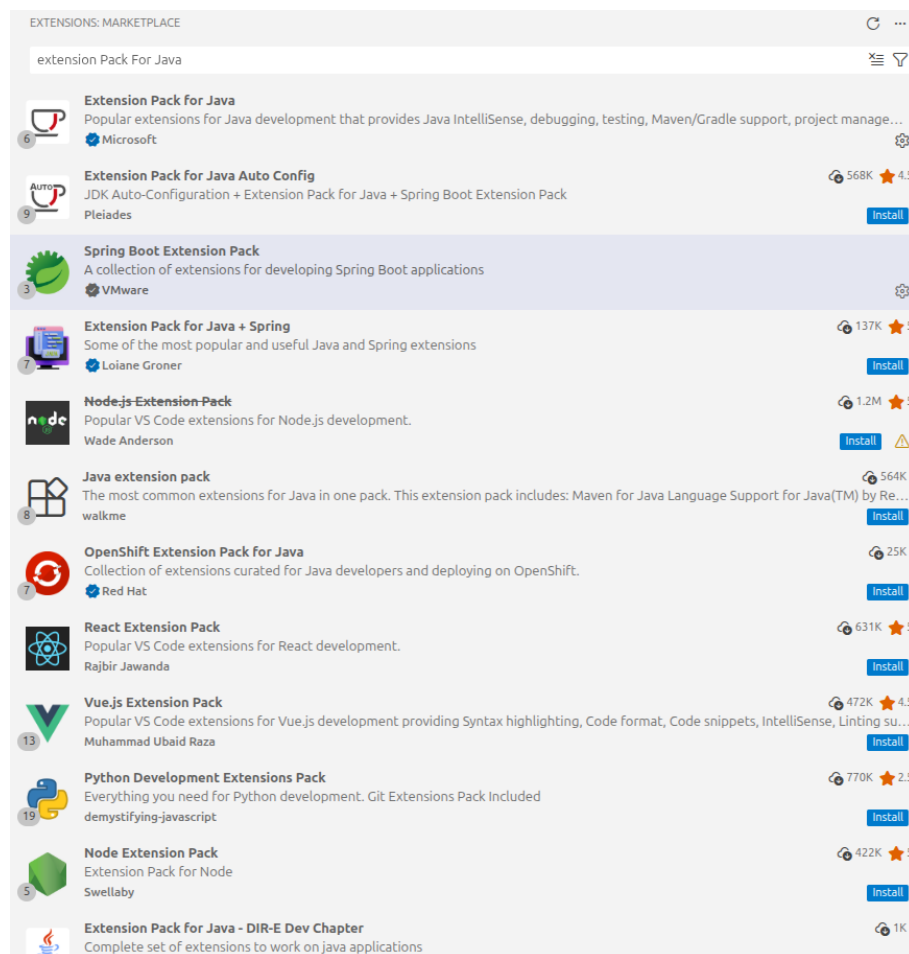
Sử dụng để phát triển mã nguồn phía Backend (Java Spring Boot) và UI Web (Vue.js).

- **Mô tả:** IDE nhẹ, hỗ trợ nhiều extension mạnh mẽ cho hệ sinh thái Java và JavaScript. Cung cấp các chức năng đầy đủ bao gồm gợi ý code, debugger, git bash...vv Sử dụng một công cụ duy nhất mang lại sự đồng bộ và trải nghiệm mượt mà hơn so với việc sử dụng nhiều IDE nặng và riêng rẽ.
- **Cài đặt:** Truy cập [Visual Studio Code](#) để tải bản phù hợp.



Hình 4.1: Trang chủ Visual Studio Code

- **Các Extension cần thiết:**

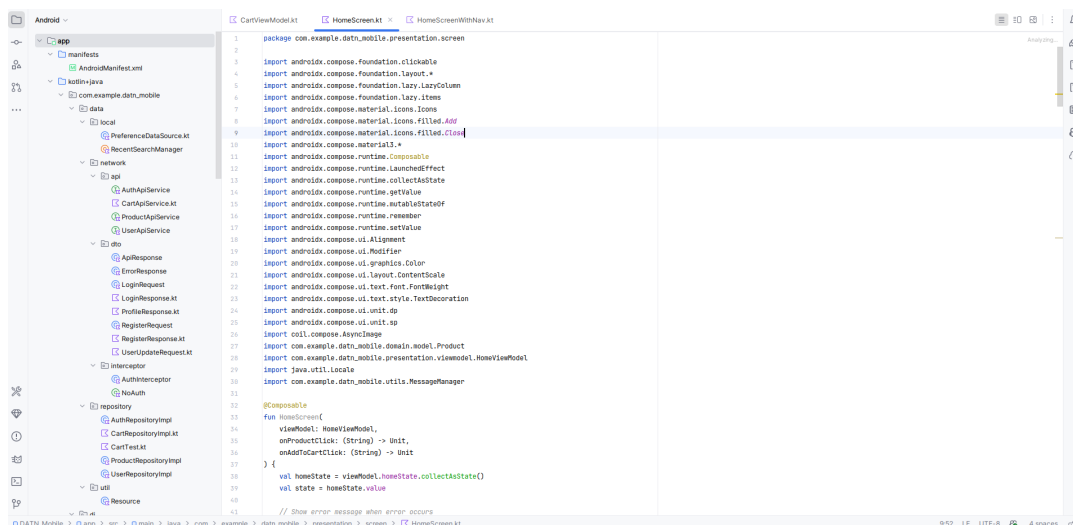


Hình 4.2: Giao diện cài đặt Extensions trên Visual Studio Code

- Extension Pack for Java: Quản lý mã nguồn, debug và IntelliSense cho Java.
- Spring Boot Extension Pack: Hỗ trợ xây dựng và cấu hình Spring Boot Application.
- Vue.js Extension Pack: Hỗ trợ cú pháp và template cho Framework Vue.js.

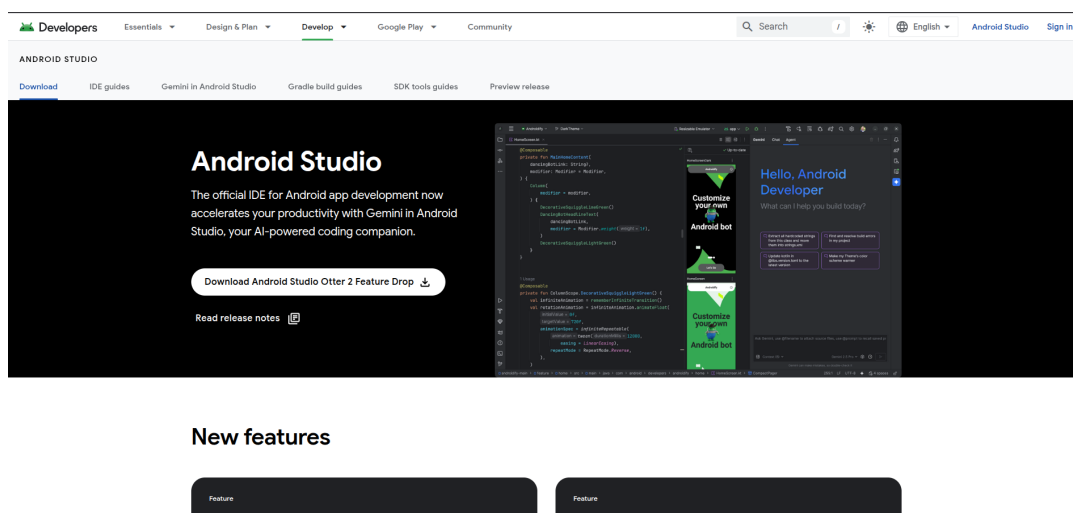
4.2.2 Android Studio

Sử dụng để phát triển ứng dụng di động Android Native App.



Hình 4.3: Giao diện Android Studio

- **Mô tả:** IDE chính thức từ Google, tối ưu cho việc xây dựng ứng dụng Android bằng ngôn ngữ Java/Kotlin. Cung cấp SDK cũng như giả lập máy ảo và debugger cho việc lập trình ứng dụng Android Native trên thiết bị thật hoặc máy ảo.
- **Cài đặt:** Lựa chọn phiên bản phù hợp tại trang chủ [Android Developers](#)



Hình 4.4: Trang chủ Android Studio

4.2.3 Docker

Công cụ hỗ trợ đóng gói và triển khai ứng dụng.

- **Mô tả:** Sử dụng để tạo các container chứa môi trường chạy độc lập cho Backend và Database, giúp quy trình Deploy diễn ra đồng nhất.

4.3 Cài đặt hệ thống chi tiết

4.3.1 Backend (Java Spring Boot)

Cấu hình các thư viện và package quan trọng để tối ưu hóa hiệu năng:

- **Project Lombok:** Tự động sinh mã (getters, setters, constructors) giúp rút gọn source code.
- **Netty:** Thư viện mạng hướng sự kiện (event-driven) để xử lý các kết nối bất đồng bộ hiệu suất cao.
- **Spring Data JPA:** Quản lý và tương tác với cơ sở dữ liệu quan hệ.
- **Spring Security:** Thiết lập cơ chế xác thực và phân quyền cho hệ thống.

4.3.2 Web UI (Vue.js)

4.3.3 Android App (Native)

Tích hợp các thư viện nổi bật để xử lý tác vụ:

- **Retrofit:** Thư viện HTTP Client để gửi và nhận dữ liệu từ RESTful API của Backend.
- **Glide/Picasso:** Thư viện hỗ trợ tải và hiển thị hình ảnh tối ưu.
- **Room Persistence Library:** Cung cấp lớp trừu tượng để quản lý SQLite cục bộ một cách dễ dàng.

4.4 Triển khai hệ thống và Deploy

Hệ thống được đóng gói thông qua Docker và triển khai trên môi trường máy chủ.

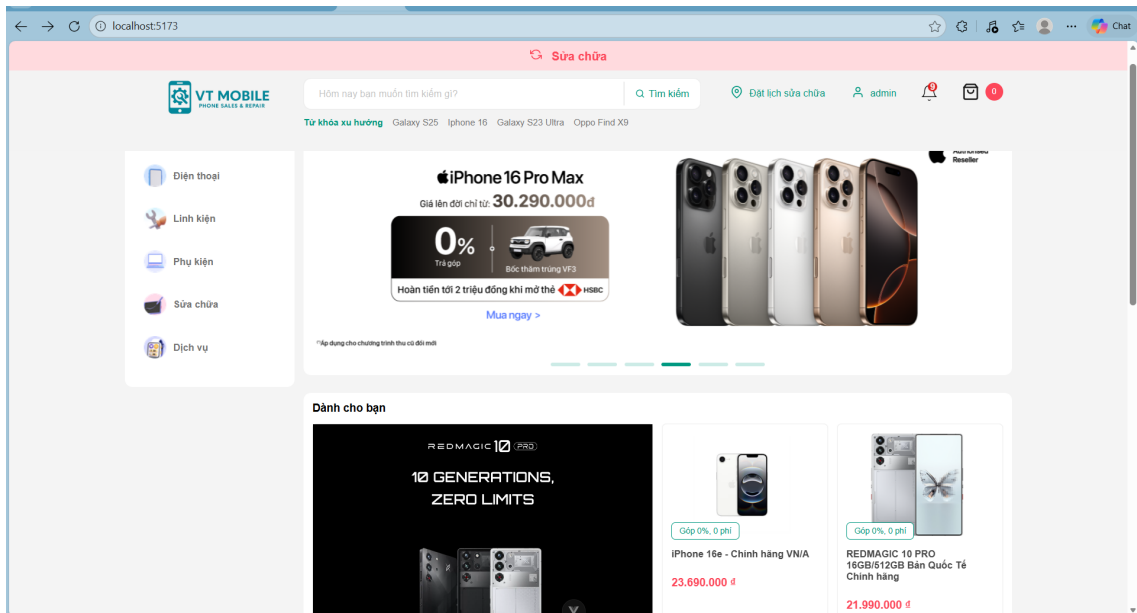
- **Build Artifact:** Đóng gói Backend thành file JAR và Web UI thành các static files thông qua tiến trình Build.
- **Dockerizing:** Viết Dockerfile cho từng phần và sử dụng Docker Compose để quản lý các dịch vụ (App, DB, Cache).
- **Deployment:** Triển khai container lên VPS hoặc môi trường Cloud.

4.5 Giao diện chương trình (Demo)

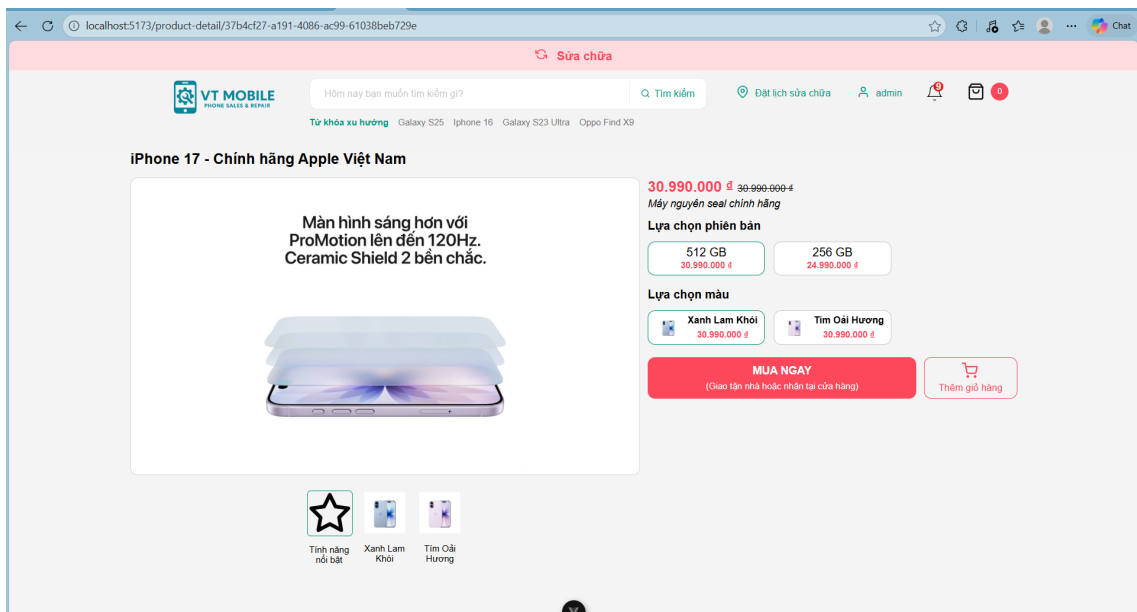
4.5.1 Giao diện Web (Khách hàng)

Phần giao diện dành cho người dùng cuối (End-user) bao gồm các chức năng xem sản phẩm, mua hàng, đặt lịch sửa chữa và quản lý thông tin cá nhân.

a, Trang chủ và Sản phẩm

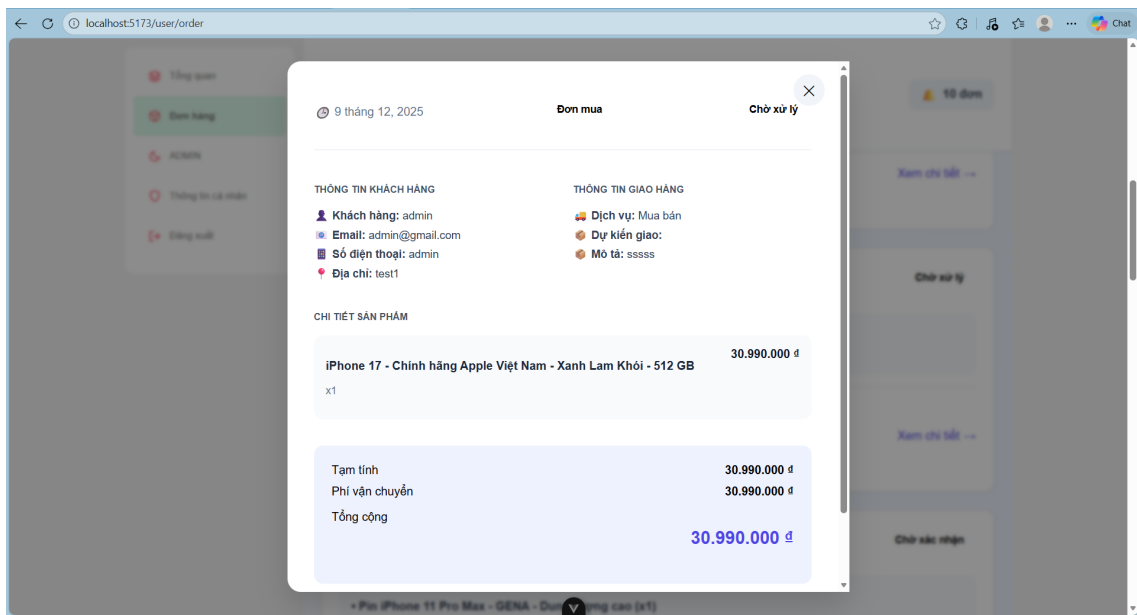


Hình 4.5: Giao diện Trang chủ với danh sách sản phẩm và banner khuyến mãi



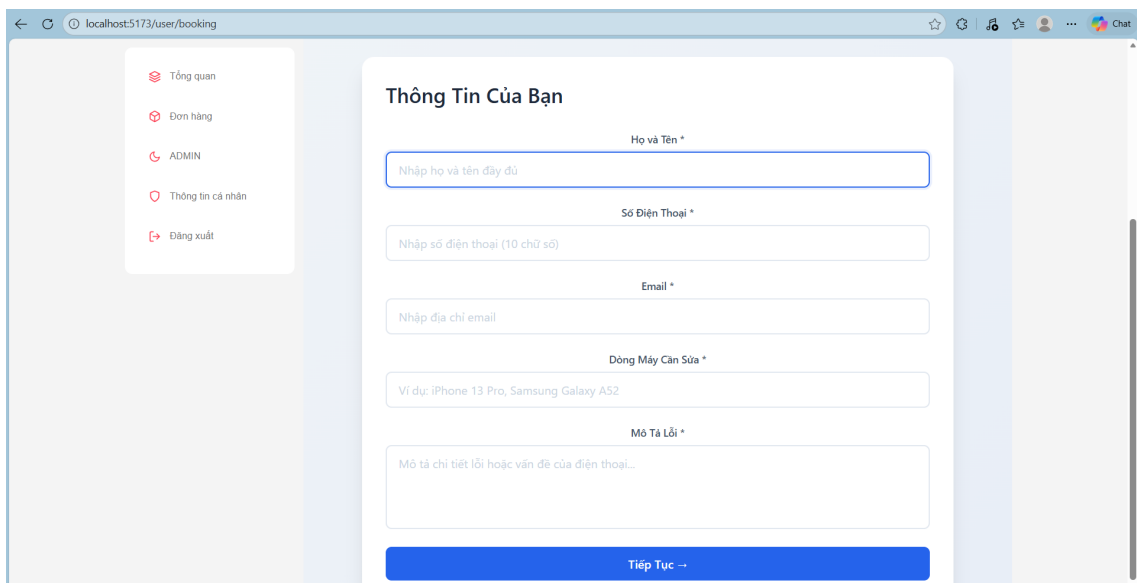
Hình 4.6: Giao diện Chi tiết sản phẩm: Xem thông tin, chọn phiên bản/màu sắc

b, Chức năng Giỏ hàng và Đặt hàng



Hình 4.7: Giao diện Giỏ hàng: Xem danh sách sản phẩm và nhập thông tin đặt hàng

c, Chức năng Đặt lịch sửa chữa



Hình 4.8: Form Đặt lịch sửa chữa: Nhập thông tin thiết bị và mô tả lỗi

d, Quản lý tài khoản và Đơn hàng

The screenshot shows the 'Cập nhật thông tin cá nhân' (Update Personal Information) form in the VT MOBILE application. The form is located on the right side of the screen, with a sidebar on the left containing navigation options: Tổng quan, Đơn hàng, Thông tin cá nhân (selected), and Đăng xuất. The form fields include: Họ tên (Last Name) with the value 'Tiến Dũng', Điện thoại (Phone Number) with '0123123123', Email with 'dtd001@gmail.com', Địa chỉ (Address) with 'An Duong, Yen Phu, Tay Ho, Ha Noi', Ngày sinh (Date of Birth) with '01/01/2003', and Mật khẩu mới (New Password) and Xác nhận mật khẩu mới (Confirm New Password) fields. A green 'XÁC NHẬN' (Confirm) button is at the bottom of the form. The top of the screen features a search bar, a 'Sửa chữa' (Repair) button, and a navigation bar with links to 'Đặt lịch sửa chữa', 'Tiến Dũng', and a notification icon.

Hình 4.9: Giao diện Cập nhật thông tin cá nhân

The screenshot shows the 'Đơn hàng đã đặt' (Orders Placed) page in the VT MOBILE application. The page is located on the right side of the screen, with a sidebar on the left containing navigation options: Tổng quan, Đơn hàng (selected), ADMIN, Thông tin cá nhân, and Đăng xuất. The main content area displays the title 'Đơn hàng đã đặt' and a subtitle 'Quản lý và theo dõi các đơn hàng của bạn'. A notification icon shows '10 đơn'. Below the title, there are five buttons: 'Tất cả' (selected), 'Chờ xác nhận', 'Đang xử lý', 'Đang giao', and 'Hoàn thành'. The main content area shows a list of orders, with the first order dated '10 tháng 12, 2025' and status 'Mua bán'. The order details include: 'Samsung Galaxy Z Fold7 5G - Xám bạc - 12 / 512 GB (x1)' and 'Samsung Galaxy S25 - Xanh Dương Nhật - 512 GB (x1)'. The total price is '30.990.000 đ'. A 'Xem chi tiết' (View details) link is at the bottom right of the order details.

Hình 4.10: Danh sách đơn hàng cá nhân

Chi tiết đơn hàng

LOẠI ĐƠN: Sửa chữa

TÊN KHÁCH HÀNG: Nguyễn Trọng Đức | SỐ ĐIỆN THOẠI: 0123123123 | ĐỊA CHỈ: An Duong, Yen Phu, Tay Ho, Ha Noi

LOẠI ĐỊA CHỈ: Cửa hàng | TRANG THÁI: Hoàn thành | MÔ TẢ: iPhone 11 | màn hình không lên

CẬP NHẬT GHI CHÚ

LỊCH SỬ CẬP NHẬT

Người thực hiện	Nội dung	Thời gian
admin	Cập nhật linh kiện sử dụng	16:56 16/12/2025

CẬP NHẬT TRẠNG THÁI

Hình 4.11: Chi tiết đơn hàng (Giao diện người dùng)

Hôm nay bạn muốn tìm kiếm gì? Q Tìm kiếm

Đặt lịch sửa chữa admin

Từ khóa xu hướng: Galaxy S25, Iphone 16, Galaxy S23 Ultra, Oppo Find X9

iPhone 16e
iPhone mới nhất với giá tốt nhất.
Đặt trước từ 28/02, trả hàng từ 07/03
Ưu đãi lên tới 4.000.000đ

Phụ kiện giảm TỚI 20% | Trả góp 4 KHÔNG | Thu cũ ĐỔI MỚI

Đặt trước ngay

Dành cho bạn

REDMAGIC 10 (PRO)
10 GENERATIONS, ZERO LIMITS

Thông báo

✓ Đánh dấu tất cả là đã đọc

Chưa đọc 3 | Tất cả

Cập nhật đơn hàng
Đơn hàng #40c9f706-279f-4d88-b5c1-1813f4b75a7b đã được cập nhật: Đã xác nhận - cập nhật
7 ngày trước

Cập nhật đơn hàng
Đơn hàng #c16d490b-b60c-4b4a-88eb-dffa03edc386 đã được cập nhật: Đã xác nhận - cập nhật linh kiện sử dụng
14 ngày trước

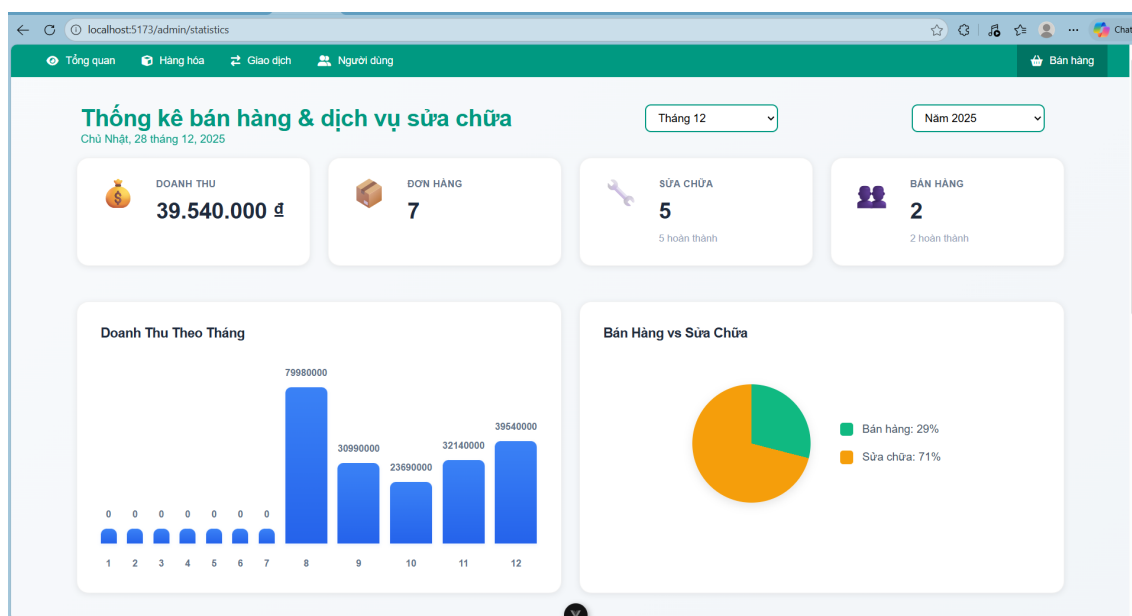
Cập nhật đơn hàng
Đơn hàng #994e8b08-ae0d-4989-b5eb-b21e6bd20b7c đã được cập nhật: Đã xác nhận - chờ xác nhận
14 ngày trước

Hình 4.12: Hệ thống thông báo trạng thái đơn hàng

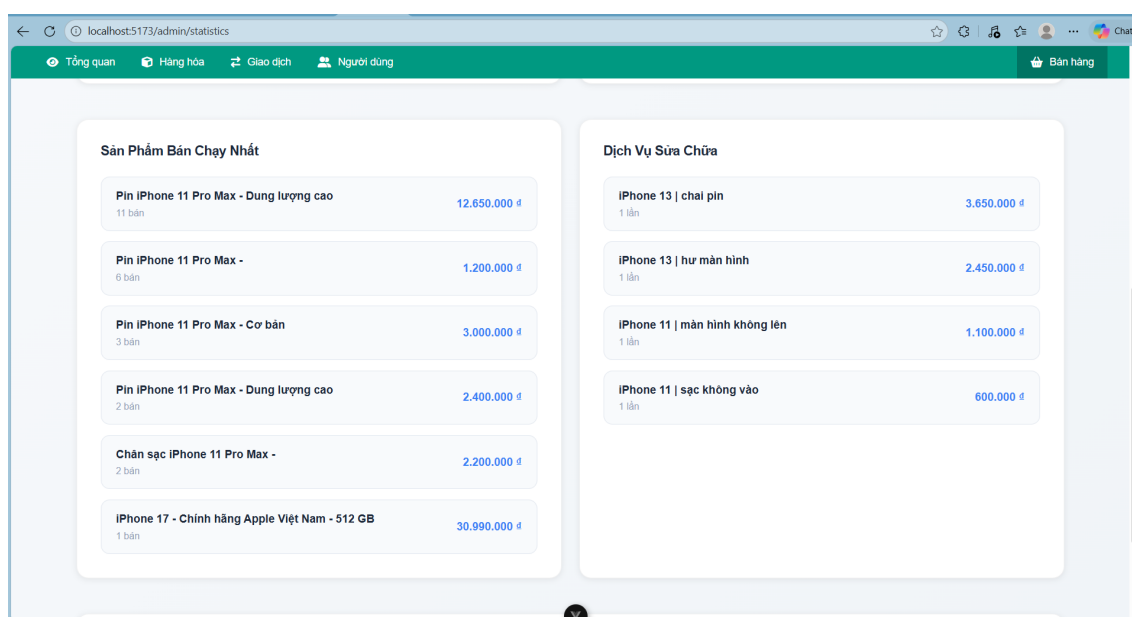
4.5.2 Giao diện Web (Quản trị viên)

Phần giao diện dành cho Admin để quản lý hệ thống, thống kê doanh thu và xử lý đơn hàng.

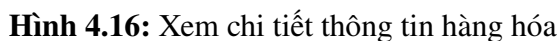
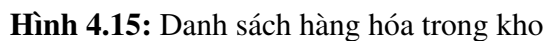
a, Thống kê (Dashboard)

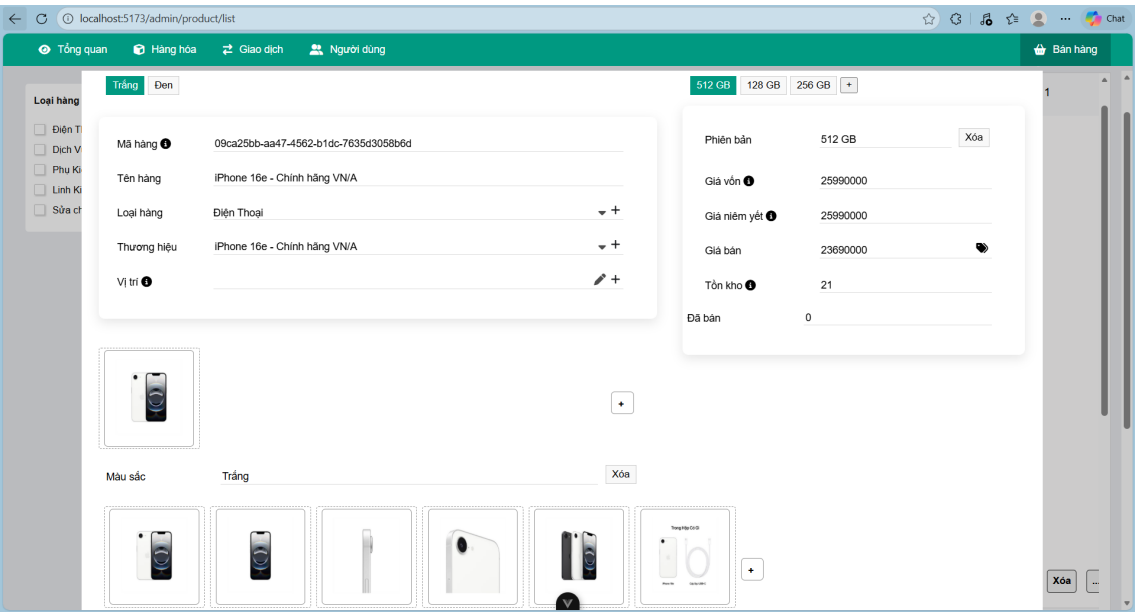


Hình 4.13: Thống kê tổng quan: Doanh thu, đơn hàng, tỷ lệ dịch vụ



Hình 4.14: Thống kê sản phẩm bán chạy và dịch vụ sửa chữa phổ biến





Hình 4.17: Giao diện Thêm mới / Cập nhật hàng hóa

c, Quản lý Đơn hàng

STT	LOẠI ĐƠN	KHÁCH HÀNG	ĐỊA CHỈ	TỔNG GIÁ	TRẠNG THÁI	THAO TÁC
1	SỬA CHỮA	Tiến Dũng	Ha Dong, Ha Noi	10.550.000 đ	Đang xử lý	CHI TIẾT
2	SỬA CHỮA	Nguyễn Thị V	Ha Dong, Ha Noi	2.450.000 đ	Hoàn thành	CHI TIẾT
3	MUA BÁN	admin	Ha Dong, Ha Noi	1.350.000 đ	Hoàn thành	CHI TIẾT
4	SỬA CHỮA	Nguyễn Trọng Đức	An Duong, Yen Phu, Tay Ho, Ha Noi	1.100.000 đ	Hoàn thành	CHI TIẾT
5	SỬA CHỮA	Nguyễn Thị V	Ha Dong, Ha Noi	3.650.000 đ	Đã xác nhận	CHI TIẾT
6	MUA BÁN	Nguyễn Trọng Đức	An Duong, Yen Phu, Tay Ho, Ha Noi	30.990.000 đ	Hoàn thành	CHI TIẾT
7	SỬA CHỮA	admin	An Duong, Yen Phu, Tay Ho, Ha Noi	0 đ	Đã xác nhận	CHI TIẾT
8	MUA BÁN	admin	Ha Dong, Ha Noi	30.990.000 đ	Hoàn thành	CHI TIẾT
9	SỬA CHỮA	Nguyễn Văn A	An Duong, Yen Phu, Tay Ho, Ha Noi	1.150.000 đ	Đã xác nhận	CHI TIẾT

Hình 4.18: Danh sách tất cả đơn hàng (Mua bán và Sửa chữa)

Chi tiết đơn hàng

LOẠI ĐƠN
Sửa chữa

TÊN KHÁCH HÀNG
Nguyễn Trọng Đức

SỐ ĐIỆN THOẠI
0123123123

ĐỊA CHỈ
An Duong, Yen Phu, Tay Ho, Ha Noi

LOẠI ĐỊA CHỈ
Cửa hàng

TRANG THÁI
Hoàn thành

MÔ TẢ
iPhone 11 | màn hình không lên

CẬP NHẬT GHI CHÚ

LỊCH SỬ CẬP NHẬT

Người thực hiện	Nội dung	Thời gian
admin	Cập nhật linh kiện sử dụng	16:56 16/12/2025

CẬP NHẬT TRẠNG THÁI

Hình 4.19: Chi tiết đơn hàng và cập nhật trạng thái đơn hàng

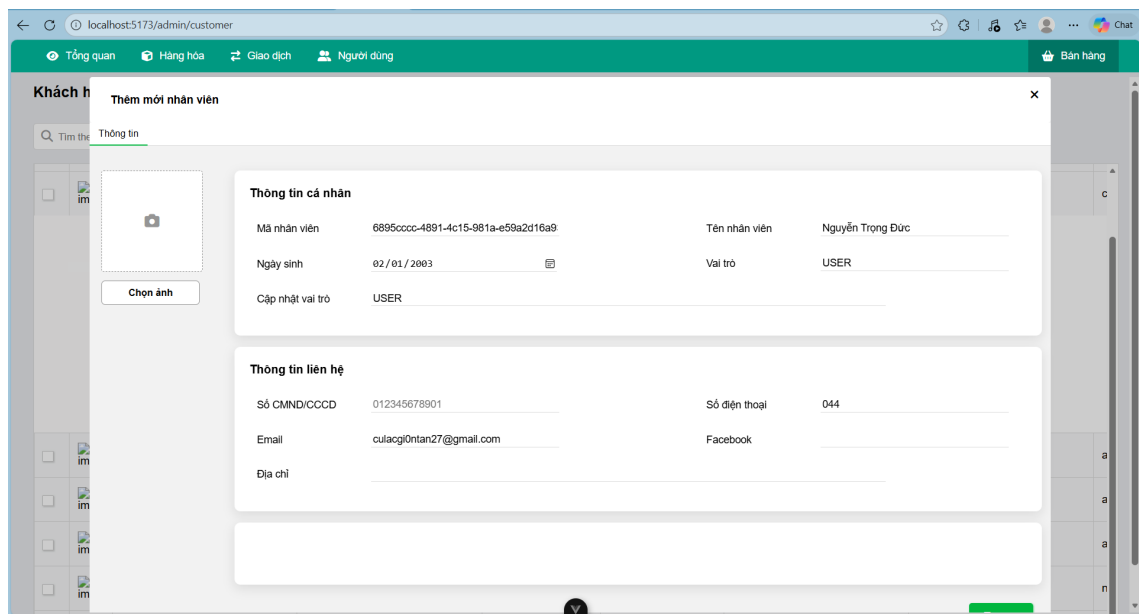
d, Quản lý Người dùng

Khách hàng

Tìm theo số điện thoại

	Mã khách hàng	Tên khách hàng	Số điện thoại	CCCD/CMND	Ngày sinh	Địa chỉ
☐	33ff2646-2922-416c-8216-c877201ed659	admin	admin	admin	2025-12-03	admin
☐	6895cccc-4891-4c15-981a-e59a2d16a939	Nguyễn Trọng Đức	044	044	2003-02-01	
☐	7caed02a-7e5b-4321-ba46-b3500ccf8589	Đặng Tiến Dũng	0986068436	0986068436		
☐	8460cfcd-ca09-4ce2-98bc-0fe5f6ba015b	Nguyễn Thị V	0333333333	0333333333		
☐	8d04670e-170d-40fe-9477-545169f62832		0222222222	0222222222		
☐	a1b2c3d4-e5f6-7890-abcde-f1234567890	Nguyễn Văn A	0912345678	0912345678	1995-05-15	Hà Nội
☐	b2c3d4e5-f6a7-8901-bcdef-12345678901	Trần Thị B	0923456789	0923456789	1998-08-20	TP.HCM
☐	d4e5f6a7-b8c9-0123-def0-234567890123	Phạm Thị D	0945678901	0945678901	1992-03-25	Hải Phòng
☐	ff5e7dc7-0c39-4b37-aba3-da4b267c88cf	Tiến Dũng	0123123123	0123123123	2003-01-01	An Duong, Yen Phu, Tay Ho, Ha Noi

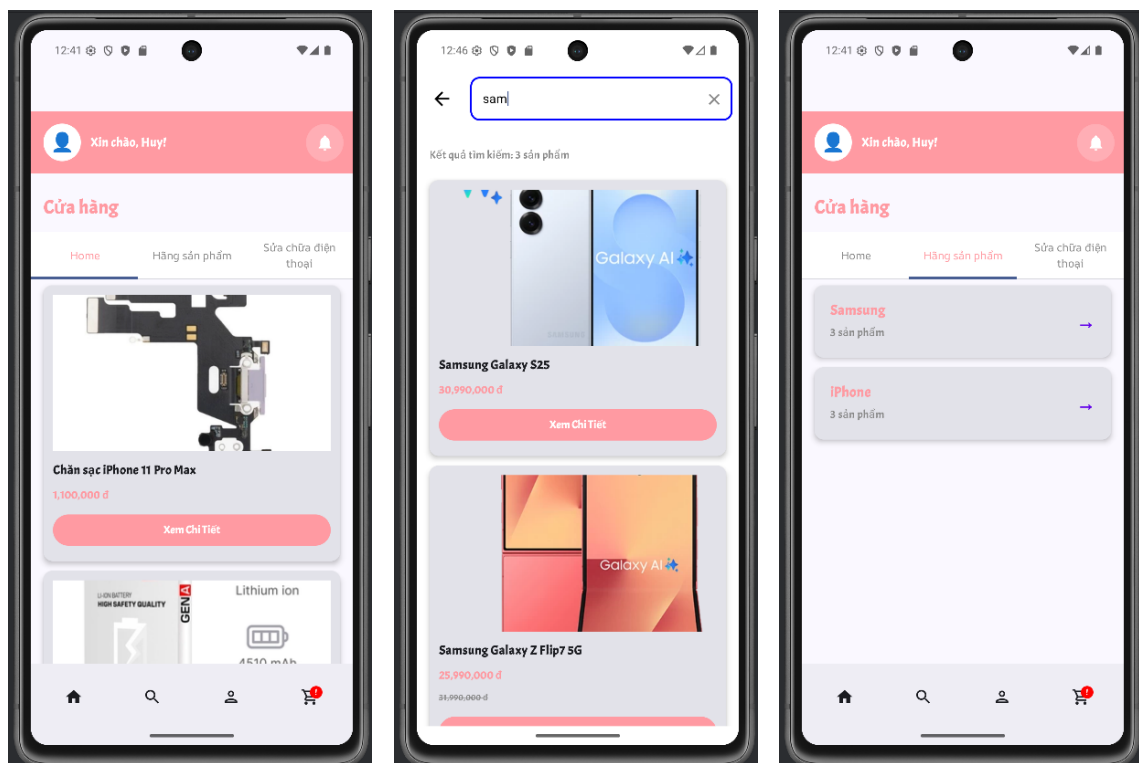
Hình 4.20: Danh sách Khách hàng và Nhân viên



Hình 4.21: Cập nhật thông tin và phân quyền người dùng

4.5.3 Giao diện Mobile App (Khách hàng)

Giao diện ứng dụng được thiết kế tối ưu cho trải nghiệm trên thiết bị di động, cho phép hiển thị nhiều thông tin theo chiều dọc. Dưới đây là các màn hình chính được nhóm theo luồng nghiệp vụ.

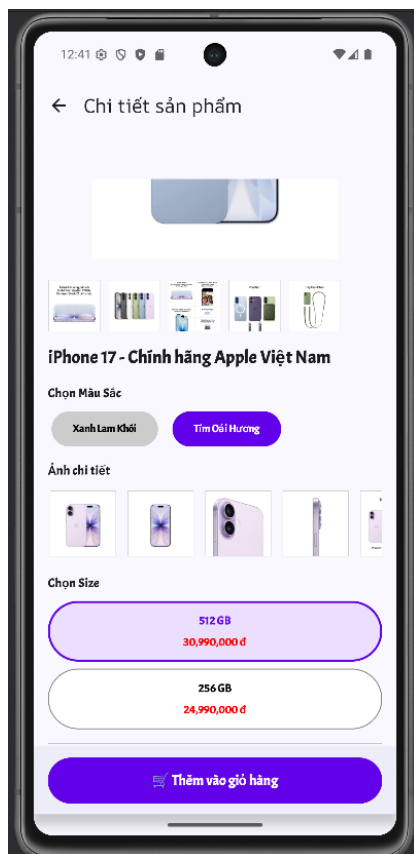


(a) Trang chủ

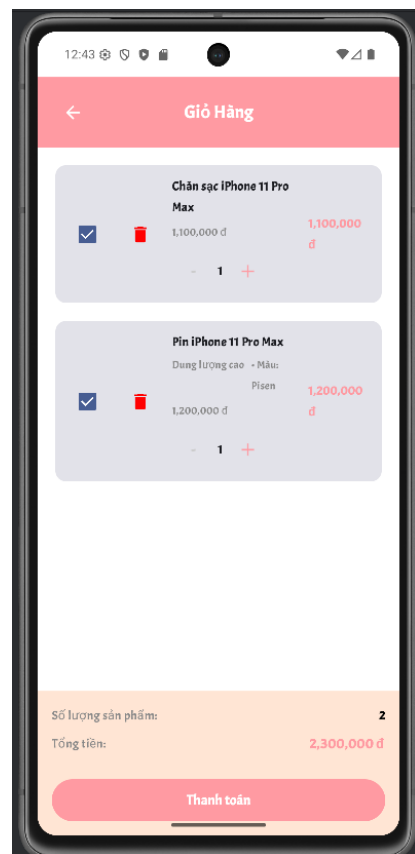
(b) Tìm kiếm

(c) Danh mục

Hình 4.22: Giao diện điều hướng và tìm kiếm sản phẩm

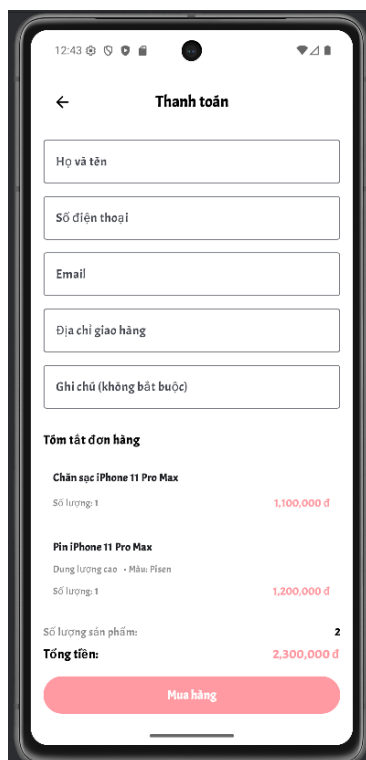


(a) Chi tiết sản phẩm & Chọn cấu hình

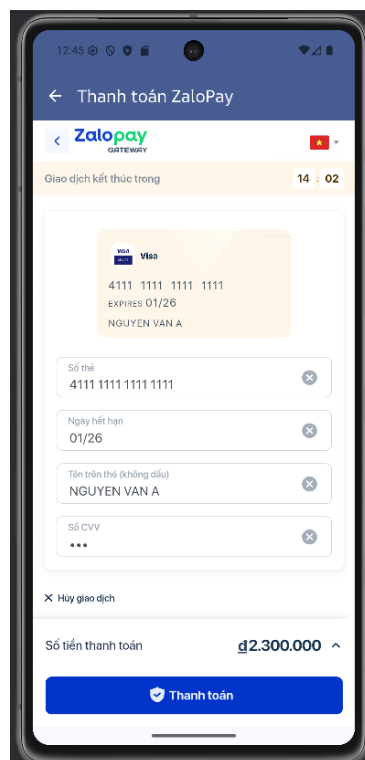


(b) Giỏ hàng & Tùy chọn số lượng

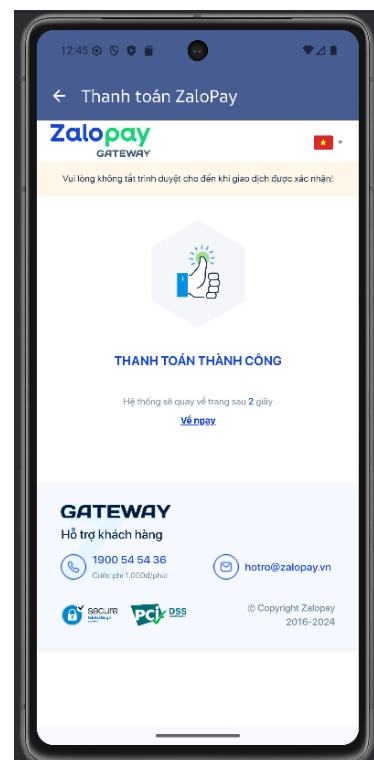
Hình 4.23: Chức năng xem chi tiết và quản lý giỏ hàng



(a) Thông tin nhận hàng

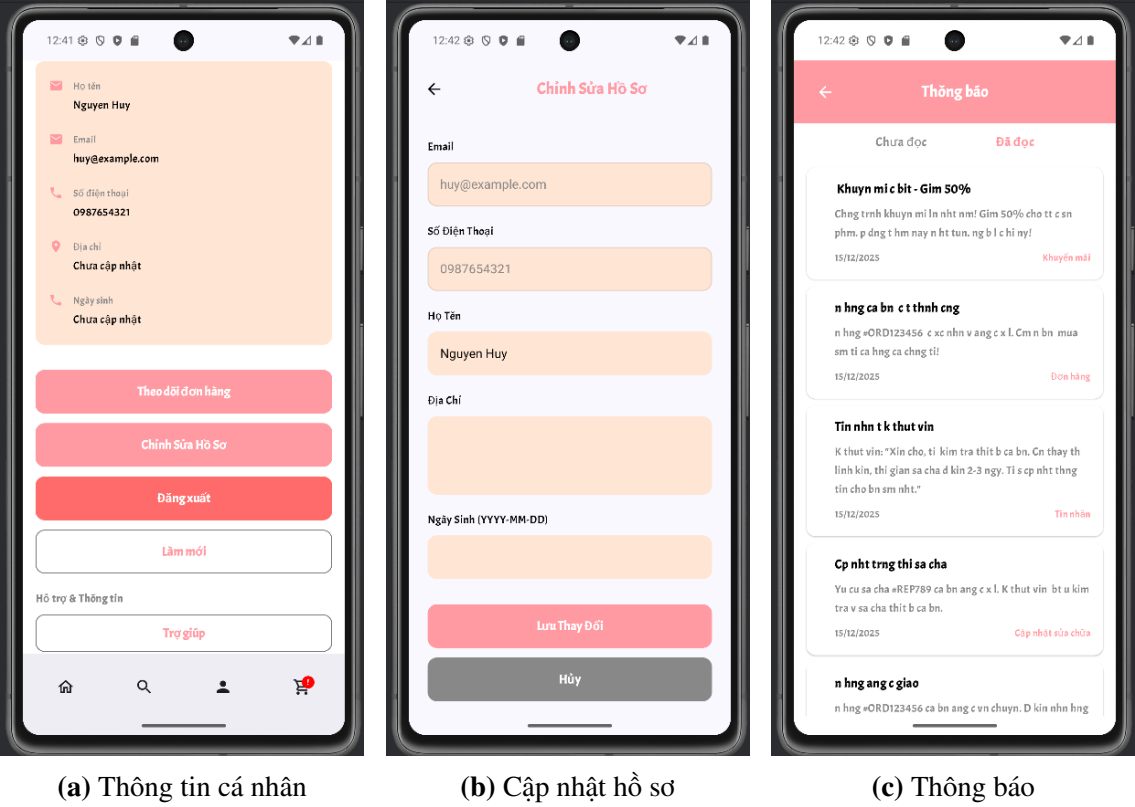


(b) Cổng ZaloPay

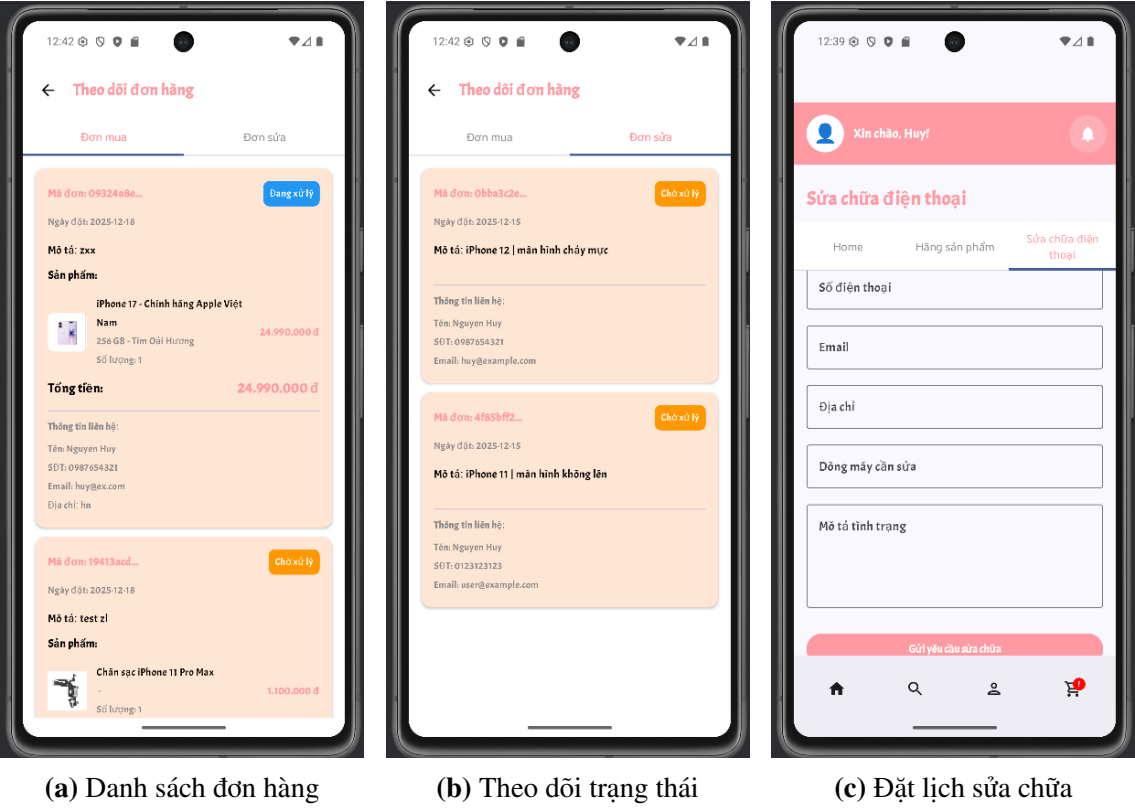


(c) Thanh toán thành công

Hình 4.24: Luồng thanh toán trực tuyến qua ví điện tử



Hình 4.25: Các chức năng quản lý tài khoản người dùng



Hình 4.26: Quản lý đơn hàng và Dịch vụ sửa chữa

4.5.4 Đánh giá kết quả

Sau quá trình thiết kế, cài đặt và kiểm thử, hệ thống quản lý bán hàng và dịch vụ sửa chữa thiết bị di động đã hoàn thành các mục tiêu ban đầu đề ra. Kết quả đạt được có thể được đánh giá chi tiết trên các khía cạnh sau:

- **Về mặt chức năng nghiệp vụ:** Hệ thống đã đáp ứng đầy đủ các quy trình nghiệp vụ cốt lõi của một cửa hàng kinh doanh thiết bị di động. Phân hệ Admin trên Web hỗ trợ đắc lực cho quản trị viên trong việc theo dõi doanh thu thông qua biểu đồ trực quan, quản lý kho hàng, và xử lý đơn hàng/đơn sửa chữa một cách chính xác. Đối với khách hàng, các chức năng tìm kiếm, đặt mua sản phẩm, và đặc biệt là quy trình đặt lịch sửa chữa thiết bị hoạt động trơn tru, logic chặt chẽ từ khâu gửi yêu cầu đến khi hoàn tất thanh toán.
- **Về hiệu năng và khả năng đồng bộ:** Hệ thống thể hiện sự ổn định cao trong quá trình vận hành. Cơ chế giao tiếp qua RESTful API đảm bảo việc đồng bộ dữ liệu giữa hai nền tảng Website và Mobile App diễn ra gần như tức thời (real-time). Khi trạng thái đơn hàng hoặc tiến độ sửa chữa được cập nhật từ phía quản trị viên, người dùng nhận được thông báo và thấy sự thay đổi ngay lập tức trên ứng dụng di động mà không gặp độ trễ đáng kể.
- **Về giao diện và trải nghiệm người dùng (UI/UX):** Giao diện được thiết kế nhất quán, hiện đại và thân thiện với người dùng. Các thao tác trên ứng dụng Mobile được tối ưu hóa cho màn hình cảm ứng, giúp người dùng dễ dàng thao tác một tay. Tốc độ tải trang và phản hồi của hệ thống nhanh, mang lại trải nghiệm mượt mà. Việc tích hợp thành công cổng thanh toán điện tử (ZaloPay) cũng là một điểm sáng, giúp hoàn thiện trải nghiệm mua sắm khép kín và tiện lợi.
- **Về tính bảo mật và mở rộng:** Hệ thống đã thiết lập tốt cơ chế xác thực và phân quyền người dùng (Authentication & Authorization), đảm bảo an toàn dữ liệu cá nhân và dữ liệu giao dịch. Cấu trúc hệ thống được tổ chức khoa học, tạo tiền đề thuận lợi cho việc bảo trì, nâng cấp hoặc mở rộng thêm các tính năng mới (như tích hợp thêm các cổng thanh toán khác, mở rộng quản lý chuỗi cửa hàng) trong tương lai.

4.6 Kết luận chương

Chương 4 đã mô tả toàn bộ quá trình hiện thực hóa hệ thống, từ việc thiết lập môi trường phát triển, cài đặt các thành phần Backend/Frontend, đến quy trình đóng gói và triển khai ứng dụng bằng Docker. Các giao diện chức năng chính đã được hoàn thiện, đảm bảo tính thẩm mỹ và trải nghiệm người dùng tốt trên cả nền

tăng Web và Mobile.

Kết quả cho thấy hệ thống hoạt động ổn định, các module giao tiếp trơn tru và đáp ứng được các yêu cầu nghiệp vụ đặt ra ban đầu. Đây là cơ sở quan trọng để khẳng định tính khả thi và ứng dụng thực tiễn của đề tài.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

- **Tóm tắt**

Đây là phần tổng kết lại toàn bộ quá trình nghiên cứu và thực hiện đồ án "Xây dựng hệ thống bán hàng và quản lý sửa chữa điện thoại". Dựa trên các kết quả triển khai thực tế đã trình bày ở các chương trước, phần này sẽ đánh giá mức độ hoàn thành so với các mục tiêu ban đầu, tổng hợp những thành tựu đạt được cũng như thẳng thắn nhìn nhận những hạn chế còn tồn tại.

Đồng thời, nhóm sẽ đề xuất các định hướng phát triển tiếp theo nhằm khắc phục các điểm yếu và nâng cấp hệ thống, đảm bảo tính khả thi và giá trị thực tiễn lâu dài của sản phẩm.

- **Kết luận chung**

Đồ án đã hoàn thành mục tiêu xây dựng hệ thống bán hàng và quản lý sửa chữa điện thoại toàn diện trên hai nền tảng Web và Mobile. Hệ thống được phát triển dựa trên kiến trúc hiện đại, sử dụng Spring Boot cho Backend, kết hợp với cơ sở dữ liệu MySQL và các công nghệ tiên tiến khác, đảm bảo hiệu năng cao và khả năng mở rộng tốt.

Các kết quả chính đạt được bao gồm:

- **Kiến trúc hệ thống vững chắc:** Xây dựng thành công hệ thống RESTful API chuẩn mực, đóng gói bằng Docker, đảm bảo tính nhất quán dữ liệu và dễ dàng triển khai trên nhiều môi trường.
- **Quản lý nghiệp vụ toàn diện:** Hệ thống đáp ứng đầy đủ các quy trình nghiệp vụ cốt lõi từ quản lý sản phẩm đa biến thể, đơn hàng, kho hàng đến quy trình tiếp nhận và sửa chữa thiết bị. Module quản lý lịch làm việc giúp tối ưu hóa nhân sự cho cửa hàng.
- **Bảo mật và phân quyền:** Triển khai thành công cơ chế xác thực JWT kết hợp OAuth2 Resource Server. Hệ thống phân quyền chi tiết (RBAC) đến từng chức năng, đảm bảo an toàn dữ liệu và kiểm soát chặt chẽ quyền truy cập của các nhóm người dùng (Admin, Manager, User).
- **Tích hợp thanh toán và thông báo:** Tích hợp thành công cổng thanh toán phổ biến ZaloPay với quy trình xử lý giao dịch tự động. Hệ thống thông báo đa kênh (Real-time WebSocket, Firebase Cloud Messaging) giúp tương tác tức thời với người dùng.
- **Trải nghiệm người dùng:** Cung cấp giao diện thân thiện cho cả khách hàng

và quản trị viên, hỗ trợ các tính năng tiện ích như tìm kiếm, lọc sản phẩm và theo dõi trạng thái đơn hàng/sửa chữa theo thời gian thực.

- **Những hạn chế còn tồn tại**

Mặc dù đã đáp ứng được các yêu cầu cơ bản và nâng cao của đề tài, hệ thống vẫn còn một số hạn chế cần khắc phục:

- **Thiếu các tính năng thông minh (AI/ML):** Hệ thống chưa tích hợp trí tuệ nhân tạo để gợi ý sản phẩm cá nhân hóa, dự báo nhu cầu nhập hàng hay Chatbot hỗ trợ tự động.
- **Tự động hóa chưa triệt để:** Các quy trình liên kết với đơn vị vận chuyển (tạo vận đơn, tra cứu hành trình) và các chiến dịch Marketing (Email, SMS) vẫn còn thực hiện thủ công hoặc chưa được tích hợp sâu.
- **Hạn chế về báo cáo - thống kê:** Các tính năng báo cáo doanh thu và hiệu quả kinh doanh mới dừng lại ở mức cơ bản, chưa có các công cụ Business Intelligence (BI) để phân tích sâu hành vi khách hàng và xu hướng thị trường.

- **Hướng phát triển trong tương lai**

Để nâng cao giá trị thực tiễn và tính cạnh tranh của sản phẩm, hướng phát triển tiếp theo của đề tài sẽ tập trung vào các nội dung sau:

1. **Tích hợp Trí tuệ nhân tạo (AI):**

- Xây dựng hệ thống gợi ý sản phẩm (Recommendation System) dựa trên lịch sử mua hàng và hành vi người dùng.
- Phát triển Chatbot thông minh hỗ trợ tư vấn và giải đáp thắc mắc khách hàng 24/7.
- Ứng dụng AI để dự báo nhu cầu linh kiện sửa chữa, giúp tối ưu hóa tồn kho.

2. **Nâng cao trải nghiệm và Marketing:**

- Phát triển ứng dụng theo tiêu chuẩn Progressive Web App (PWA) để hỗ trợ truy cập offline và cài đặt như ứng dụng Native.
- Xây dựng hệ thống khách hàng thân thiết (Loyalty), tích điểm đổi quà và các chương trình khuyến mãi (Voucher, Flash Sale) tự động.

3. **Mở rộng tích hợp và Tự động hóa:**

- Kết nối API với các đơn vị vận chuyển lớn (GHN, Viettel Post) để tự động hóa quy trình giao nhận.
- Tích hợp thêm các cổng thanh toán quốc tế (Stripe, PayPal) và hỗ trợ đa

tiền tệ.

4. Tối ưu hiệu năng và Hạ tầng:

- Chuyển đổi kiến trúc sang Microservices hoàn chỉnh để tăng khả năng mở rộng độc lập từng module.
- Ứng dụng Caching (Redis) và CDN để tăng tốc độ tải trang và chịu tải tốt hơn khi lượng người dùng tăng cao.

• Kết luận

Tóm lại, đồ án đã giải quyết thành công bài toán quản lý bán hàng và sửa chữa điện thoại thông qua một giải pháp công nghệ hiện đại và toàn diện. Mặc dù vẫn còn những hạn chế nhất định, nhưng hệ thống đã đáp ứng tốt các yêu cầu cốt lõi và tạo ra một nền tảng vững chắc cho việc mở rộng sau này.

Quá trình thực hiện đồ án không chỉ mang lại sản phẩm thực tế mà còn giúp nhóm em tích lũy được nhiều kiến thức và kinh nghiệm quý báu trong quy trình phát triển phần mềm chuyên nghiệp, từ khâu phân tích, thiết kế, lập trình đến triển khai và vận dụng vào thực tiễn cuộc sống. Đây là hành trang quan trọng để nhóm em, những kỹ sư công nghệ thông tin tương lai, phát triển thêm nhiều dự án chất lượng hơn sau này.

Xin cảm ơn các thầy/cô đã đọc bài báo cáo của nhóm. Mong nhận được các góp ý từ thầy/cô.

Xin hết./.

TÀI LIỆU THAM KHẢO

- [1] C. Walls, *Spring Boot in action*. Manning Publications, 2016.
- [2] R. Johnson, *Expert One-on-One J2EE Design and Development*. Wrox, 2004.
- [3] C. Macrae, *Vue.js: Up and Running: Building Accessible and Performant Web Apps*. " O'Reilly Media, Inc.", 2018.
- [4] S. E. Peyrott, *The JSON Web Token Handbook*. Auth0 Inc, 2016.
- [5] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Origin of RESTful API concepts, Ph.D. dissertation, University of California, Irvine, 2000.
- [6] M. Fowler, *UML distilled: a brief guide to the standard object modeling language*. Addison-Wesley Professional, 2004.
- [7] R. Elmasri and S. B. Navathe, *Fundamentals of database systems*. Pearson, 2015.
- [8] ZaloPay, *Zalopay gateway integration documentation*, <https://docs.zalopay.vn/>, Accessed: 2025-12-30, 2024.
- [9] Google Developers, *Android developers documentation*, <https://developer.android.com/docs>, Accessed: 2025-12-30, 2024.